

**BSc Project**

CS Department

Project ID: UQU-CSAI-471-P2-F35

June 2026



A PLATFORM FOR CHILDREN'S STORIES

# Gheras: A Web Platform for Generating Customized Children's Stories Using Artificial Intelligence for Reinforcing Good Behaviors in Children

**Department of Computer Science and Artificial Intelligence**

College of Computing · Umm Al-Qura University

Academic Year 2025-2026

## CONTACT INFORMATION

This project report is submitted to the Department of Computer Science and Artificial Intelligence at Umm Al-Qura University in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science / Artificial Intelligence.

---

### DEPARTMENT

**Analysis and Design**  
College of Computing  
Umm Al-Qura University

### CONTACT

WEB [uqu.edu.sa](http://uqu.edu.sa)  
TEL +966 12 542 6222

# **INTELLECTUAL PROPERTY RIGHT DECLARATION**

This is to declare that the work under the supervision of Dr. Arwa Alsubhi having title “ Ghera: A web platform for generating customized children’s stories using artificial intelligence to treat bad behaviours in children” carried out in partial fulfilment of the requirements of Bachelor of Science in computer, and Artificial Intelligence is the sole property of the Umm Al Qura University and the respective supervisor and is protected under the intellectual property right laws and conventions. It can only be considered/ used for purposes like extension for further enhancement, product development, adoption for commercial/organizational usage, etc., with the permission of the University and respective supervisor.

The above statement applies to all students and faculty members.

Date: 1 June 2026

## **ANTI-PLAGIARISM DECLARATION**

This is to declare that the work under the supervision of Dr. Arwa Alsubhi having title “ Ghera: A web platform for generating customized children’s stories using artificial intelligence to treat bad behaviours in children” carried out in partial fulfilment of the requirements of Bachelor of Science in computer, and Artificial Intelligence is the sole property of the Umm Al Qura University and the respective supervisor and is protected under the intellectual property right laws and conventions. It can only be considered/ used for purposes like extension for further enhancement, product development, adoption for commercial/organizational usage, etc., with the permission of the University and respective supervisor.

The above statement applies to all students and faculty members.

Date: 1 June 2026



## **ACKNOWLEDGMENTS**

First, we thank Almighty Allah for granting us success and facilitating the completion of this report. Second, we express our sincere gratitude to our supervisor, Dr. Arwa Alsubhi, for her guidance, suggestions, and assistance during our work on the project. Third, we thank the project team members for their efforts and cooperation. Finally, we would like to thank our family and friends for their support, as well as everyone who helped us throughout this project.

## ABSTRACT

The rapid advancement of generative Artificial Intelligence (AI) has created new opportunities to develop intelligent, personalized educational systems that support both learning and behavioral development. Gheras is an AI-based web platform that generates personalized, culturally appropriate stories for children aged 4–9, aiming to address specific behavioral challenges through storytelling. The system integrates multiple artificial intelligence services, including large language models (LLMs) for text story generation, AI-based image generation for illustrations, and text-to-speech synthesis for audio narration, enabling the creation of multimodal educational stories in which the child becomes the central character. The implemented platform uses a modular, service-oriented architecture, with Django REST Framework for backend development and React for frontend interaction. Story personalization is achieved through the integration of child-specific attributes such as name, age group, gender, and appearance characteristics, in addition to parent-selected behavioral objectives, story genres, and illustration styles. To improve linguistic accessibility for young Arabic readers, the generated stories incorporate Arabic diacritization (tashkeel). The platform also emphasizes alignment with cultural and religious norms by generating content grounded in Saudi cultural contexts and Islamic values.

The implemented system supports AI-driven story and illustration generation, audio narration, PDF story export, and progress tracking across a unified platform. Experimental evaluation and system testing demonstrated that the platform provides coherent personalized stories, multimodal engagement, and behavioral guidance while maintaining acceptable usability and system performance. Furthermore, the platform contributes to the field of AI-driven educational software by introducing a structured framework for culturally adaptive, behavior-oriented storytelling that combines educational objectives with modern generative AI technologies.

Furthermore, the platform contributes to the field of AI-driven behavioral modification systems by introducing a structured framework for culturally adaptive, behavior-oriented storytelling that combines behavioral guidance and child-centered interaction with modern generative AI technologies.

**Keywords:** Generative AI, Personalized Storytelling, Arabic Children’s Stories, Behavioral Development, Multimodal Storytelling

# TABLE OF CONTENTS

|                                                   |      |
|---------------------------------------------------|------|
| CONTACT INFORMATION .....                         | i    |
| INTELLECTUAL PROPERTY RIGHT DECLARATION .....     | ii   |
| ANTI-PLAGIARISM DECLARATION .....                 | iii  |
| ACKNOWLEDGMENTS .....                             | iv   |
| ABSTRACT .....                                    | v    |
| LIST OF ACRONYMS .....                            | xiii |
| 1 Introduction and Motivation .....               | 1    |
| 1.1 Purpose of the Project .....                  | 1    |
| 1.2 Purpose of this Document .....                | 1    |
| 1.3 Project Scope .....                           | 1    |
| 1.4 Motivation .....                              | 2    |
| 1.5 Overview of this Document .....               | 2    |
| 1.6 Existing Systems .....                        | 2    |
| 1.6.1 Existing systems description .....          | 2    |
| 1.6.2 Problems in the existing systems .....      | 5    |
| 2 System Analysis .....                           | 7    |
| 2.1 Data Analysis .....                           | 7    |
| 2.1.1 Use case diagram .....                      | 7    |
| 2.1.2 Data Flow Diagrams .....                    | 11   |
| 2.1.3 System Requirements .....                   | 13   |
| 2.1.4 Class Diagram .....                         | 17   |
| 2.2 Proposed solution .....                       | 18   |
| 2.3 Alternative solutions .....                   | 18   |
| 3 System Considerations .....                     | 20   |
| 3.1 Design Constraints .....                      | 20   |
| 3.1.1 Hardware and software environment .....     | 20   |
| 3.1.2 End user characteristics .....              | 20   |
| 3.2 Architectural Strategies .....                | 20   |
| 3.2.1 Algorithm/models to be used .....           | 20   |
| 3.2.2 Reuse of existing software components ..... | 21   |
| 3.2.3 Project management strategies .....         | 21   |
| 3.2.4 Development method .....                    | 23   |
| 4 System Design .....                             | 24   |
| 4.1 Detailed System Design .....                  | 24   |
| 4.1.1 Sequence Diagram .....                      | 24   |
| 4.1.2 Activity Diagram .....                      | 30   |
| 4.2 Data Storage Design .....                     | 37   |
| 4.2.1 Entity Relationships Diagram .....          | 37   |
| 4.3 Architectural Design .....                    | 38   |
| 4.3.1 Model View Controller (MVC) .....           | 38   |
| 4.4 Interface design .....                        | 39   |
| 4.4.1 Windows Navigation Diagram .....            | 39   |
| 4.4.2 User Interface Prototype .....              | 40   |
| 5 Implementation .....                            | 50   |
| 5.1 Detailed System Implementation .....          | 50   |
| 5.1.1 Backend Implementation .....                | 50   |
| 5.1.2 Frontend Implementation .....               | 52   |

|                                                       |     |
|-------------------------------------------------------|-----|
| 5.2 Algorithms and Models .....                       | 54  |
| 5.2.1 Algorithms .....                                | 54  |
| 5.2.2 Models .....                                    | 55  |
| 5.2.3 Model Evaluation and Selection .....            | 57  |
| 5.3 System Components and Integration .....           | 83  |
| 5.3.1 Frontend–Backend Integration .....              | 83  |
| 5.3.2 Backend–Database Integration .....              | 84  |
| 5.3.3 Backend–AI Services Integration .....           | 84  |
| 5.4 Tools, Frameworks, and Libraries .....            | 84  |
| 5.5 Development Challenges .....                      | 86  |
| 5.5.1 Background Thread Database Connections .....    | 86  |
| 5.5.2 RTL Layout .....                                | 86  |
| 5.5.3 Multi-Step Form State .....                     | 86  |
| 5.5.4 Polling and Timeout Coordination .....          | 86  |
| 5.5.5 Static Files in Production vs Development ..... | 86  |
| 5.5.6 Sequential Generation Latency .....             | 86  |
| 5.5.7 Progressive Story Loading .....                 | 87  |
| 5.5.8 Character Consistency Across Scenes .....       | 87  |
| 5.5.9 Arabic Diacritics Quality .....                 | 87  |
| 6 Testing and Evaluation .....                        | 88  |
| 6.1 Testing Strategy and Scenarios .....              | 88  |
| 6.1.1 API Testing with curl .....                     | 88  |
| 6.1.2 Frontend Unit Tests .....                       | 88  |
| 6.1.3 Backend Unit Tests .....                        | 89  |
| 6.1.4 End-to-End Test Scenarios .....                 | 90  |
| 6.1.5 TypeScript Type Checking .....                  | 91  |
| 6.1.6 AI Pipeline Testing .....                       | 92  |
| 6.2 Test Cases and Input Configurations .....         | 94  |
| 6.2.1 Story Generation Pipeline .....                 | 94  |
| 6.2.2 Reference Data .....                            | 94  |
| 6.3 Evaluation Metrics .....                          | 94  |
| 6.4 Performance Benchmarks .....                      | 96  |
| 6.4.1 API Response Times .....                        | 96  |
| 6.4.2 Story Generation Time .....                     | 96  |
| 6.4.3 Polling Efficiency .....                        | 96  |
| 6.4.4 Frontend Build .....                            | 97  |
| 6.5 Reliability and Usability Evaluation .....        | 97  |
| 6.5.1 Mobile Responsiveness .....                     | 97  |
| 6.5.2 Error Handling .....                            | 97  |
| 6.5.3 Security .....                                  | 97  |
| 7 Results and Analysis .....                          | 98  |
| 7.1 Quantitative Performance Results .....            | 98  |
| 7.2 Qualitative Observations .....                    | 101 |
| 7.2.1 User Experience .....                           | 101 |
| 7.2.2 Arabic Content .....                            | 102 |
| 7.2.3 Onboarding .....                                | 102 |
| 7.2.4 Arabic Story Quality .....                      | 102 |
| 7.2.5 Image-Text Alignment .....                      | 103 |

|                                                           |     |
|-----------------------------------------------------------|-----|
| 7.2.6 Character Visual Consistency .....                  | 103 |
| 7.2.7 Art-Style Consistency .....                         | 103 |
| 7.2.8 Audio Narration Clarity .....                       | 103 |
| 7.3 Error Analysis .....                                  | 104 |
| 7.3.1 Known Issues .....                                  | 104 |
| 7.4 Comparison with Similar Systems .....                 | 104 |
| 7.5 Discussion of Limitations .....                       | 105 |
| 7.5.1 Dependence on External APIs .....                   | 105 |
| 7.5.2 Scalability and Platform Limitations .....          | 105 |
| 7.5.3 Offline Accessibility Limitation .....              | 105 |
| 8 Contribution and Impact .....                           | 106 |
| 8.1 Novel Features and Capabilities .....                 | 106 |
| 8.1.1 Improvements Over Existing Solutions .....          | 106 |
| 8.2 Practical and Industrial Relevance .....              | 107 |
| 8.3 Lessons Learned .....                                 | 107 |
| 8.3.1 Technical Lessons .....                             | 107 |
| 8.3.2 Project Management Lessons .....                    | 108 |
| 9 Conclusion and Future Work .....                        | 109 |
| 9.1 Project Outcomes .....                                | 109 |
| 9.2 Achievement of Objectives .....                       | 109 |
| 9.3 Future Extensions .....                               | 111 |
| 9.3.1 Backend Scalability and Task Management .....       | 111 |
| 9.3.2 Mobile Application Development .....                | 111 |
| 9.3.3 Story and Content Enhancements .....                | 112 |
| 9.3.4 Multilingual and Adaptive Educational Support ..... | 112 |
| 9.3.5 Advanced Analytics and Behavioral Insights .....    | 112 |
| 9.3.6 Educational and Behavioral Collaboration .....      | 112 |
| 9.3.7 Interactive Voice-Based Story Engagement .....      | 112 |
| 9.4 Deployment, Scalability, and Sustainability .....     | 112 |
| 9.4.1 Deployment .....                                    | 112 |
| 9.4.2 Scalability .....                                   | 113 |
| 9.4.3 Sustainability .....                                | 113 |
| 9.5 Conclusion .....                                      | 113 |
| References .....                                          | 114 |
| APPENDICES .....                                          | 116 |
| Project Links .....                                       | 116 |
| Supplementary Materials .....                             | 116 |

# LIST OF FIGURES

|             |                                                                                                                                                 |    |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 2.1  | System Use Case Diagram .....                                                                                                                   | 7  |
| Figure 2.2  | Context diagram .....                                                                                                                           | 11 |
| Figure 2.3  | DFD level 1 .....                                                                                                                               | 12 |
| Figure 2.4  | Class Diagram for Gheras platform .....                                                                                                         | 17 |
| Figure 3.1  | Research and Documentation Timeline (Gantt Chart) .....                                                                                         | 22 |
| Figure 3.2  | Gheras Platform Implementation Timeline (Gantt Chart) .....                                                                                     | 23 |
| Figure 4.1  | Sequence Diagram – Registration/login .....                                                                                                     | 25 |
| Figure 4.2  | Sequence Diagram – child profile management .....                                                                                               | 26 |
| Figure 4.3  | Sequence Diagram – Story Preference Selection .....                                                                                             | 27 |
| Figure 4.4  | Sequence Diagram – Story Generation .....                                                                                                       | 27 |
| Figure 4.5  | Sequence Diagram – Story Generation Status Tracking .....                                                                                       | 28 |
| Figure 4.6  | Sequence Diagram – Reading a Generated Story .....                                                                                              | 29 |
| Figure 4.7  | Sequence Diagram – Story PDF Export .....                                                                                                       | 29 |
| Figure 4.8  | Sequence Diagram – Progress Tracking and Analytics .....                                                                                        | 30 |
| Figure 4.9  | Child Profile Management Workflow .....                                                                                                         | 31 |
| Figure 4.10 | Parent Story Generation Workflow in the Gheras Platform .....                                                                                   | 32 |
| Figure 4.11 | Child Story Viewing and Interaction Workflow .....                                                                                              | 33 |
| Figure 4.12 | Progress Tracking and Behavioral Analytics Workflow .....                                                                                       | 34 |
| Figure 4.13 | Story PDF Download Workflow .....                                                                                                               | 35 |
| Figure 4.14 | Administrative Configuration Management Workflow .....                                                                                          | 36 |
| Figure 4.15 | conceptual ER Diagram .....                                                                                                                     | 37 |
| Figure 4.16 | Model–View–Controller (MVC) Architecture .....                                                                                                  | 38 |
| Figure 4.17 | Windows Navigation Diagram .....                                                                                                                | 39 |
| Figure 4.18 | Authentication user interface mockup – Sign-Up and Login screens .....                                                                          | 40 |
| Figure 4.19 | Dashboard user interface mockup .....                                                                                                           | 41 |
| Figure 4.20 | Child management user interface mockup .....                                                                                                    | 42 |
| Figure 4.21 | Create child profile user interface mockup .....                                                                                                | 43 |
| Figure 4.22 | Story creation – Genre selection (Step 1) .....                                                                                                 | 44 |
| Figure 4.23 | Story creation – Artistic style selection (Step 2) .....                                                                                        | 45 |
| Figure 4.24 | Story creation – Target behavior selection (Step 3) .....                                                                                       | 46 |
| Figure 4.25 | Story-reading user interface mockup .....                                                                                                       | 47 |
| Figure 4.26 | Story library user interface mockup .....                                                                                                       | 48 |
| Figure 4.27 | Progress-tracking user interface mockup .....                                                                                                   | 49 |
| Figure 5.1  | Django administration panel showing application records, configuration entities,<br>authentication tokens, and user-management components. .... | 51 |
| Figure 5.2  | Story creation wizard showing the child selector and the three-step flow. ....                                                                  | 53 |
| Figure 5.3  | Art style selection step with sample illustrations for each style. ....                                                                         | 53 |
| Figure 5.4  | Library page listing the created stories with search and filters. ....                                                                          | 54 |
| Figure 5.5  | Generated Arabic story output sample produced by GPT-4o mini. ....                                                                              | 59 |
| Figure 5.6  | Generated Arabic story output sample produced by Jais-2-8B-Chat. ....                                                                           | 59 |
| Figure 5.7  | Generated Arabic story output sample produced by Qwen3-Coder-Next. ....                                                                         | 60 |
| Figure 5.8  | Generated Arabic story output sample produced by LFM2.5-1.2B-Instruct. ....                                                                     | 61 |
| Figure 5.9  | Lexical diversity comparison using Type-Token Ratio (TTR). ....                                                                                 | 62 |
| Figure 5.10 | BERTScore comparison across text generation models. ....                                                                                        | 63 |
| Figure 5.11 | Unified prompt used for evaluating text-to-image models under controlled conditions. ....                                                       | 65 |
| Figure 5.12 | Reference child image used to support character consistency and cultural representation<br>during image-generation evaluation. ....             | 66 |

|             |                                                                                                                                                                                                                                 |     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Figure 5.13 | Comparative illustration outputs generated by Z-Image-Turbo, Flux.1 [schnell], GPT-Image-1-mini, and Flux.2 Max. ....                                                                                                           | 67  |
| Figure 5.14 | Semantic evaluation of generated illustrations using Multilingual-CLIP adherence scores. ....                                                                                                                                   | 68  |
| Figure 5.15 | Consistency evaluation of generated illustrations using character preservation similarity. ....                                                                                                                                 | 69  |
| Figure 5.16 | Visual evaluation of generated illustrations using LAION-Aesthetics scores. ....                                                                                                                                                | 69  |
| Figure 5.17 | LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model. ....                                                                                                                                           | 70  |
| Figure 5.18 | LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model. ....                                                                                                                                           | 71  |
| Figure 5.19 | LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model. ....                                                                                                                                           | 72  |
| Figure 5.20 | LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model. ....                                                                                                                                           | 73  |
| Figure 5.21 | Naturalness MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS. ....                                                                                                                                                      | 76  |
| Figure 5.22 | Intelligibility MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS. ....                                                                                                                                                  | 76  |
| Figure 5.23 | Expressiveness & Prosody MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS. ....                                                                                                                                         | 77  |
| Figure 5.24 | Overall MOS comparison across sub-metrics: GPT-4o-mini-TTS versus OmniVoice TTS. ....                                                                                                                                           | 78  |
| Figure 5.25 | MOSNet predicted MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS. ....                                                                                                                                                 | 79  |
| Figure 5.26 | NISQA multidimensional quality comparison between GPT-4o-mini-TTS and OmniVoice TTS. ....                                                                                                                                       | 80  |
| Figure 5.27 | Acoustic feature comparison: F0 mean, F0 standard deviation, and voiced frame ratio. ....                                                                                                                                       | 81  |
| Figure 7.1  | Runtime benchmark output for the 4–5 age-group multimodal story-generation pipeline, showing LLM response time, image-generation latency, TTS latency, first-viewable readiness time, and total end-to-end execution time. .... | 100 |
| Figure 7.2  | Runtime benchmark output for the 6–7 age-group multimodal story-generation pipeline, illustrating the effect of increased scene count on image-generation workload and total pipeline latency. ....                             | 100 |
| Figure 7.3  | Runtime benchmark output for the 8–9 age-group multimodal story-generation pipeline, demonstrating the highest overall latency due to expanded narrative complexity and additional generated multimedia assets. ....            | 101 |
| Figure 7.4  | Mobile-responsive story-viewer interface during multimodal story reading. ....                                                                                                                                                  | 102 |
| Figure 7.5  | Progressive-loading interface displayed during asynchronous multimodal story generation. ....                                                                                                                                   | 102 |
| Figure 7.6  | Consecutive generated scenes demonstrating relative character consistency across the multimodal storytelling pipeline. ....                                                                                                     | 103 |
| Figure 7.7  | Comparison of generated illustrations across some supported visual styles within the Gheras platform. ....                                                                                                                      | 103 |

## LIST OF TABLES

|            |                                                                                                                                                                                               |    |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Table 1.1  | A comparison table evaluates the proposed solution “Gheras” against traditional, digital, and existing AI-based storytelling systems based on key features . . . . .                          | 5  |
| Table 2.1  | Use Case Scenario for Sign Up . . . . .                                                                                                                                                       | 8  |
| Table 2.2  | Use Case Scenario for Login . . . . .                                                                                                                                                         | 8  |
| Table 2.3  | Use Case Scenario for Create Child Profile . . . . .                                                                                                                                          | 8  |
| Table 2.4  | Use Case Scenario for Select Behavior & Story Preferences. Available preferences are predefined and managed by the system administrator to ensure controlled personalization. . . . .         | 9  |
| Table 2.5  | Use Case Scenario for Generate Personalized Story. Story content is generated dynamically using artificial intelligence models based on the child’s profile and selected preferences. . . . . | 9  |
| Table 2.6  | Use Case Scenario for Track Child Progress. Progress metrics support monitoring of learning outcomes and behavioral development over time. . . . .                                            | 9  |
| Table 2.7  | Use Case Scenario for View & Read Story. Interaction data is recorded to support engagement analytics and progress tracking. . . . .                                                          | 9  |
| Table 2.8  | Use Case Scenario for Download Story PDF. Download functionality enables offline access to generated educational content. . . . .                                                             | 10 |
| Table 2.9  | Use Case Scenario for Manage User Accounts & Access. Administrative controls enforce secure authentication, authorization, and role-based access. . . . .                                     | 10 |
| Table 2.10 | Use Case Scenario for Manage Story Configuration Settings. Configuration settings act as controlled inputs for AI-driven story personalization. . . . .                                       | 10 |
| Table 2.11 | Use Case Scenario for Monitor System & AI Models. Monitoring applies to externally integrated AI services. . . . .                                                                            | 10 |
| Table 2.12 | Use Case Scenario for Database Management. Database administration ensures data integrity, security, and system performance. . . . .                                                          | 11 |
| Table 5.1  | GPT-4o-mini invocation parameters. . . . .                                                                                                                                                    | 55 |
| Table 5.2  | Structured prompt components used for image generation. . . . .                                                                                                                               | 55 |
| Table 5.3  | Age-adaptive scene count and token budget mapping. . . . .                                                                                                                                    | 56 |
| Table 5.4  | Comprehensive human-centric evaluation scores for the text generation models. . . . .                                                                                                         | 61 |
| Table 5.5  | Quantitative evaluation of text generation models using latency and diacritic density. . . . .                                                                                                | 61 |
| Table 5.6  | LLM-as-a-Judge evaluation summary for text generation models. . . . .                                                                                                                         | 64 |
| Table 5.7  | Average latency comparison for candidate image generation models. . . . .                                                                                                                     | 73 |
| Table 5.8  | Naturalness MOS evaluation results (E1–E3: representative evaluator scores). . . . .                                                                                                          | 75 |
| Table 5.9  | Intelligibility MOS evaluation results (E1–E3: representative evaluator scores). . . . .                                                                                                      | 76 |
| Table 5.10 | Expressiveness & Prosody MOS evaluation results (E1–E3: representative evaluator scores). . . . .                                                                                             | 77 |
| Table 5.11 | Comprehensive MOS evaluation summary with weighted composite score. . . . .                                                                                                                   | 78 |
| Table 5.12 | MOSNet predicted speech quality scores and corresponding ITU-T P.800 quality tiers. . . . .                                                                                                   | 79 |
| Table 5.13 | NISQA multidimensional predicted speech quality scores across five perceptual dimensions. . . . .                                                                                             | 80 |
| Table 5.14 | Comparative acoustic feature analysis extracted from GPT-4o-mini-TTS and OmniVoice TTS outputs. . . . .                                                                                       | 81 |
| Table 5.15 | Comprehensive comparative evaluation summary across all assessment dimensions. . . . .                                                                                                        | 82 |
| Table 5.16 | Backend tools and libraries . . . . .                                                                                                                                                         | 85 |
| Table 5.17 | Frontend tools and libraries . . . . .                                                                                                                                                        | 85 |
| Table 6.1  | Frontend unit test scenarios and results . . . . .                                                                                                                                            | 88 |
| Table 6.2  | Backend unit test scenarios and results . . . . .                                                                                                                                             | 89 |



|           |                                                  |     |
|-----------|--------------------------------------------------|-----|
| Table 6.3 | End-to-end test scenarios and results .....      | 90  |
| Table 6.4 | AI Pipeline Test Scenarios .....                 | 93  |
| Table 6.5 | AI Services Test Cases .....                     | 94  |
| Table 6.6 | System-Level Evaluation Metrics .....            | 95  |
| Table 6.7 | Average API response times .....                 | 96  |
| Table 7.1 | AI Pipeline Runtime Benchmarks .....             | 98  |
| Table 7.2 | Backend and Frontend Runtime Measurements .....  | 99  |
| Table 7.3 | Feature comparison with existing platforms ..... | 104 |
| Table 9.1 | Achievement of Project Objectives .....          | 110 |

## LIST OF ACRONYMS

| Acronym | Expansion                                               |
|---------|---------------------------------------------------------|
| AI      | Artificial Intelligence                                 |
| API     | Application Programming Interface                       |
| AWS     | Amazon Web Services                                     |
| BERT    | Bidirectional Encoder Representations from Transformers |
| BFL     | Black Forest Labs                                       |
| BLEU    | Bilingual Evaluation Understudy                         |
| CDN     | Content Delivery Network                                |
| CLI     | Command-Line Interface                                  |
| CLIP    | Contrastive Language-Image Pre-training                 |
| CORS    | Cross-Origin Resource Sharing                           |
| CRUD    | Create, Read, Update, Delete                            |
| CSRF    | Cross-Site Request Forgery                              |
| CSS     | Cascading Style Sheets                                  |
| DFD     | Data Flow Diagram                                       |
| DPI     | Dots Per Inch                                           |
| DRF     | Django REST Framework                                   |
| DST     | Digital Storytelling                                    |
| FR      | Functional Requirement                                  |
| GPT     | Generative Pre-trained Transformer                      |
| HF      | High Frequency                                          |
| HTTP    | Hypertext Transfer Protocol                             |
| ITU     | International Telecommunication Union                   |
| JSON    | JavaScript Object Notation                              |
| KSA     | Kingdom of Saudi Arabia                                 |
| LAION   | Large-scale Artificial Intelligence Open Network        |
| LLM     | Large Language Model                                    |
| MOS     | Mean Opinion Score                                      |
| MSA     | Modern Standard Arabic                                  |
| MVC     | Model-View-Controller                                   |
| NFR     | Non-Functional Requirement                              |
| NISQA   | Non-Intrusive Speech Quality Assessment                 |
| NLG     | Natural Language Generation                             |
| NLP     | Natural Language Processing                             |
| ORM     | Object-Relational Mapping                               |
| PDF     | Portable Document Format                                |
| REST    | Representational State Transfer                         |
| RMS     | Root Mean Square                                        |
| RNN     | Recurrent Neural Network                                |
| ROUGE   | Recall-Oriented Understudy for Gisting Evaluation       |
| RTL     | Right-to-Left                                           |
| SDK     | Software Development Kit                                |
| SQL     | Structured Query Language                               |
| STT     | Speech-to-Text                                          |

|      |                              |
|------|------------------------------|
| TTR  | Type-Token Ratio             |
| TTS  | Text-to-Speech               |
| UI   | User Interface               |
| URL  | Uniform Resource Locator     |
| VPS  | Virtual Private Server       |
| WSGI | Web Server Gateway Interface |
| XSS  | Cross-Site Scripting         |

# 1 INTRODUCTION AND MOTIVATION

## 1.1 Purpose of the Project

Stories have long been considered an effective and central educational tool in child development, transcending their function as mere entertainment to become a powerful tool for shaping behavior and developing basic skills. Educational studies have confirmed the positive effectiveness of stories in modifying undesirable behaviors in preschool children, highlighting their ability to instill values and concepts in an indirect and influential manner [1]. In addition, stories play a vital role in developing language skills in kindergarten children, enriching their vocabulary and improving their expressive and comprehension abilities, making them a fundamental pillar of the early educational process [2]. This dual importance of stories, both in behavioral and linguistic aspects, justifies the search for innovative methods to maximize their benefits.

In the context of rapid technological development, artificial intelligence (AI) technologies have emerged as a new horizon for enhancing educational tools, especially in the field of guided stories. Recent research has shown growing interest in experiments using AI-generated social stories as a tool for effective behavioral intervention with children [3]. Other studies have indicated that AI-generated stories, which blend art and science, can have a positive impact on children's attitudes and behaviors, such as promoting pro-environmental attitudes [4]. However, there remains a clear gap in providing AI-based solutions that are culturally and religiously customized, take into account the specificities of Arab and Islamic communities, and directly target modifying individual behaviors of children in their specific environment.

This proposed project aims to develop a comprehensive AI-powered web application to generate personalized children's stories in a cartoon style, with the specific goal of addressing and correcting undesirable behavioral traits or habits in children. The project also seeks to support cultural and religious identity by producing stories that emphasize the values of respect, Islamic principles, and the traditions of Saudi society through stories and culture targeted at children. Each story will be tailored to the individual child's life, helping them learn to distinguish between right and wrong behavior. Furthermore, the project relies on the use of simplified and precise Arabic to ensure the content is linguistically appropriate for children. By combining modern pedagogical methods with creative storytelling, this project will create a fun and memorable learning environment.

## 1.2 Purpose of this Document

This document serves as a comprehensive technical and design specification for a web-based AI platform that generates personalized children's stories for behavioural modification. It is intended to guide the development team, project supervisors, and evaluators.

The primary purposes of this document are to:

1. Define Project Scope: Clearly articulate the objectives, features, and boundaries of the proposed system.
2. Document Requirements: Provide detailed functional and non-functional requirements that address the unique needs of Arabic-speaking children, including cultural, religious, and linguistic considerations specific to Saudi society.
3. Establish Design Specifications: Present the architectural design, AI models, algorithms, and technical strategies that will be employed to create personalized, culturally appropriate, and educationally effective stories.
4. Guide Implementation: Offer a structured roadmap for the development process.

## 1.3 Project Scope

This project aims to create a web platform. The platform focuses on utilizing Generative AI (LLM, Image Generating, TTS) to create personalized, Saudi culture (e.g., Saudi clothing or landmarks), Islamic-

aligned stories in Arabic that target specific negative behaviors in children aged 4–9, while providing a tracking dashboard for parents.

## 1.4 Motivation

Our motivation comes from the power of storytelling; research confirms it's the best way to modify a child's behavior and build their vocabulary [2]. We noticed a major gap: current AI platforms ignore our Saudi identity and Islamic values. We created Gheras to bridge this gap, using AI to make the child the hero of a story that treats bad behaviors while preserving our cultural roots.

## 1.5 Overview of this Document

This document consists of several chapters that aim to provide a comprehensive overview of a project to develop a web platform for generating personalized children's stories using artificial intelligence.

Chapter one begins with a general introduction to the project, outlining its purpose and the purpose of this document, as well as a brief overview of existing systems and the challenges they face. This chapter lays the foundation for understanding the overall context and importance of the project.

Chapter two focuses on system analysis, detailing the functional and non-functional requirements of the proposed platform. It also discusses performance, usability, and security considerations, in addition to proposed and alternative solutions.

Chapter three addresses the design considerations, the architectural strategies, algorithms to be used, project management methods, and software development approach. This chapter also discusses possible future enhancements to improve the platform's scalability and educational impact.

Chapter four presents the system design, illustrating the overall system architecture, data flow, and the interrelation between major and submodules. It also provides a detailed description of each component that contributes to generating and managing personalized AI-driven stories.

## 1.6 Existing Systems

### 1.6.1 Existing systems description

The landscape of children's storytelling content delivery has evolved significantly, encompassing traditional methods, early digital adaptations, and, more recently, sophisticated AI-driven platforms. Understanding these existing systems provides crucial context for positioning the proposed solution, "Gheras," and identifying its unique contributions.

#### 1.6.1.1 Traditional systems

The current landscape of systems offering stories to children ranges from traditional approaches to artificial intelligence applications. Historically, societies relied on printed storybooks and oral storytelling as primary tools for education and entertainment. These methods, while effective in transmitting culture and values, lack individual personalization and the ability to adapt to each child's evolving needs. They also do not offer direct mechanisms for behavior tracking or immediate interaction with content.

These systems rely on stories as both educational and therapeutic tools. They are essentially traditional and non-mechanical in nature, depending on direct human interaction between the educator, such as a teacher or parent, and the child. The main purpose of these systems is to modify behavior and instill moral and religious values [1]. This is typically achieved through the storytelling method, which is recognized as an effective means of teaching children social, behavioral, moral, and religious values. Researchers have cited the Holy Quran as one of the oldest and most influential sources of purposeful storytelling [5].

In application, traditional storytelling is often implemented through structured, story-based training programs designed to address specific misbehaviors such as aggression, stubbornness, and lying [5]. Other studies have confirmed the positive influence of stories on modifying preschool children's behavior, showing effectiveness across genders [1]. These traditional systems are characterized by their

reliance on printed or oral stories and their dual educational and behavioral (therapeutic) purpose. They are typically conducted in Arabic, as many studies in this area have been developed and applied in Arab contexts [1], [5]. However, personalization in these systems remains limited and non-automated, depending primarily on the educator's judgment to select a story that aligns with the child's behavioral challenge or the moral lesson being taught.

Despite the effectiveness of traditional storytelling methods, they suffer from several key limitations that justify the move toward intelligent, AI-driven solutions. First, there is a lack of deep and automated individualization. Personalization in traditional systems is restricted to choosing a story appropriate for a specific age or behavior [5]. These systems do not allow for integrating the child as the main character by including their image, name, thereby reducing the emotional connection that more personalized systems could achieve. Second, traditional storytelling depends heavily on human effort and resources. Effective interactive storytelling requires educators to have advanced expressive abilities, creativity, and deep knowledge, which can be a challenge for parents or teachers with limited time or experience.

These limitations illustrate that while traditional storytelling remains a powerful behavioral and educational tool, it lacks the flexibility, scalability, and deep personalization needed in today's digital age. The proposed project addresses these gaps by merging the proven pedagogical strength of storytelling with the creative and adaptive capabilities of generative artificial intelligence, enabling the instant creation of culturally, religiously, and behaviorally relevant stories tailored to each individual child.

#### **1.6.1.2 From Traditional Systems to Digital Systems**

The shift from traditional storytelling methods to digital story systems marks a transformative change in educational and behavioral development practices for children. Traditional storytelling delivered orally or through printed books has long served as an effective tool for instilling values and shaping character. However, as technology advanced, educators and researchers began integrating digital media to make storytelling more interactive, visually engaging, and accessible. Digital storytelling (DST) platforms allow the combination of multimedia elements such as images, animations, sound, and text to create a more immersive learning experience.

Studies such as in [6] highlight how digital storytelling enhances children's creativity, communication, and emotional engagement by allowing them to become active participants rather than passive listeners. Through interactive story elements, children can explore narratives, visualize storylines, and express themselves using technology. These systems have shown promise in early education by improving literacy skills, imagination, and collaboration among learners.

Despite these advantages, existing digital storytelling systems continue to exhibit significant limitations that constrain overall educational and behavioral effectiveness.

First, customization remains static, and limited personalization within these systems is generally superficial and non-automated, often confined to selecting a story suitable for a particular age group or behavioral theme. Such minimal customization fails to create a deeply engaging and emotionally resonant experience for the child.

Second, these platforms exhibit a high dependency on human resources for content creation. Producing culturally authentic and pedagogically accurate stories, particularly those that integrate Quranic values, Islamic teachings, or Saudi cultural elements, requires extensive manual effort, time, and expertise from educators and content developers. This dependency restricts scalability and the ability to provide timely, personalized content tailored to each child's needs.

#### **1.6.1.3 From Digital Systems to AI-Based Systems**

There are now more chances to turn storytelling into an engaging and flexible learning environment thanks to the quick development of Artificial Intelligence (AI) technologies, especially in the areas of Natural Language Processing (NLP) and image creation. These days, AI-based platforms can automatically create visually appealing and interactive stories for individual users. However, the majority of these systems are still only used for simple educational or entertainment purposes. Particularly in the

context of Arabic-speaking and Islamic communities, they hardly ever incorporate cultural representation, linguistic accuracy, or behavioral treatment. Examples of these systems include:

**Storypark**, is a web-based artificial intelligence storytelling platform developed to enhance children's story learning through Child-AI collaborative interaction. It employs the narrative generation and conversational capabilities of advanced Large Language Models (LLMs), GPT-4, to create coherent and contextually rich narratives that promote creativity, key values understanding, and language development.

The system follows an interactive "Question-Feedback-Story Generation" model, wherein the AI poses open-ended questions to elicit children's narrative input, expands the storyline based on their responses, and integrates moral and educational themes throughout the story. Storypark's design supports a five-step storytelling process: generating question prompts, expanding storylines, assisting in visual completion through sketch interpretation, evaluating comprehension using "truth questions," and providing scaffolding explanations to reinforce the story's core moral. To support multimodal creativity, the platform offers a visual-voice interface that allows children to communicate using voice and drawings, assisted by both a conversational generation and a sketch generation module. Technically, the system integrates multiple AI techniques, including LLM-based narrative generation, prompt engineering to define story context and tone, speech-to-text (STT) and text-to-speech (TTS) functionalities, and Sketch-RNN for enhancing children's drawing. Storypark also features personalized and adaptive learning, as the AI provides real-time, context-aware responses that align with the child's imaginative input, offering scaffolded feedback based on their understanding of story morals. Moreover, story content is tailored to different age groups, ensuring appropriate cognitive engagement for younger and older children alike. Through this integration of linguistic, visual, and interactive elements, Storypark helps children understand the story's key values and educational enrichment and creativity development, narrative thinking, and emotional expression [7].

**TinyTeller** is an application engineered to remedy the inherent limitations of conventional, fixed-narrative storytelling by targeting young children. Its core idea is to leverage generative AI for crafting personalized, limitless story content. Its primary purpose is educational and entertainment-focused, aiming to nurture language skills, imagination, and a passion for reading. The system primarily focuses on utilizing AI models to adapt storylines based on children's feedback, interests, and developmental levels. The core AI technique used is the integration of GPT-4-turbo-preview for sophisticated text generation, guided by meticulous Prompt Engineering that incorporates child profiles (age, name) and explicit Child Preferences (story types, characters, themes, and tones) [5].

The Personalization Level is high, as the system tailors the narrative, complexity, and vocabulary (e.g., controlling word count based on age) and continuously refines its approach using a feedback loop that includes direct user ratings (Relevance, Engagement, Educational Value). Each criterion is evaluated as a number between 0 to 5, and an optional textual comments form, in which users can enter additional textual evaluations. Moreover, Additional AI support includes Amazon Polly for Text-to-Speech (TTS) functionality, which reads the story aloud while highlighting the text for easy readability. Language Support is inherent to the LLM and TTS services used.

One of its limitations is that it does not generate illustrations; it generates only text. While children interact more with illustrations.

**StoryBee** is an interactive website that combines AI storytelling with illustrations to create personalized stories for children aged 3 to 12, focusing primarily on blending education with entertainment. StoryBee focuses on a sequential, end-to-end content creation process designed for accessibility. The user begins by describing their story idea (the prompt), which the platform rapidly translates into a complete story with accompanying illustrations. The resulting narratives adhere to a traditional storybook format.

The level of personalization offered by StoryBee is high, where the user can choose: genre, image style, age group, language, and characters when creating the story. extending across the multimodal outputs of the platform. Users possess granular control over the narrative structure, enabling them to specify

core elements such as characters, settings, plot points, and the intended moral or educational lesson. Furthermore, the narrative’s complexity can be customized to meet the cognitive needs of the child, ensuring suitability for the entire target demographic of ages 3–12.

The personalization element transcends the textual domain, moving into the visual and auditory spheres. Visually, users can tailor the illustration style. Auditively, the voice cloning feature allows for the ultimate personalization: embedding a familiar voice into the narrative experience. In addition to that, it supports multiple languages, including Arabic [8].

Although it supports Arabic, its customization capabilities do not indicate the incorporation of authentic Islamic teachings or specific local cultural aspects like Saudi clothing or landmarks.

### 1.6.2 Problems in the existing systems

Despite the notable advancements in generative storytelling systems, significant limitations and gaps remain unaddressed by current solutions, highlighting the need for our proposed project:

1. **Absence of Cultural and Religious Customization:** All of the systems studied primarily focused on English and Western cultural contexts. None of these systems’ capabilities indicates incorporating authentic Islamic teachings (values and morals) into the narrative, or incorporating local cultural aspects (such as Saudi clothing, and landmarks). This leaves a significant gap in the local market, where users need content that aligns with their values and identities.
2. **Lack of Authentic Arabic Language Support:** While some systems may offer Arabic translation, few support the generation of stories in fluent and linguistically accurate Arabic, considering grammatical rules and morphology. Our project primarily focuses on generating stories in correct standard Arabic.
3. **Limited Behavioral Customization:** Most systems lack a clear framework for behavior modification or the ability to target specific behaviors in children. Systems like StoryPark AI, TinyTeller, and StoryBee AI focus on general educational purposes [5], [7], [8]. They do not offer direct mechanisms to address behavioral issues such as aggression or lying.
4. **Lack of Parent/Teacher Dashboards:** Existing systems often do not provide comprehensive dashboards for parents or teachers to track a child’s progress, further customize stories, or monitor the impact of stories on behavior. The proposed solution can incorporate an integrated Parent/Teacher Dashboard, enabling caregivers to monitor the child’s behavioral progress and customize the storytelling experience more effectively.
5. **Limited Visual Customization:** Although some systems generate images, customization of the child character’s appearance in the story (e.g., gender, hair color, facial features) remains limited. Our project will allow for customization of the story’s hero to resemble the child, enhancing engagement and interaction.

These issues highlight the urgent need for a new system that addresses these gaps and offers a comprehensive and innovative solution for generating personalized children’s stories using AI, combining cultural and religious personalization with linguistic accuracy, to treat children’s behavior.

#### **Our proposed solution:**

The proposed solution, Gheras, will enhance the limitations identified in traditional, digital, and existing AI storytelling systems by including high levels of Saudi cultural and Religious Customization, and a Progress Tracking Dashboard. Unlike general educational tools, Gheras is fundamentally designed to fill the market gap for culturally and linguistically appropriate content in Standard Arabic that is personalized to the individual child, extending to visual customization to allow the story hero to resemble the child. This distinct combination of features, as summarized in Table 1.1, justifies its development over leveraging existing alternatives.

Table 1.1. A comparison table evaluates the proposed solution “Gheras” against traditional, digital, and existing AI-based storytelling systems based on key features



| Feature                    | Traditional Systems (Printed & Oral)                                                                                                                                                                                                                                         | Digital Storytelling Systems                                                                                            | StoryPark [7]                                                                                                                                                                                                       | TinyTeller [5]                                                                                                                                                                                                      | StoryBee [8]                                                                                                                                                                                                        | Proposed solution (Gheras)                                                     |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Story Format</b>        | Printed books / oral stories                                                                                                                                                                                                                                                 | Multimedia (text, sound, video)                                                                                         | AI-generated text, illustrated (cartoon) stories, sound                                                                                                                                                             | AI-generated text stories, sound                                                                                                                                                                                    | AI-generated text, illustrated (cartoon) stories, sound                                                                                                                                                             | AI-generated text, illustrated (cartoon) stories, sound                        |
| <b>Purpose / Focus</b>     | Educational & behavioral treatment                                                                                                                                                                                                                                           | Educational & entertainment                                                                                             | Child–AI collaboration for creative learning                                                                                                                                                                        | Educational & entertainment                                                                                                                                                                                         | Educational & entertainment                                                                                                                                                                                         | Behavioral treatment                                                           |
| <b>Customization Level</b> | <b>Low</b> Restricted to educator selection based on age/behavior                                                                                                                                                                                                            | <b>Limited</b> Often confined to selecting from pre-defined stories or basic name changes.                              | <b>High</b> based on the child's Feedback, drawing visualization                                                                                                                                                    | <b>High</b> based on User Profile (Age, Name, feedback, and Story Type Preference)                                                                                                                                  | <b>High</b> story style, genre, age level, voice                                                                                                                                                                    | <b>High</b> (child's name, age, child's picture, behavior, story genre, style) |
| <b>Limitations</b>         | <ul style="list-style-type: none"> <li>• Time-consuming, nonautomated, (printed), static content.</li> <li>• Personalization is limited to selecting a story appropriate for a specific age or a targeted behavior.</li> <li>• Lack of Parent/Teacher Dashboards.</li> </ul> | <ul style="list-style-type: none"> <li>• Time-consuming, static content.</li> <li>• Personalization limited.</li> </ul> | <ul style="list-style-type: none"> <li>• Lack of Arabic/Islamic context.</li> <li>• Lack of behavioral treatment, focus on general educational enrichment.</li> <li>• Lack of Parent/Teacher Dashboards.</li> </ul> | <ul style="list-style-type: none"> <li>• Lack of Arabic/Islamic context.</li> <li>• Lack of behavioral treatment, focus on general educational enrichment.</li> <li>• Lack of Parent/Teacher Dashboards.</li> </ul> | <ul style="list-style-type: none"> <li>• Lack of Arabic/Islamic context.</li> <li>• Lack of behavioral treatment, focus on general educational enrichment.</li> <li>• Lack of Parent/Teacher Dashboards.</li> </ul> | Support only the Arabic language & Saudi culture                               |

## 2 SYSTEM ANALYSIS

This chapter provides a detailed analysis of the Gheras platform. The analysis begins by examining the flow of data through the system using Data Flow Diagrams (DFDs) to visualize interactions between system components and external entities. Subsequently, the chapter defines the system's stakeholders and their functional roles, outlines the underlying data structure, and presents a comprehensive list of functional requirements that will serve as the technical blueprint for the platform's development.

### 2.1 Data Analysis

Data analysis plays a fundamental role in understanding how information is collected, processed, stored, and exchanged within the Gheras platform. This phase focuses on identifying the system's functional behavior and the interactions between users, system components, and data repositories. To achieve this, the system was analyzed using Use Case Diagrams and Data Flow Diagrams (DFDs), which together provide a structured representation of both functional requirements and data movement within the system.

#### 2.1.1 Use case diagram

As Use Case Diagrams were employed to capture the interactions between external actors and the system, clearly defining the services provided by the platform. These interactions were further detailed through comprehensive use case scenario tables, which describe each system function in terms of actors, data, stimuli, system responses, and operational notes.

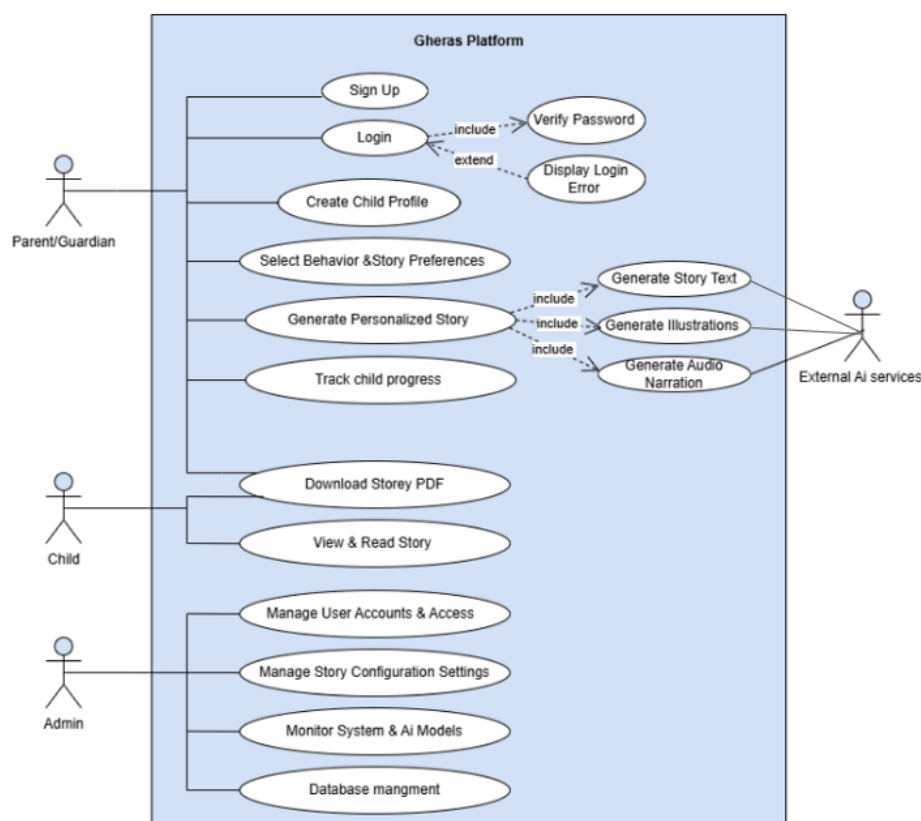


Figure 2.1. System Use Case Diagram

As illustrated in Figure 2.1, the use case diagram represents the interaction between the Gheras platform and its main actors: the Parent/Guardian, Child, Administrator, and External AI Services. It shows the system functions available to each actor, including user authentication, child profile creation, story

preference selection, personalized story generation, story reading, PDF downloading, progress tracking, and administrative management. Each use case is further explained in detail in the use case scenario tables.

The External AI Services actor represents the third-party AI components used by the platform to support story generation. These services are involved in generating the story text, producing illustrations, and creating audio narration. Therefore, the AI-related functionalities are modeled as interactions with an external actor because they depend on external AI APIs and infrastructure rather than being fully implemented inside the Gheras platform.

Table 2.1. Use Case Scenario for Sign Up

| Field              | Description                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------|
| <b>Actor</b>       | Parent / Guardian                                                                                        |
| <b>Description</b> | The process through which a new parent or guardian creates an account in the system.                     |
| <b>Data</b>        | Name, email address, password                                                                            |
| <b>Stimulus</b>    | Parent selects the Sign-Up option, then completes and submits the registration form                      |
| <b>Response</b>    | System validates input and creates a new user account are displayed, highlighting the issues identified. |
| <b>Comments</b>    | Required before login and system usage.                                                                  |

Table 2.2. Use Case Scenario for Login

| Field              | Description                                                                                                                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Actor</b>       | Parent / Guardian                                                                                                                                                                                                                                                                          |
| <b>Description</b> | Authenticates a registered user to allow access to the system.                                                                                                                                                                                                                             |
| <b>Data</b>        | Email address, password                                                                                                                                                                                                                                                                    |
| <b>Stimulus</b>    | The parent clicks on the “Log In” or “Log Out” button.                                                                                                                                                                                                                                     |
| <b>Response</b>    | <ul style="list-style-type: none"> <li>• Upon logging in: The system verifies the entered information and grants access to the user’s account.</li> <li>• Upon logging out, the system terminates the user’s session and redirects them to a confirmation page or the homepage.</li> </ul> |
| <b>Comments</b>    | Password verification and error handling are handled internally as part of the login process.                                                                                                                                                                                              |

Table 2.3. Use Case Scenario for Create Child Profile

| Field              | Description                                                                                                                                                                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Actor</b>       | Parent / Guardian                                                                                                                                                                                                                                                                                                                          |
| <b>Description</b> | Allows the parent to create or update a child’s profile for story personalization                                                                                                                                                                                                                                                          |
| <b>Data</b>        | Child name, age group, gender, picture (or characteristics).                                                                                                                                                                                                                                                                               |
| <b>Stimulus</b>    | Parent selects “Create / Manage Child Profile.”                                                                                                                                                                                                                                                                                            |
| <b>Response</b>    | Child profile saved or updated in the system                                                                                                                                                                                                                                                                                               |
| <b>Comments</b>    | <ul style="list-style-type: none"> <li>• Multiple profiles for more than one child may be created.</li> <li>• For each profile, include the child’s name, picture, age, and gender to tailor story language levels and complexity.</li> <li>• all generated stories saved and associated with each corresponding child profile.</li> </ul> |

Table 2.4. Use Case Scenario for Select Behavior & Story Preferences. Available preferences are predefined and managed by the system administrator to ensure controlled personalization.

| Field       | Description                                             |
|-------------|---------------------------------------------------------|
| Actor       | Parent / Guardian                                       |
| Description | Enables selection of behavioral and story preferences.  |
| Data        | Behavioral traits, genre, and illustration styles       |
| Stimulus    | Parent confirms selected preferences                    |
| Response    | Preferences stored for story generation                 |
| Comments    | Options are defined and maintained by the administrator |

Table 2.5. Use Case Scenario for Generate Personalized Story. Story content is generated dynamically using artificial intelligence models based on the child's profile and selected preferences.

| Field       | Description                                                                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Actor       | Parent / Guardian                                                                                                                       |
| Description | Generates a personalized story based on the child's profile and preferences.                                                            |
| Data        | Child profile, story preferences, behavioral trait, illustration style                                                                  |
| Stimulus    | Parent clicks and selects "Generate Personalized Story."                                                                                |
| Response    | <ul style="list-style-type: none"> <li>Personalized story content (text, images, and optional audio) is generated and stored</li> </ul> |
| Comments    | Text and images are generated using AI models via an external TTS/image/text generating service.                                        |

Table 2.6. Use Case Scenario for Track Child Progress. Progress metrics support monitoring of learning outcomes and behavioral development over time.

| Field       | Description                                                                                                                                                                                                                                                                                           |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor       | Parent / Guardian                                                                                                                                                                                                                                                                                     |
| Description | Tracks the child's engagement and learning progress, including: <ul style="list-style-type: none"> <li>Number of stories generated per behavioral trait (e.g., honesty, empathy)</li> <li>Story completion status (read or listened)</li> <li>Completion of post-story tasks or activities</li> </ul> |
| Data        | Reading activity logs, story interaction data, progress metrics                                                                                                                                                                                                                                       |
| Stimulus    | Parent requests to view the child's progress report                                                                                                                                                                                                                                                   |
| Response    | The system generates and displays a progress report                                                                                                                                                                                                                                                   |
| Comments    | Used to monitor learning outcomes and behavioral development over time                                                                                                                                                                                                                                |

Table 2.7. Use Case Scenario for View & Read Story. Interaction data is recorded to support engagement analytics and progress tracking.

| Field       | Description                                                |
|-------------|------------------------------------------------------------|
| Actor       | Child                                                      |
| Description | Allows the child to view, read, or listen to stories.      |
| Data        | Stored story content                                       |
| Stimulus    | The child opens a story, reads, or clicks the audio button |

| Field           | Description                                                                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Response</b> | Story content displayed or played                                                                                                                           |
| <b>Comments</b> | <ul style="list-style-type: none"> <li>• Read-only access (can't modify the story)</li> <li>• Interaction data is recorded for progress tracking</li> </ul> |

Table 2.8. Use Case Scenario for Download Story PDF. Download functionality enables offline access to generated educational content.

| Field              | Description                                                         |
|--------------------|---------------------------------------------------------------------|
| <b>Actor</b>       | Child / Parent                                                      |
| <b>Description</b> | Allow the parent/child to download a generated story in PDF format. |
| <b>Data</b>        | Generated story content                                             |
| <b>Stimulus</b>    | Parent/child clicks the download option                             |
| <b>Response</b>    | The story is downloaded as a PDF file                               |
| <b>Comments</b>    | Enables offline access to story content                             |

Table 2.9. Use Case Scenario for Manage User Accounts & Access. Administrative controls enforce secure authentication, authorization, and role-based access.

| Field              | Description                                                                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Actor</b>       | System Administrator                                                                                                                                                   |
| <b>Description</b> | Manages user (parent) accounts and access control, including creating, updating, disabling, or removing user accounts and assigning appropriate roles and permissions. |
| <b>Data</b>        | User records, access roles, authentication status                                                                                                                      |
| <b>Stimulus</b>    | Administrator initiates a user management action                                                                                                                       |
| <b>Response</b>    | User account information and access permissions are updated                                                                                                            |
| <b>Comments</b>    | Ensures secure authentication, authorization, and controlled system access                                                                                             |

Table 2.10. Use Case Scenario for Manage Story Configuration Settings. Configuration settings act as controlled inputs for AI-driven story personalization.

| Field              | Description                                                                                                                                                                                                                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Actor</b>       | System Administrator                                                                                                                                                                                                                                                                                                                   |
| <b>Description</b> | Configures and maintains story-related settings used for personalization and AI conditioning.                                                                                                                                                                                                                                          |
| <b>Data</b>        | Behavioral traits, story genres, illustration styles, age groups (4-9): <ul style="list-style-type: none"> <li>• A pre-defined, administrator-managed list of common childhood behaviors.</li> <li>• Perform full CRUD (Create, Read, Update, Delete) operations on the list of behavioral_traits &amp; illustration_styles</li> </ul> |
| <b>Stimulus</b>    | Administrator modifies story configuration settings                                                                                                                                                                                                                                                                                    |
| <b>Response</b>    | Configuration data is updated and stored                                                                                                                                                                                                                                                                                               |
| <b>Comments</b>    | Configuration settings are used as controlled inputs for AI-based story generation                                                                                                                                                                                                                                                     |

Table 2.11. Use Case Scenario for Monitor System & AI Models. Monitoring applies to externally integrated AI services.

| Field       | Description                                                                |
|-------------|----------------------------------------------------------------------------|
| Actor       | System Administrator                                                       |
| Description | Observes the behavior and performance of AI models used within the system. |
| Data        | AI model logs, inference statistics, error reports                         |
| Stimulus    | Administrator reviews AI model activity                                    |
| Response    | AI model performance indicators are presented                              |
| Comments    | Applies to externally integrated AI services                               |

Table 2.12. Use Case Scenario for Database Management. Database administration ensures data integrity, security, and system performance.

| Field       | Description                                                                                          |
|-------------|------------------------------------------------------------------------------------------------------|
| Actor       | System Administrator                                                                                 |
| Description | Manages the system's SQL-based database to ensure security, scalability, integrity, and performance. |
| Data        | User accounts data, child profiles, stories repository, progress records                             |
| Stimulus    | The administrator performs a database management operation                                           |
| Response    | Database records are updated, maintained, or backed up                                               |
| Comments    | Covers backups, integrity checks, and performance optimization                                       |

### 2.1.2 Data Flow Diagrams

Data Flow Diagrams were developed based on the identified use cases to illustrate how data moves between external entities, system processes, and internal data stores at different levels of abstraction.

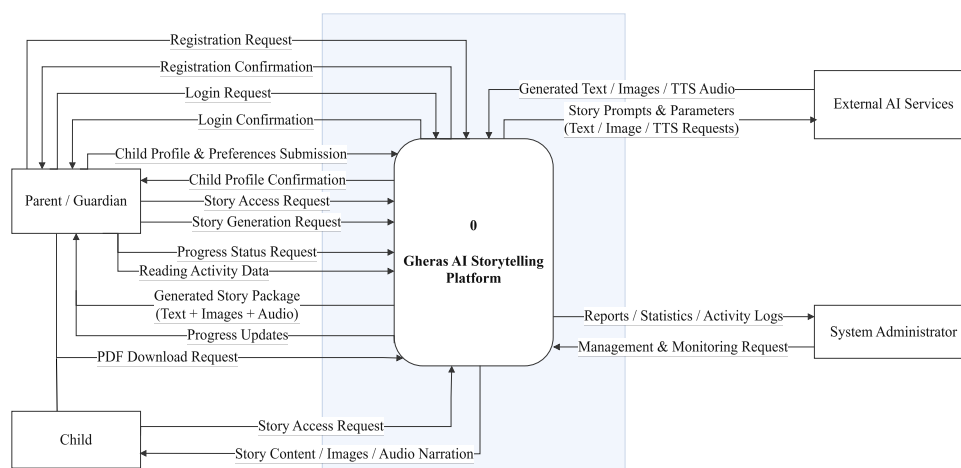


Figure 2.2. Context diagram

The Level 0 Data Flow Diagram (DFD), also referred to as the Context Diagram, shown in Figure 2.2, represents the Gheras platform as a single unified process and illustrates its interactions with its external entities, namely the Parent/Guardian, Child, System Administrator, and External AI Services. The diagram defines the overall system boundary and presents the primary request–response data exchanges between the platform and its surrounding entities. Parent/Guardian interactions include registration and login requests with their corresponding confirmations, child profile and preference submission with profile confirmation, story access and story generation requests, progress status requests with progress updates, PDF download requests with generated story PDFs, and reading activity data. The platform

also provides generated story packages that include text, images, and audio narration. Child interactions focus on consuming and accessing generated content through story access requests and receiving story content, illustrations, and audio narration appropriate for different age groups. Communication with External AI Services involves sending story prompts and generation parameters and receiving AI-generated text, images, and text-to-speech (TTS) audio outputs. Furthermore, the System Administrator interacts with the platform through management and monitoring requests and receives reports, statistics, and activity logs.

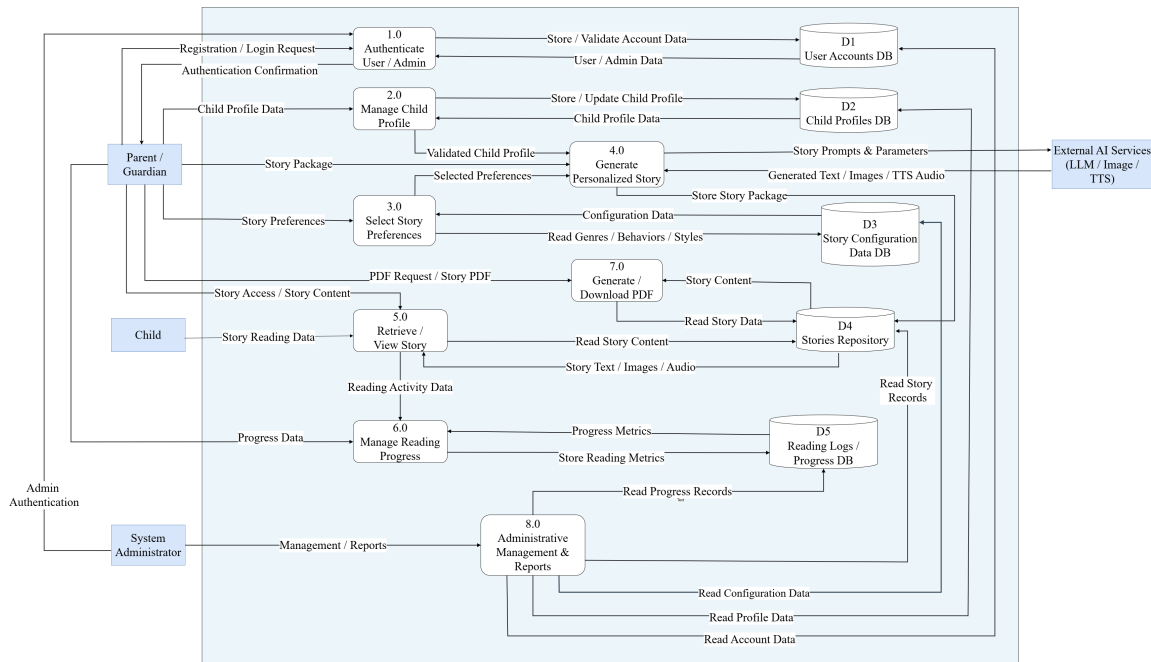


Figure 2.3. DFD level 1

Figure 2.3 presents the Level 1 Data Flow Diagram (DFD) of the Gheras AI Storytelling Platform, providing a detailed representation of the system's primary functional processes and data interactions. The diagram decomposes the single process shown in the Context Diagram into eight major processes: user authentication, child profile management, story preference selection, personalized story generation, story retrieval and viewing, reading progress management, PDF generation and download, and administrative management and reporting. Together, these processes represent the complete workflow of the platform, illustrating how user inputs are transformed into personalized behavioral stories tailored to the child's profile, selected preferences, and target behavior. Each process exchanges data with the relevant data stores to support system functionality and maintain data consistency: user account information is managed through the User Accounts Database (D1), child-related information through the Child Profiles Database (D2), story configurations through the Story Configuration Data DB (D3), generated stories through the Stories Repository (D4), and reading activity through the Reading Logs / Progress DB (D5). Special attention was given to distinguishing between internal system processes and external services. Story text, illustration generation, and audio narration generation rely on an external Text-to-Speech service and are therefore modeled as an external entity interacting with the system through defined data flows.

The decomposition of the Data Flow Diagrams was intentionally limited to Level 1, as this level sufficiently represents the system's major processes and data interactions. Further decomposition into lower levels would introduce implementation-specific details that fall outside the scope of system analysis and requirements modeling. The detailed behavior of each process is instead captured through

the corresponding use case scenario tables, providing an appropriate balance between abstraction and clarity.

### 2.1.3 System Requirements

This section introduces the System requirements, which define the expected behavior and constraints of the Gheras platform by translating user needs and system objectives into clearly specified requirements. These requirements were derived through the analysis of the Use Case Diagram, detailed use case scenario tables, and Data Flow Diagrams, ensuring consistency between user interactions, system functionality, and data processing activities.

The system requirements are categorized into clients, customers, and users, functional requirements, and nonfunctional requirements. This classification provides a structured understanding of who interacts with the system and what operations the system must perform to support personalized story generation for children.

#### 2.1.3.1 Clients, Customers, and Users

The *Gheras* platform is designed to serve a distinct set of users, each with specific roles and objectives within the system.

- **Parent/Guardian:** The primary customer and user of the platform. Their goal is to access a safe, effective, and culturally appropriate tool to address specific behavioral challenges in their children. They create accounts, manage child profiles, select behavioral traits and story preferences, generate personalized stories, download story outputs (PDF format), and review child progress reports.
- **Child (Target Age: 4–9):** The primary end-user. The child consumes the generated stories through reading, viewing illustrations, and (optionally) listening to narration. The child interface is designed to be simple, engaging, and visually appealing, supporting enjoyment while encouraging positive behavior.
- **System Administrator:** A logical operational role responsible for managing user accounts and access control, maintaining story configuration settings (behavioral traits, genres, illustration styles, age groups), monitoring system performance and AI model activity, and performing database management tasks to ensure data integrity and system reliability. This role is included to reflect real-world deployment and to support required system performance testing and reporting.
- **External AI Services:** External service providers integrated into the system (a third-party text-to-speech, Image & Texts generating service) used specifically for audio narration generation.

#### 2.1.3.2 Functional Requirement Specifications (FR)

Functional requirements describe the specific actions and services that the Gheras system must provide to fulfill user needs.

##### 1. User Account Management

- The system shall allow parents or guardians to register for an account using an email address and password.
- The system shall restrict account registration to valid Gmail addresses only, requiring users to register using an email address with the “@gmail.com” domain.
- The system shall enforce password complexity rules to ensure account security.
- The system shall authenticate registered users using valid credentials.
- The system shall manage user roles and access permissions through administrative control.

##### 2. Child Profile Management

- The system shall allow a logged-in parent or guardian to create and manage one or more child profiles.
- The system shall store child profile attributes required for story personalization, including name, age group, and gender.
- The system shall associate each child profile with the corresponding parent or guardian account.



- The system shall allow optional storage of either child-related characteristics or the child's image to support personalization.
3. **Story Preferences and Customization**
    - The system shall allow parents or guardians to select a behavioral trait from a predefined list managed by the system administrator.
    - The system shall allow parents or guardians to type a behavioral trait if it does not exist in the predefined list.
    - The system shall allow parents or guardians to select story preferences prior to story generation, including story genre and illustration style.
    - Story genre (e.g., comedy, adventure, science, fantasy, etc).
    - Illustration style (e.g., watercolor, cartoons, pixel art, amine).
    - The system shall store selected preferences for use during story generation.
  4. **Personalized Story Generation**
    - The system shall generate personalized story text using an external natural language generation model.
    - The system shall generate story text appropriate to the child's age group and language level.
    - The system shall generate story illustrations using an external image generation model based on story context and child attributes.
    - The system shall incorporate child-specific attributes (from the child's profile) into generated story content to enhance personalization.
    - The system shall automatically generate a suitable story title and cover image.
    - The system shall assemble generated text and illustrations and audio into a complete story.
    - The system shall generate audio narration by invoking an external text-to-speech service.
  5. **Story Storage, Retrieval, and Download**
    - The system shall automatically store all generated stories and associate them with the corresponding child profile.
    - The system shall allow users to retrieve and view previously generated stories.
    - The system shall allow parents or guardians to download generated stories in PDF format.
    - The system shall preserve generated story metadata to support future retrieval and reuse.
  6. **Progress Tracking and Reporting**
    - The system shall record and analyze child interaction data related to story viewing or listening activities, to generate progress metrics.
    - The system shall track story completion status for each generated story.
    - The system shall provide parents or guardians with access to a progress dashboard displaying story history for each child profile.
    - The system shall display behavioral progress indicators based on story interaction data and completed post-story activities.
    - The system shall present progress information in a clear and understandable visual format.
  7. **Administrative Functions**
    - The system shall allow system administrators to view and manage user accounts and access permissions.
    - The system shall allow system administrators to perform full CRUD (Create, Read, Update, Delete) operations on system configuration data, including behavioral traits, story genres, illustration styles, and age groups.
    - The system shall allow system administrators to manage story configuration settings (behavioral traits, illustration styles)
    - The system shall provide system administrators with monitoring capabilities for system performance and AI model activity.

- The system shall support administrative database management operations to ensure data integrity and availability.
- The system shall store all data in a secure and scalable relational database.
- The system shall maintain persistent storage of user accounts, child profiles, generated stories, and progress records.
- The system shall ensure that stored data supports efficient retrieval and reporting operations.

### **2.1.3.3 Non-functional Requirements Specifications (NFR)**

Non-functional requirements (NFRs) define the qualitative attributes and constraints that govern the overall operation of the Gheras platform. While functional requirements (FRs) specify what the system must do, non-functional requirements describe how well the system should perform its functions. Ensuring these requirements are met is crucial for system acceptance, stability, and long-term viability. The following subsections outline the key non-functional requirements categorized under look and feel, usability, security, performance, and portability.

#### **2.1.3.3.1 Look and feel requirements**

- The Gheras web application shall utilize an interactive, visually appealing design theme appropriate for a children's storytelling platform, with a color palette and typography that promote readability for both children and parents.
- Each page shall maintain visual consistency, including uniform typography, color schemes, and iconography, to enhance cognitive ease and navigation.
- The design, iconography, and visual language of the application shall be aligned with Islamic and Saudi cultural values, ensuring all elements are respectful and contextually appropriate.
- The storybook display must simulate a digital storytelling experience, with side-by-side pages, one for text and one for corresponding illustrations.

#### **2.1.3.3.2 Usability requirements**

- The system shall be intuitive and easy to use for users of varying technical abilities.
- Parents or guardians shall be able to complete all key operations, account registration, profile creation, story generation, and download without requiring technical assistance. The system shall provide contextual guidance to assist users during key interactions (e.g., Contextual tooltip or placeholder text to guide the user).
- The child-facing story viewing interface shall be simple and intuitive, enabling children to read or listen to stories without assistance.
- The child's interactive reading mode shall feature simple navigation buttons (e.g., "Next Page," "Back," "Play Audio").
- To enhance the reading experience, the system shall include dual modes: Self-Reading Mode (The story is displayed as text and images for the user to read manually) and Read-Aloud Mode (The story is narrated using synthesized speech, synchronized with the text), allowing flexibility in user engagement.

#### **2.1.3.3.3 Security requirements**

- The system shall provide secure user authentication for parents to log in and enforce password complexity rules to ensure account security.
- Data Access Control to user accounts and generated stories shall be strictly controlled based on user roles, with Parents/Guardians only accessing their associated data.
- The system shall enforce role-based access control to distinguish administrative privileges from regular user access (controls to manage user accounts, view generated stories, and monitor system performance and database integrity).
- All sensitive data, including child profiles and passwords, shall be stored in a secure, scalable SQL-based database with robust security protocols.

**2.1.3.3.4 Performance requirement**

- The system shall handle concurrent user requests without significant degradation in performance.
- The generation of story text, illustrations, and audio narration shall be completed within a reasonable time to maintain a smooth user experience.
- Database Performance: The system shall be optimized for fast retrieval and efficient querying, allowing users to load their dashboard and view previously generated stories with minimal perceived delay.

**2.1.3.3.5 Portability requirements**

- The web application shall be rendered consistently across all major, modern web browsers (Chrome, Edge, Safari).
- The system shall integrate with external AI services through standardized APIs.

**2.1.3.3.6 Scalability requirements**

- The system shall support an increasing number of users, child profiles, and generated stories without requiring architectural changes.

**2.1.3.3.7 Maintainability requirements**

- The system shall be designed in a modular manner to facilitate maintenance and future enhancements.
- The system shall allow system administrators to update configuration data without affecting core system functionality.

**2.1.3.3.8 Data Privacy requirements**

- The system shall handle child-related data with a high level of privacy and confidentiality.
- The system shall store only data necessary for system operation and personalization.

### 2.1.4 Class Diagram



Figure 2.4. Class Diagram for Gheras platform

Figure 2.4 presents a high-level structural view of the Gheras system by modeling the main entities, their attributes, and the relationships among them. It captures the static architecture of the system and reflects how data is organized and managed to support the functional requirements identified in the use case diagram and data flow diagrams.

At the core of the system is the *User* class, which represents authenticated system users and is specialized into *ParentGuardian* and *SystemAdministrator* through inheritance. This design enables role-based access and responsibilities, where parents manage child-related activities and administrators oversee system configuration and monitoring. Each *ParentGuardian* can own and manage multiple *ChildProfile* instances, representing individual children for whom personalized stories are generated.

The *ChildProfile* class plays a central role in personalization, as it stores demographic and descriptive attributes that influence story generation. Each child profile can produce multiple *Story* objects, which encapsulate the generated narrative content, metadata, and output formats. A story is further decomposed

into *StoryPage*, *Illustration*, and *AudioNarration* classes, reflecting the system’s multimedia storytelling approach.

Customization is supported through the *StoryPreference* class, which links a story to selected *BehavioralTrait*, *AgeGroup*, *StoryGenre*, and *IllustrationStyle* entities. This separation ensures flexibility and extensibility, allowing administrators to manage configuration data independently from story instances. To support learning assessment and monitoring, the diagram includes *ReadingActivity* and *ProgressMetric* classes. These entities record children’s interaction with stories, completion status, and behavioral progress over time, enabling the generation of meaningful insights for parents through dashboards and reports. System-level monitoring is represented by *SystemLog* and *AIModelLog*, which support administrative oversight, performance evaluation, and auditing.

## 2.2 Proposed solution

The proposed solution, the Gheras platform, is an interactive web platform that integrates artificial intelligence (AI), natural language generation (NLG), image generation, and text-to-speech (TTS) technologies to create personalized children’s stories for ages 4-9. This solution aims to provide a powerful and flexible tool that enables parents to address negative behaviors in children (e.g., aggression, lying) in a creative and impactful manner by merging the proven pedagogical power of storytelling with the adaptive and creative capabilities of Generative Artificial Intelligence (AI) to produce personalized, culturally relevant, and behavior-modifying content for children.

The platform aims to bridge the gap between traditional storytelling, digital learning, and existing generative AI systems by automating the creation of interactive, multimedia stories customized for each child’s profile. Unlike conventional methods or generic digital storytelling tools, Gheras delivers dynamic, AI-driven experiences where children become the main protagonists in their own stories, complete with custom illustrations, names, and even personal characteristics [2].

### Key Pillars of the Proposed Solution:

1. **Hyper-Personalization:** The platform allows users to input specific details about the child (such as name, age, gender, picture, or characteristics) and the negative behavior to be addressed (e.g., stubbornness, fear of the dark, not sharing toys). The system integrates these inputs into both the generated story text and the accompanying illustrations to generate a unique story where the child is the hero, thereby enhancing the story’s impact and the child’s ability to relate to it, a concept supported by studies on personalized reading platforms [9].
2. **Targeted Behavioral Treatment:** Unlike systems focused only on general education or entertainment, Gheras integrates a clear framework for behavior modification. The solution generates stories targeting specific negative behaviors (e.g., aggression, lying) selected by the parent [7], [8].
3. **Cultural and Religious Adaptation:** Unlike generic global solutions, the proposed solution Gheras, is fundamentally designed for the Arabic-speaking and Islamic context. The solution constructs detailed prompts for the AI models that are intentionally grounded in Islamic and Saudi cultural values, incorporating local references (e.g., Saudi landmarks). This addresses the major absence of cultural and religious customization in current market solutions.
4. **Progress Tracking and Parent/Guardian Dashboard:** The system automatically collects data about the child’s interactions, including completed stories, identified behavioral traits, and participation in post-story activities/tips and tasks. The system then displays the collected behavioral data in a clear visual format (such as charts or graphs) on the parent/guardian dashboard, showing the story reading frequency for a specific target behavior and the completion status of post-story-related tasks.

## 2.3 Alternative solutions

Although the proposed solution represents the best option for achieving the project’s objectives, several alternative solutions (existing systems) were considered and evaluated, and the shortcomings of each were examined based on their features.

One primary alternative was Traditional Storytelling, in which parents would narrate stories or purchase/borrow a written storybook. because Effective, interactive storytelling relies heavily on the educator's (parent/teacher) advanced expressive abilities, creativity, and time, therefore This approach was excluded because it critically lacks scalability and speed, and cannot guarantee the level of hyper-personalization required for addressing specific, nuanced behaviors, a limitation noted in the context of developing language skills in kindergarten children [2]. As for printed storybooks, they remain static resources that cannot be adjusted to the level of hyper-personalization needed to address nuanced behavioral traits, nor can they adapt stories dynamically to include the child's identity or environment.

A second alternative was Digital Storytelling Systems

Digital storytelling (DST) platforms, which utilize multimedia elements like images, animations, and sound, enhance engagement and accessibility compared to purely traditional methods.

These platforms still exhibit a high dependency on human resources for content creation. Producing culturally authentic and pedagogically accurate stories (integrating Quranic values, Islamic teachings, or Saudi cultural elements) requires extensive manual effort, time, and expertise from content developers. This restriction severely limits scalability and the ability to provide personalized content on time.

Like traditional systems, DST systems suffer from static and limited personalization, typically confined to selecting a story suitable for an age group or behavioral theme. This superficial customization fails to create the deep, emotionally resonant experience the Gheras platform requires. A limitation frequently discussed in the literature on adaptive educational systems [7]. For these reasons, they were not selected as a viable solution.

A third option was to utilize existing AI-based storytelling systems, such as StoryPark, TinyTeller, StoryBee, and other general-purpose AI generators. While these systems demonstrate the potential of generative AI in producing personalized or developmental-level stories, they suffer from several critical shortcomings in the context of this project. Many do not prioritize or ensure adherence to Islamic values, Saudi cultural norms, or age-appropriate, simple Arabic vocabulary, focusing primarily on English and Western contexts.

In addition, most existing systems do not integrate behavioral modification as a primary objective. Systems like Storypark and TinyTeller focus on general educational & entertainment purposes and lack a clear, direct framework to address specific negative behaviors (e.g., aggression or lying). in addition, although some systems generate images, they lack the ability to they do not embed the child's real image, distinctive characteristics, or environment into the generated story and illustrations. Their customization options remain limited, often restricted to basic preferences such as theme or reading mode. Furthermore, these systems do not provide customized behavioral stories based on the targeted behavior or progress-tracking dashboards for parents and teachers. Given these substantial gaps, existing AI platforms were deemed unsuitable as a direct solution.

#### **Justification for Selecting the Proposed Solution:**

The development of the AI platform was ultimately chosen as the proposed solution because it uniquely combines Hyper-Personalization, Systematic Behavioral Treatment, Scalability, and Cultural & Islamic Adaptation. These are critical factors that cannot be achieved efficiently and effectively through the alternative solutions mentioned. This solution represents the intersection of modern technology and educational needs, making it the most innovative and impactful choice.

## 3 SYSTEM CONSIDERATIONS

This chapter outlines the key constraints, assumptions, and considerations that influence the design and development of the Gheras platform. These considerations include technical constraints related to hardware and software environments, characteristics of the intended end users, and potential directions for future system enhancements.

### 3.1 Design Constraints

#### 3.1.1 *Hardware and software environment*

The Gheras platform is designed as a web-based application that operates within a standard client–server architecture. On the client side, the system requires only a modern web browser, eliminating the need for specialized hardware or additional software installations. This design choice ensures broad accessibility across commonly used devices, including desktop computers, laptops, tablets, and smartphones.

On the server side, the system operates within a cloud-compatible software environment capable of hosting web services, databases, and AI model integrations. The platform relies on a backend infrastructure that supports secure data storage, API communication, and interaction with external text, image, and text-to-speech generating services. The database environment is based on a relational database management system to ensure structured storage, data integrity, and efficient querying. On the server side, the system operates within a cloud-compatible software environment capable of hosting web services, databases, and AI model integrations. The platform relies on a backend infrastructure that supports secure data storage, API communication, and interaction with external text, image, and text-to-speech generating services. The database environment is based on a relational database management system to ensure structured storage, data integrity, and efficient querying.

#### 3.1.2 *End user characteristics*

The Gheras platform serves a diverse group of users with varying levels of technical expertise, which directly influences its design and usability considerations.

- **Parents or Guardians** represent the primary users of the system. They are assumed to possess basic digital literacy and the ability to interact with standard web applications. Their primary objectives include managing child profiles, generating personalized stories, and monitoring behavioral progress. The system interface is designed to minimize complexity and guide users through all major operations.
- **Children**, within the target age range of 4 to 9 years, are the primary consumers of the generated story content. Due to their limited reading and technical skills, the child-facing interface emphasizes simplicity, visual engagement, and intuitive interaction. Story consumption features are designed to require minimal or no adult assistance.
- **System Administrators** are assumed to have a higher level of technical familiarity. Their responsibilities include managing user accounts, configuring system settings, monitoring system performance, and maintaining data integrity. The administrative interface prioritizes clarity and control rather than visual engagement.

### 3.2 Architectural Strategies

#### 3.2.1 *Algorithm/models to be used*

The Gheras platform employs a combination of artificial intelligence algorithms to generate personalized, behavior-focused children’s stories. These algorithms are selected based on their suitability for natural language understanding, content generation, and multimedia synthesis.

For story text generation, the system integrates an external Large Language model (LLM) model to produce grammatically correct, age-appropriate, and culturally relevant Arabic story content. The fine-tuning process enables the model to incorporate child-specific attributes, selected behavioral traits, and moral guidance into the generated narratives.

For illustration generation, the system integrates an external text-to-image generation service capable of producing visual content aligned with the story context. The model generates illustrations based on structured prompts derived from the story text and child profile attributes, ensuring visual consistency and personalization.

For audio narration, the system integrates an external text-to-speech service provided through a standardized API. This approach allows the system to convert generated story text into natural-sounding speech while avoiding the computational overhead of training and maintaining an in-house speech synthesis model.

Together, these algorithms operate in a coordinated pipeline, where the output of one stage serves as input to the next. This pipeline-based strategy ensures coherence between text, images, and audio, resulting in a unified and engaging storytelling experience for children.

Together, these algorithms operate in a coordinated pipeline, where the output of one stage serves as input to the next. This pipeline-based strategy ensures coherence between text, images, and audio, resulting in a unified and engaging storytelling experience for children.

### ***3.2.2 Reuse of existing software components***

The development of the Gheras platform did not involve direct reuse of source code or software components from existing commercial storytelling systems such as TinyTeller or Storypark. These platforms were reviewed during the analysis phase primarily to understand common architectural patterns, functional capabilities, and design practices used in AI-based storytelling systems.

Conceptual reuse was applied in the form of adopting well-established design ideas, such as modular content generation pipelines, separation between content creation and content presentation, and the use of configurable story parameters to support personalization. These concepts informed the architectural and functional design of Gheras while ensuring that all implementation components were developed independently.

Additionally, the integration of external services, such as text-to-speech APIs, follows standard software reuse practices by leveraging publicly available and documented interfaces rather than developing equivalent functionality from scratch. This approach reduces development complexity while maintaining system reliability and scalability.

### ***3.2.3 Project management strategies***

To navigate the complexities of building an AI-driven platform, our team will adhere to a disciplined project management framework designed for clarity, collaboration, and adaptability.

- **Communication Via:**

- **Daily Asynchronous Standups:** Team members will post daily updates in the WhatsApp channel, outlining their progress, next steps, and any blockers.
- **Weekly Synchronous Meetings:** A weekly voice meeting is held with the supervisor Dr. Arwa, via Webex channel, to review the previous week's work, address major challenges, and collaboratively plan the tasks for the upcoming week.



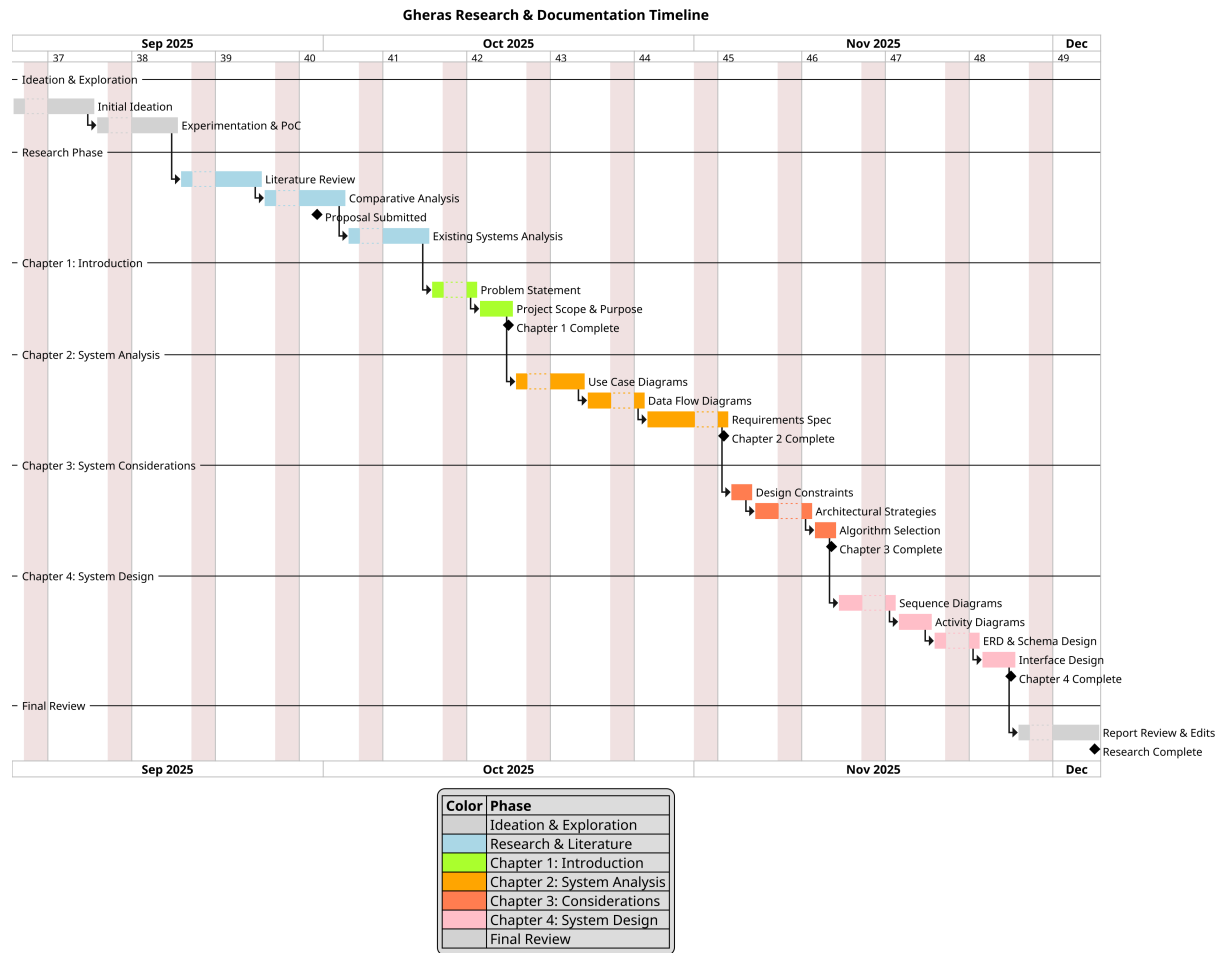


Figure 3.1. Research and Documentation Timeline (Gantt Chart)

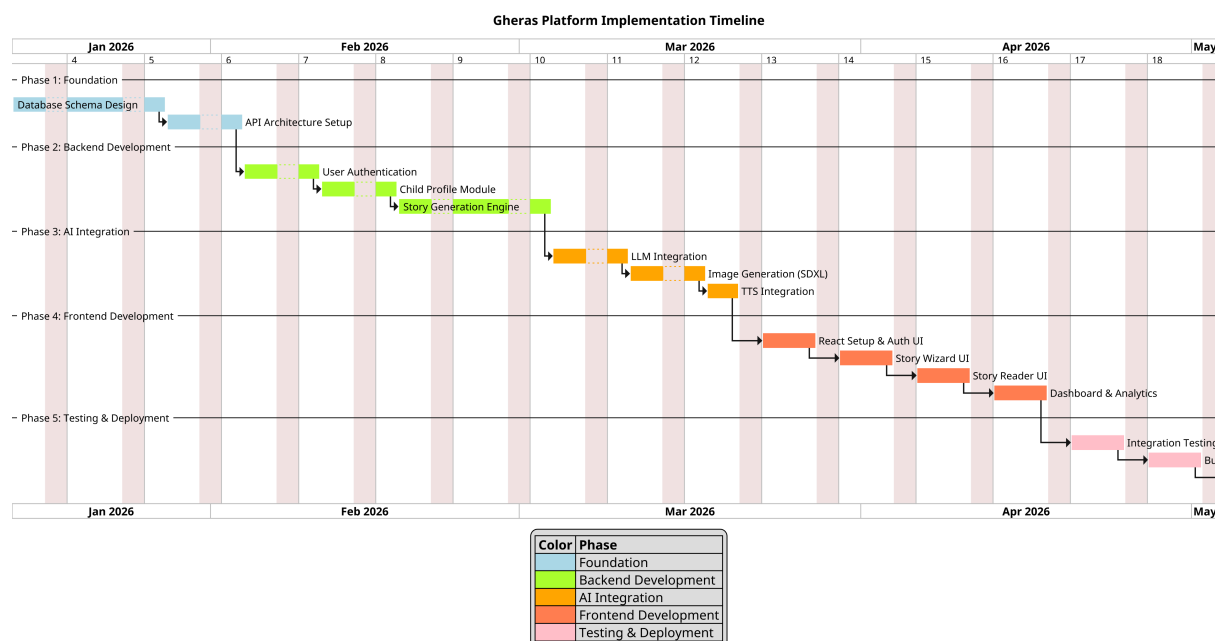


Figure 3.2. Gheras Platform Implementation Timeline (Gantt Chart)

### 3.2.4 Development method

In this project, the Waterfall Model was adopted because its steps divide the project into successive stages, where each stage must be completed entirely before starting the next. The information for each stage is based on the previous stage.

The project work is divided between two academic semesters:

- **The First Semester (Graduation Project I):** Includes the stage of analysis and design.
  1. Requirements analysis, in which the project requirements and the functions that the system provides to the user are determined.
  2. System & interfaces design, where the system architecture was designed and created based on the previous stage. Activity and sequence diagrams were drawn to illustrate the flow of each process between the user and the system.

By the end of this phase, the user interface prototype was also designed according to the system's specifications.

- **The second Semester (Graduation Project II):** Includes the stages of Design interfaces, Implementation, and Testing.

Here, the remaining stages will be completed, namely:

1. Implementation, where designs are converted into an executable application by actually coding the system.
2. Testing and Evaluation.
3. Results and Analysis.

## 4 SYSTEM DESIGN

### 4.1 Detailed System Design

#### 4.1.1 *Sequence Diagram*

Sequence diagrams are used to model the dynamic interactions between system actors, application components, external AI services, and databases during the execution of key system functionalities. In the Gheras platform, the sequence diagrams focus on illustrating how requests are processed across the frontend interface, backend APIs, AI orchestration layer, and persistent storage. Each diagram demonstrates how responsibilities are distributed across system components while maintaining data consistency, modularity, and performance efficiency.

This sequence diagram, shown in Figure 4.1, illustrates the user authentication process, including both registration and login operations. The interaction begins when the Parent/Guardian submits credentials through the frontend interface, which forwards the request to the authentication API. During registration, the system validates the input data, checks for existing accounts, and creates a new user record in the database with securely hashed credentials. In the login process, the system verifies the provided credentials against stored records. Upon successful authentication, an authorization token is generated and returned to the frontend for subsequent authenticated requests. This process ensures secure access control and proper session management within the platform.

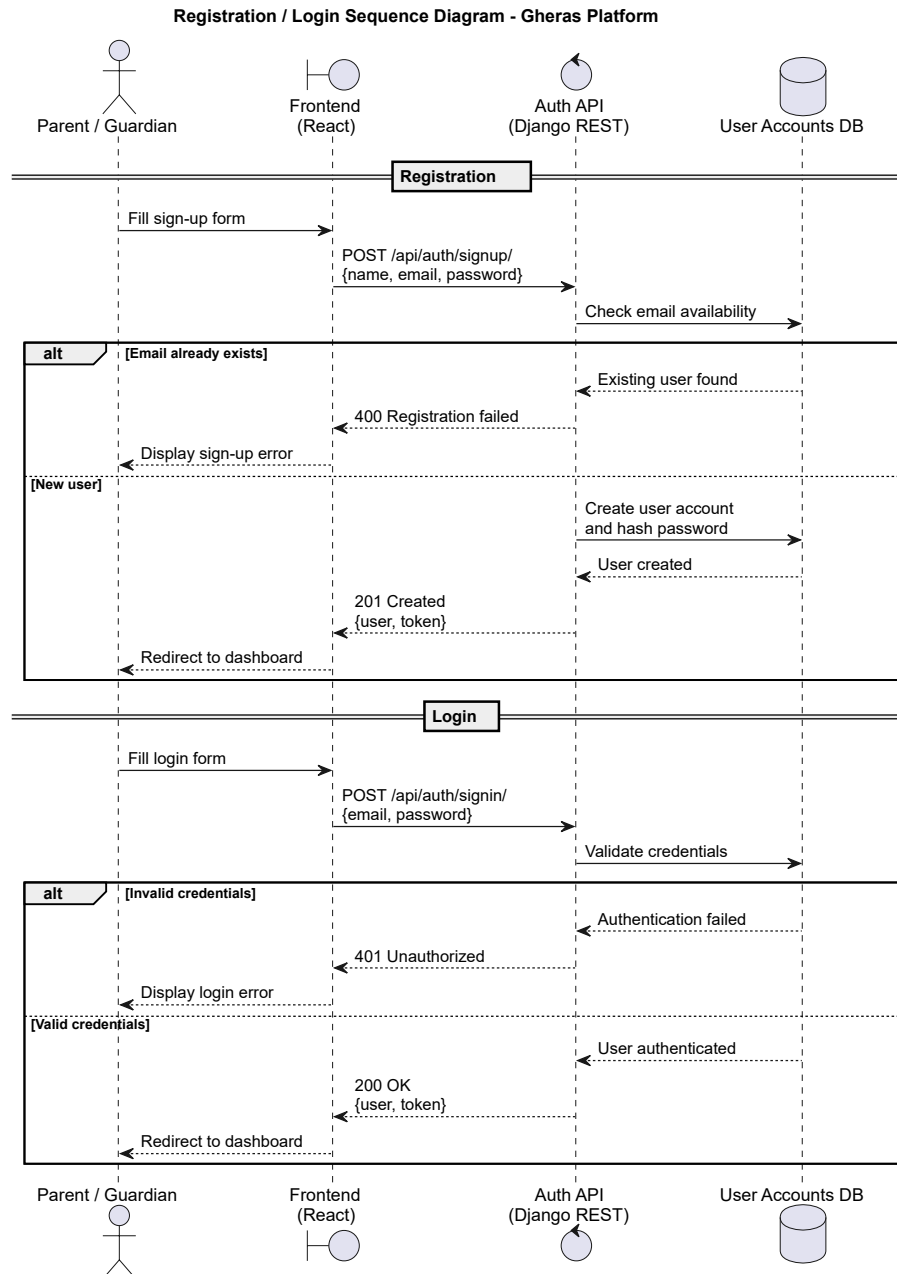


Figure 4.1. Sequence Diagram – Registration/login

This sequence diagram, shown in Figure 4.1, illustrates the user authentication process, including both registration and login operations. The interaction begins when the Parent/Guardian submits credentials through the frontend interface, which forwards the request to the authentication API. During registration, the system validates the input data, checks for existing accounts, and creates a new user record in the database with securely hashed credentials. In the login process, the system verifies the provided credentials against stored records. Upon successful authentication, an authorization token is generated and returned to the frontend for subsequent authenticated requests. This process ensures secure access control and proper session management within the platform.

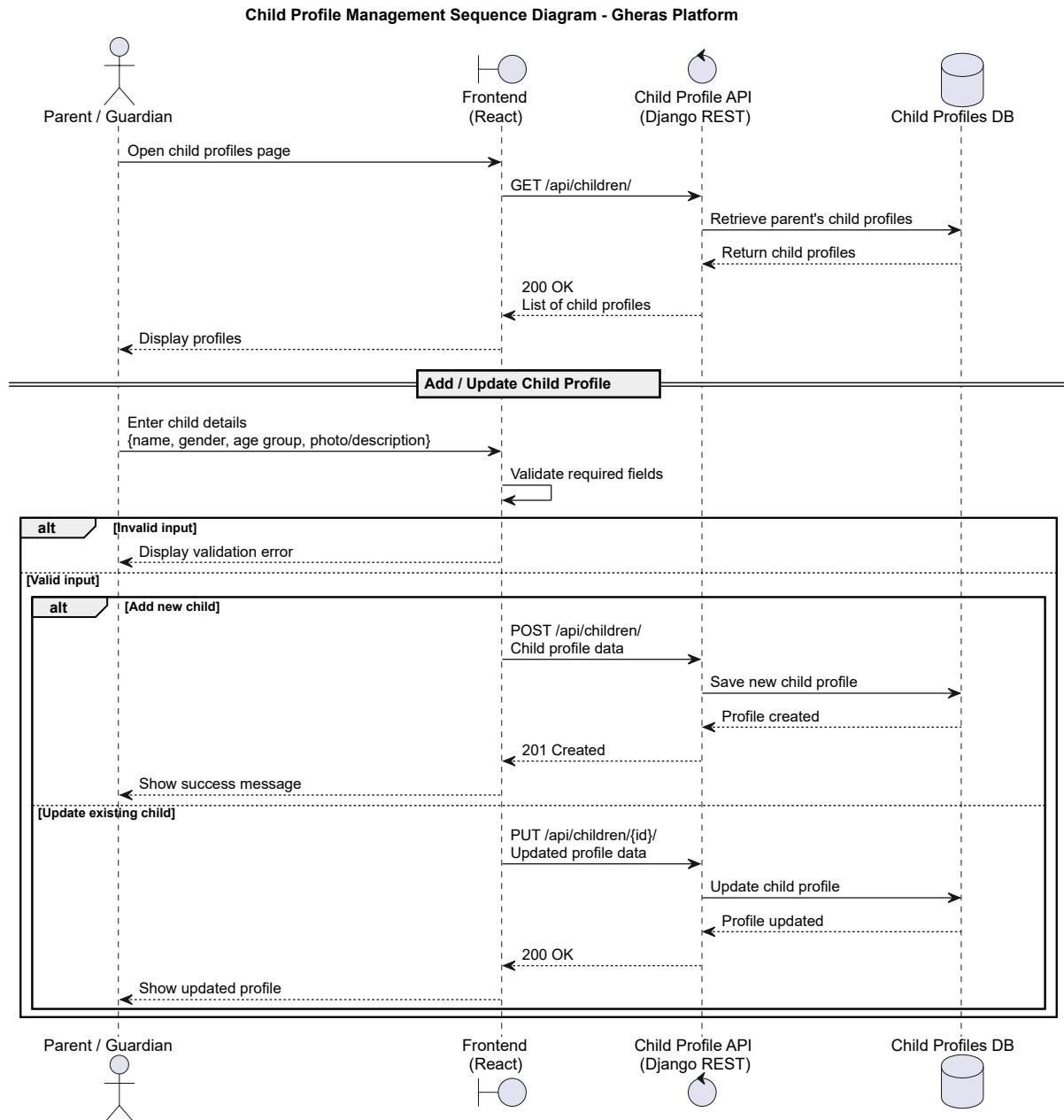


Figure 4.2. Sequence Diagram – child profile management

This sequence diagram, shown in Figure 4.2, represents the process of managing child profiles within the system. The Parent/Guardian interacts with the frontend to view, create, or update a child profile. The frontend communicates with the backend API, which retrieves or modifies the relevant data in the Child Profiles database. Input validation is performed before any operation to ensure data integrity. The system supports both creation of new profiles and updating of existing ones, enabling personalized content generation based on child-specific attributes such as age group, gender, and appearance. This interaction ensures that user-specific data is consistently maintained and readily accessible for downstream processes.

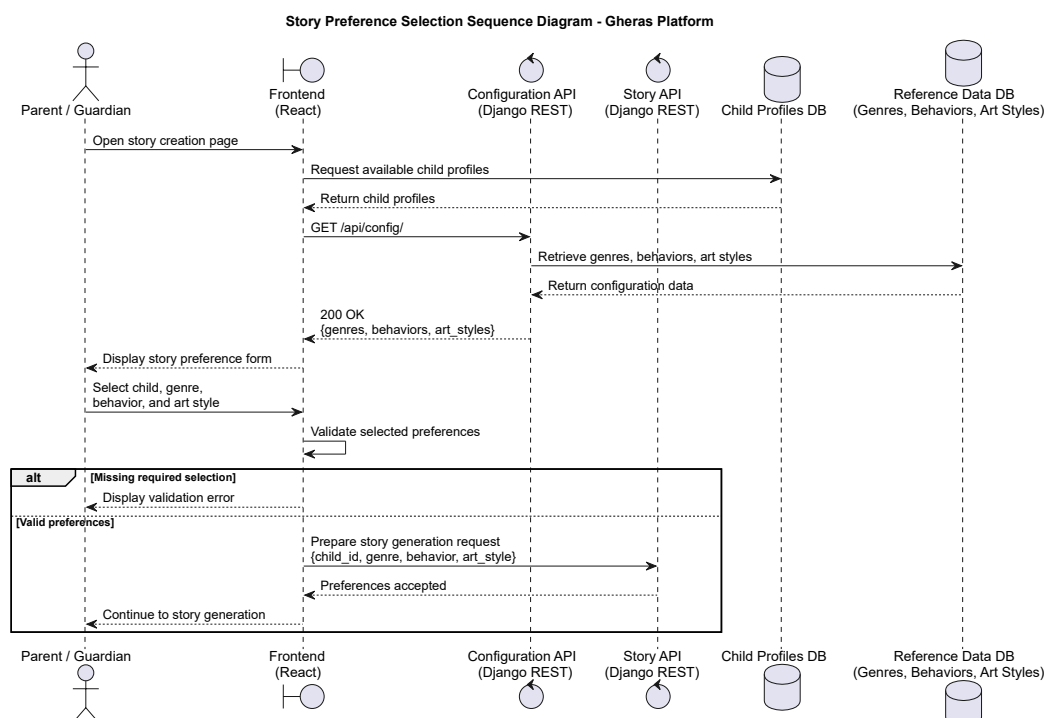


Figure 4.3. Sequence Diagram – Story Preference Selection

This sequence diagram, shown in Figure 4.3, illustrates how story preferences are selected prior to content generation. The Parent/Guardian initiates the process by accessing the story creation interface, where the system retrieves available configuration data such as genres, behavioral themes, and art styles from the backend. After selecting the desired preferences and a target child profile, the frontend validates the input and prepares a structured request. This request is then sent to the story generation module. This step ensures that all required parameters are explicitly defined, enabling accurate and context-aware story generation in subsequent stages.

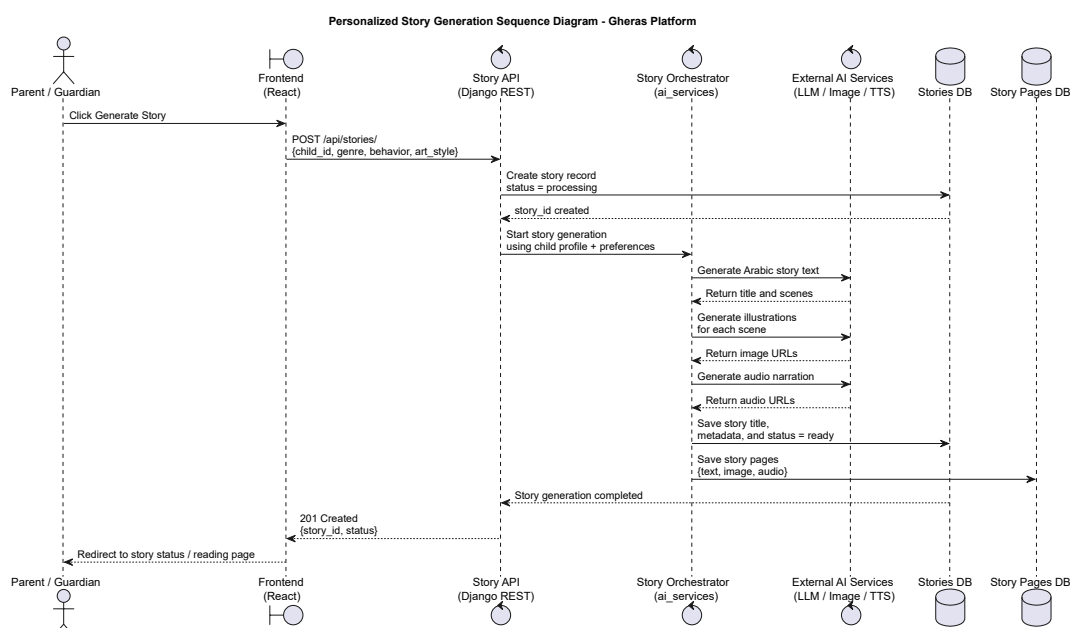


Figure 4.4. Sequence Diagram – Story Generation

This sequence diagram, shown in Figure 4.4, describes the core functionality of the Gheras platform: generating personalized stories. Upon receiving a valid request, the backend creates an initial story record with a processing status and delegates the generation task to the AI orchestration component. The orchestrator communicates with external AI services to generate story text, illustrations, and audio narration. The generated outputs are then aggregated, structured into pages, and stored in the database. Once completed, the story status is updated to indicate readiness. This process demonstrates the integration of multiple AI services within a coordinated pipeline, ensuring high-quality and personalized content generation.

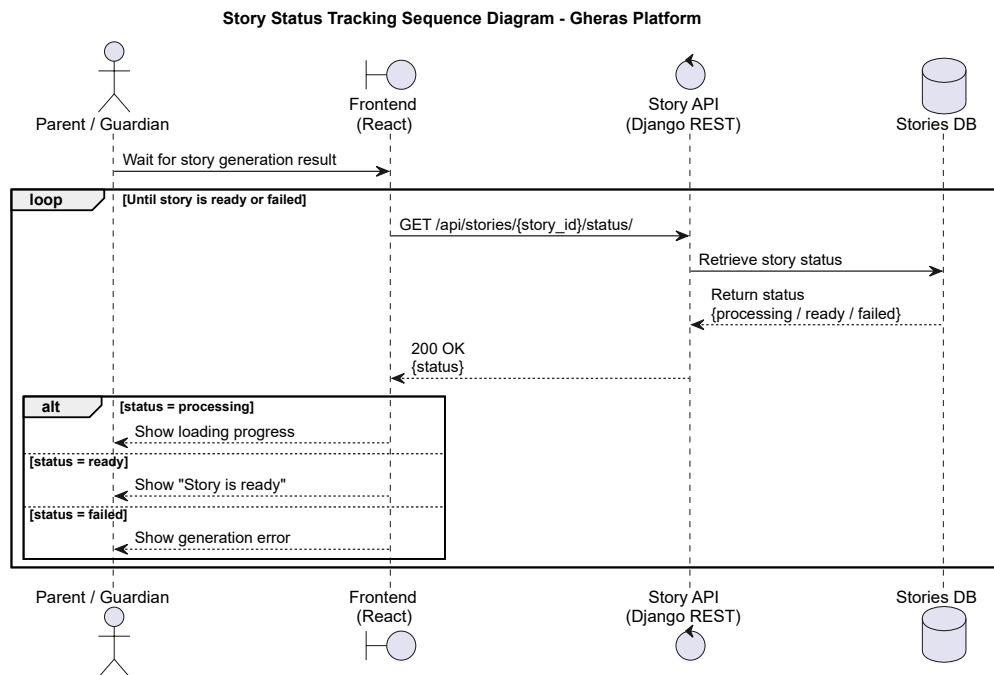


Figure 4.5. Sequence Diagram – Story Generation Status Tracking

This sequence diagram, shown in Figure 4.5, illustrates how the system tracks the status of story generation. Since story creation involves asynchronous processing, the frontend periodically sends requests to the backend to retrieve the current status of a story. The backend queries the Stories database and returns the status, which may indicate processing, completion, or failure. Based on the response, the frontend updates the user interface accordingly. This mechanism ensures a responsive user experience while handling long-running AI operations efficiently.

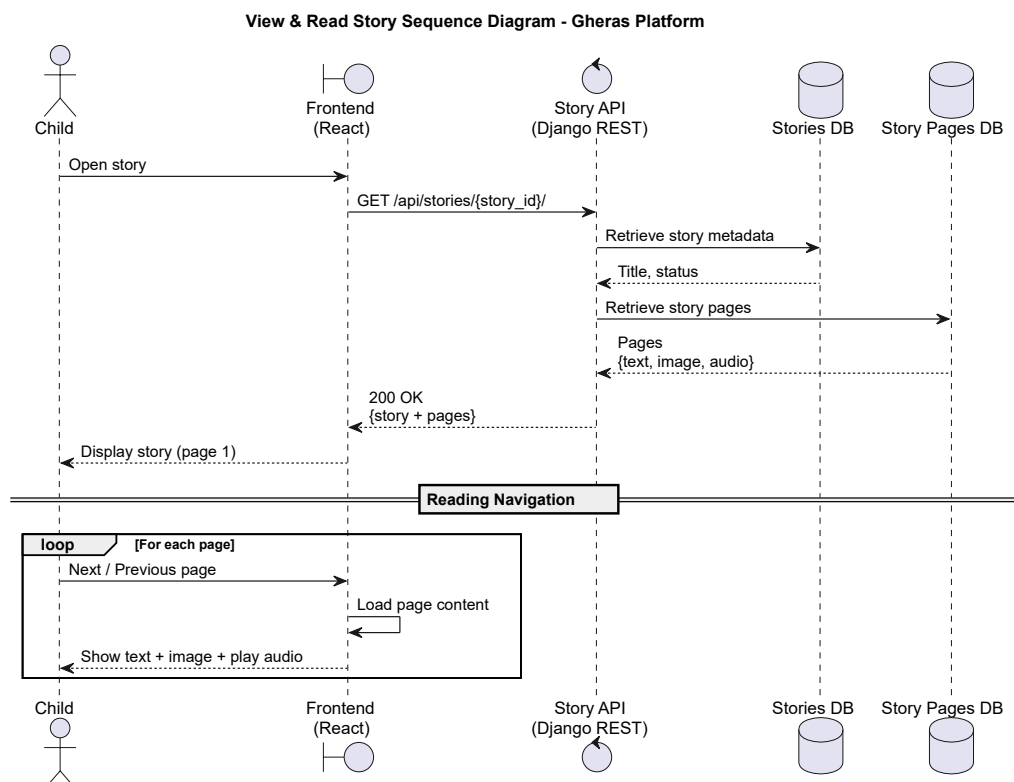


Figure 4.6. Sequence Diagram – Reading a Generated Story

This sequence diagram, shown in Figure 4.6, represents the process of accessing and reading a generated story. The Child or Parent requests a specific story through the frontend, which sends a retrieval request to the backend API. The backend fetches the story metadata and associated pages from the database, including text, images, and audio references. The complete content is then returned to the frontend for rendering. The user can navigate between pages, with each interaction handled locally on the frontend. This design ensures efficient content delivery and smooth reading experience.

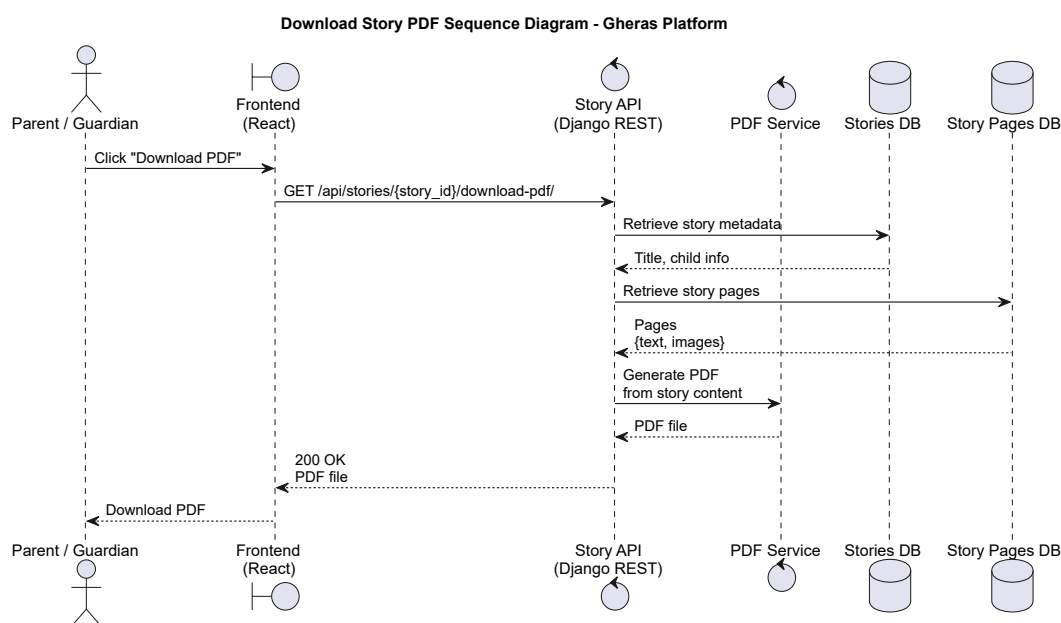


Figure 4.7. Sequence Diagram – Story PDF Export



This sequence diagram, shown in Figure 4.7, illustrates the process of exporting a story as a PDF document. Upon user request, the frontend sends a download request to the backend, which retrieves the story content from the database. The backend then invokes a PDF generation service to format the story pages into a structured document. Once the PDF is generated, it is returned to the frontend for download. This functionality enables offline access and enhances usability by allowing users to retain a permanent copy of the generated story.

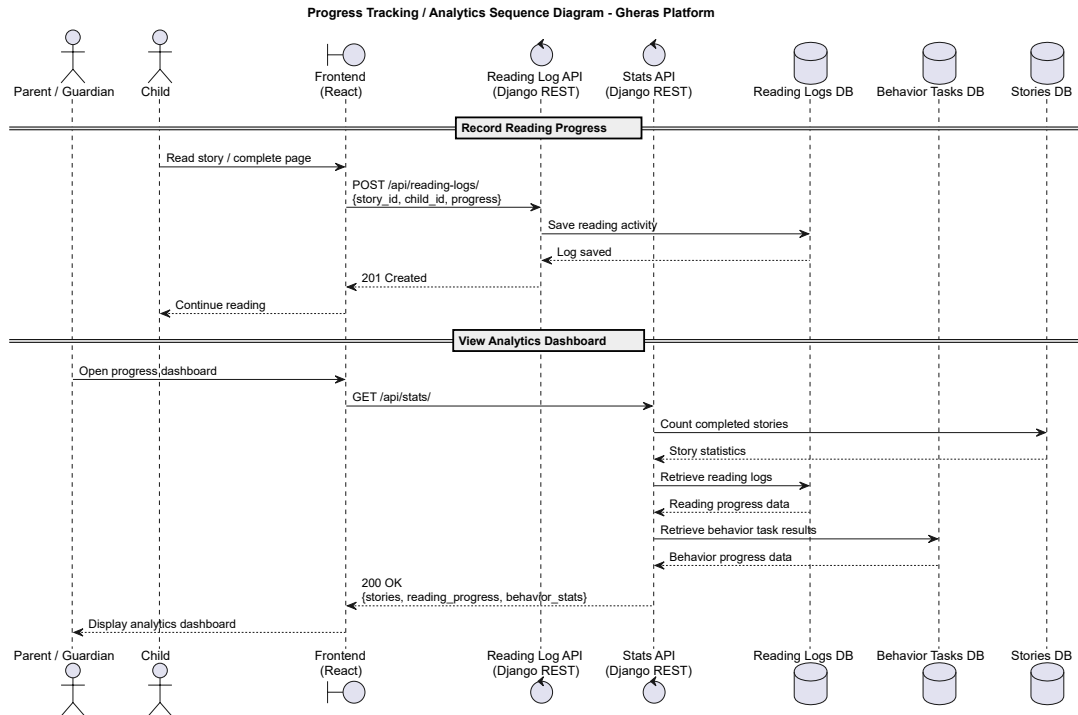


Figure 4.8. Sequence Diagram – Progress Tracking and Analytics

This sequence diagram, shown in Figure 4.8, demonstrates how user engagement and learning progress are tracked and analyzed. As the Child interacts with stories, reading activities are recorded and stored in the database. Additionally, behavioral task outcomes are logged for further analysis. When the Parent/Guardian accesses the analytics dashboard, the frontend requests aggregated data from the backend. The system retrieves and processes information from multiple data sources, including reading logs, story records, and behavioral data, and returns a summarized report. This functionality provides meaningful insights into the child's engagement and developmental progress.

#### 4.1.2 Activity Diagram

Activity diagrams are used to illustrate the dynamic behavior of the Gheras system by modeling the flow of actions, decisions, and parallel activities involved in key system processes. Unlike sequence diagrams, which emphasize message exchanges between components, activity diagrams focus on **workflow logic**, decision points, and user-system interactions from a process-oriented perspective.

The following activity diagrams are organized according to the natural operational flow of the system.

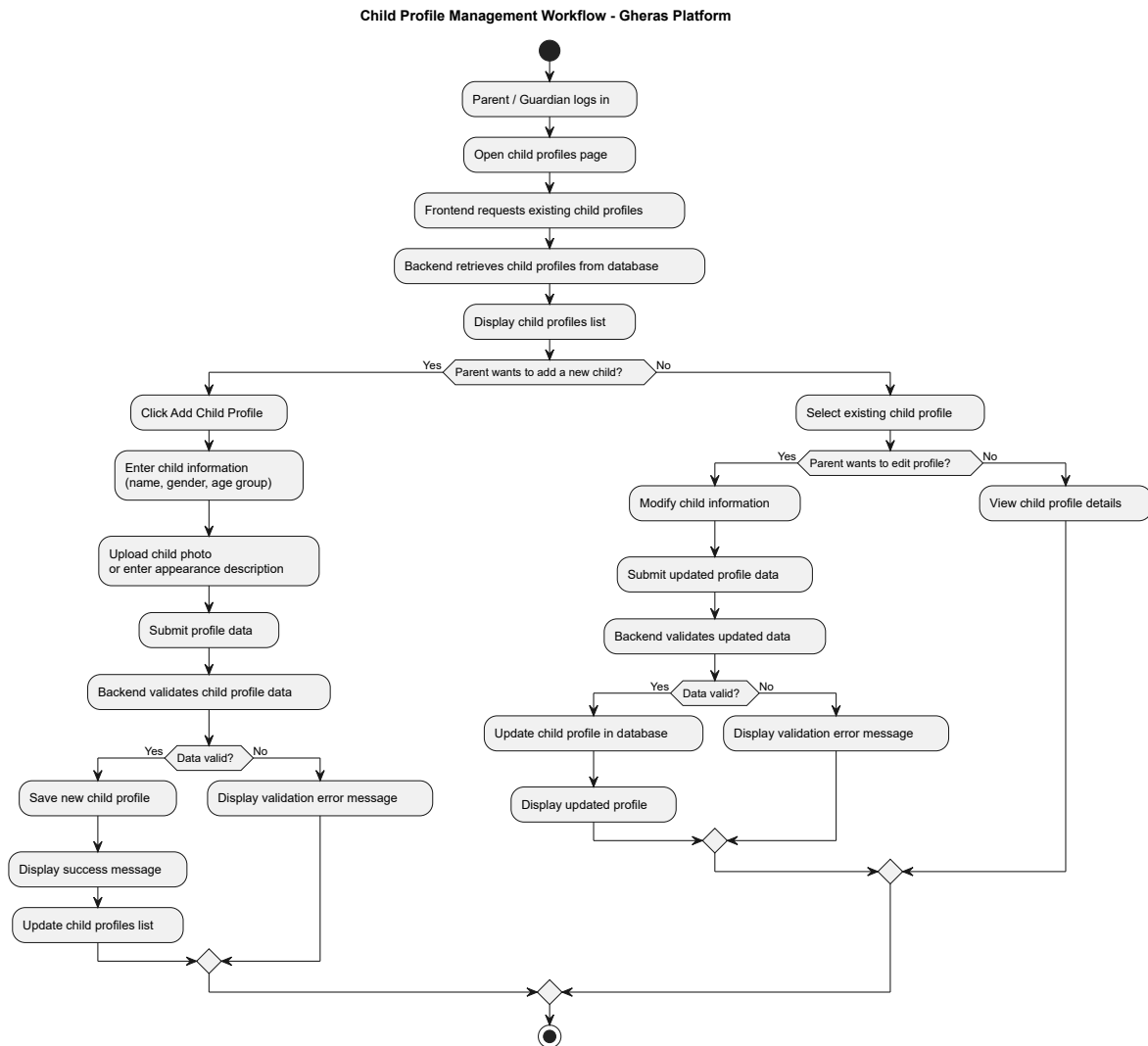


Figure 4.9. Child Profile Management Workflow

This activity diagram, shown in Figure 4.9, depicts the workflow for managing child profiles. The process starts when the Parent/Guardian accesses the profile management interface and views the list of existing child profiles. The user can either create a new profile or modify an existing one. In both cases, the system performs validation checks to ensure that the input data is complete and accurate. If the data is valid, the system saves or updates the profile in the database and provides confirmation feedback. Otherwise, an error message is displayed. The diagram also accounts for scenarios where the user simply views profile details without making changes. This workflow ensures proper management of personalized data, which is essential for tailored story generation.

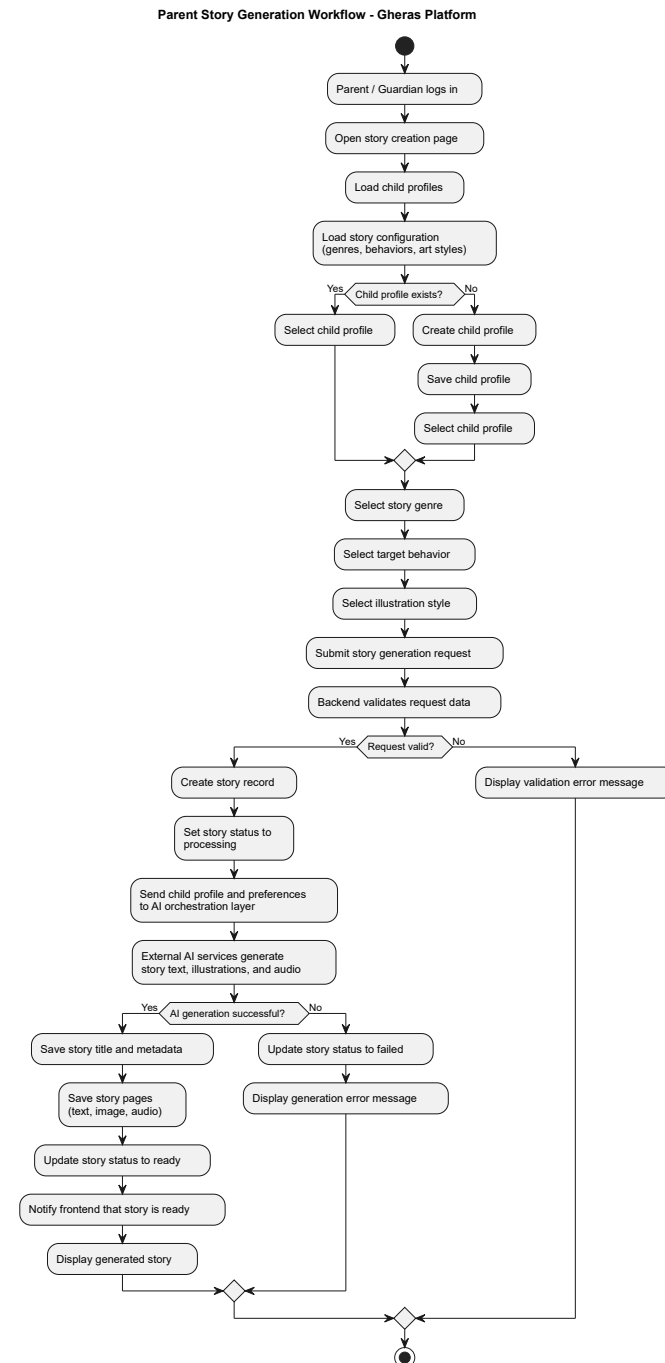


Figure 4.10. Parent Story Generation Workflow in the Gheras Platform

This activity diagram, shown in Figure 4.10, illustrates the complete workflow for generating a personalized story within the Gheras platform. The process begins when the Parent/Guardian accesses the story creation interface and selects or creates a child profile. The user then specifies the story preferences, including genre, target behavior, and illustration style. After submitting the request, the system validates the input and initiates the story generation process. The workflow includes decision points to handle validation errors and AI generation outcomes. If the request is valid, the system invokes external AI services to generate the story text, illustrations, and audio narration. Upon successful completion, the generated content is stored, and the story status is updated to “ready.” In case of failure, an appropriate error message is displayed. This workflow captures the core functionality of the system and demonstrates how multiple components are orchestrated to produce personalized content.

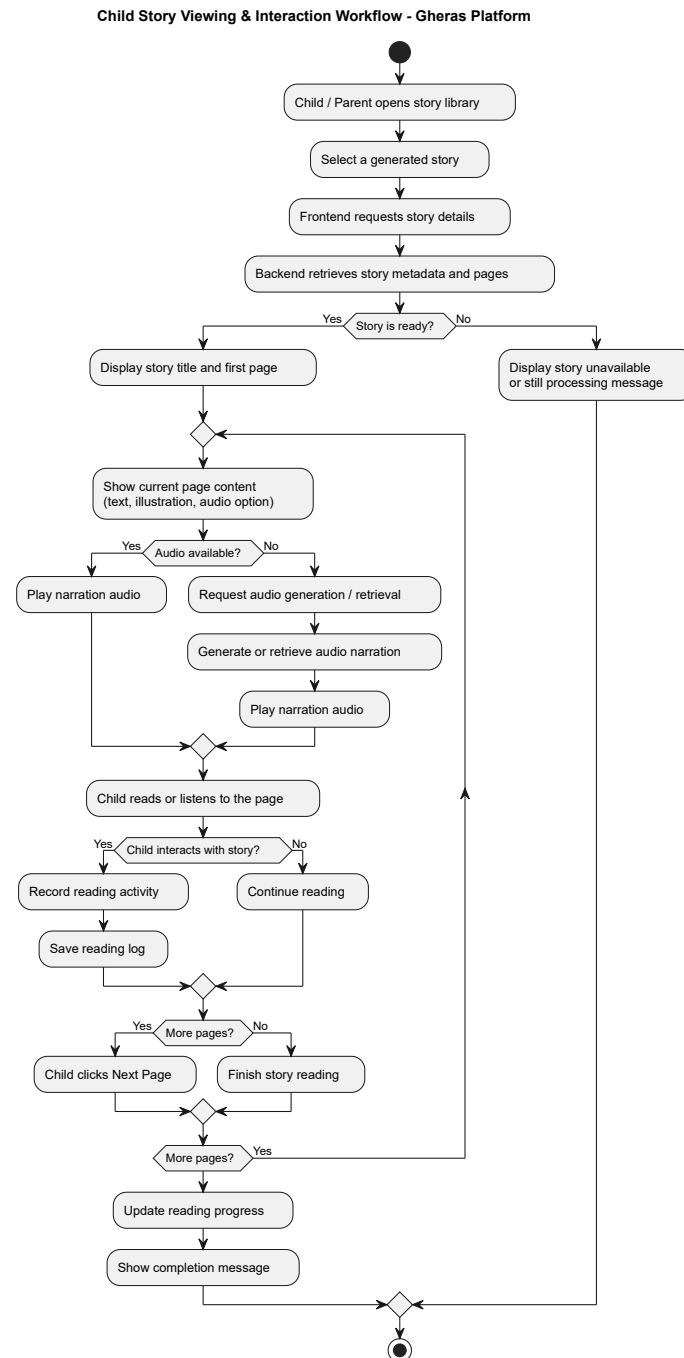


Figure 4.11. Child Story Viewing and Interaction Workflow

This activity diagram, shown in Figure 4.11, represents the process of viewing and interacting with a generated story. The workflow starts when the Child or Parent selects a story from the library. The system retrieves the story content and checks its availability status. If the story is ready, the first page is displayed, and the user can navigate through subsequent pages. Each page may include text, illustrations, and optional audio narration. The diagram includes decision points for audio availability, where the system either plays existing audio or triggers audio generation if needed. Additionally, user interactions such as page navigation and reading engagement are captured, along with the recording of reading activity. This workflow ensures a smooth and interactive reading experience while supporting dynamic content loading and engagement tracking.

Progress Tracking &amp; Behavioral Analytics Workflow - Gheras Platform

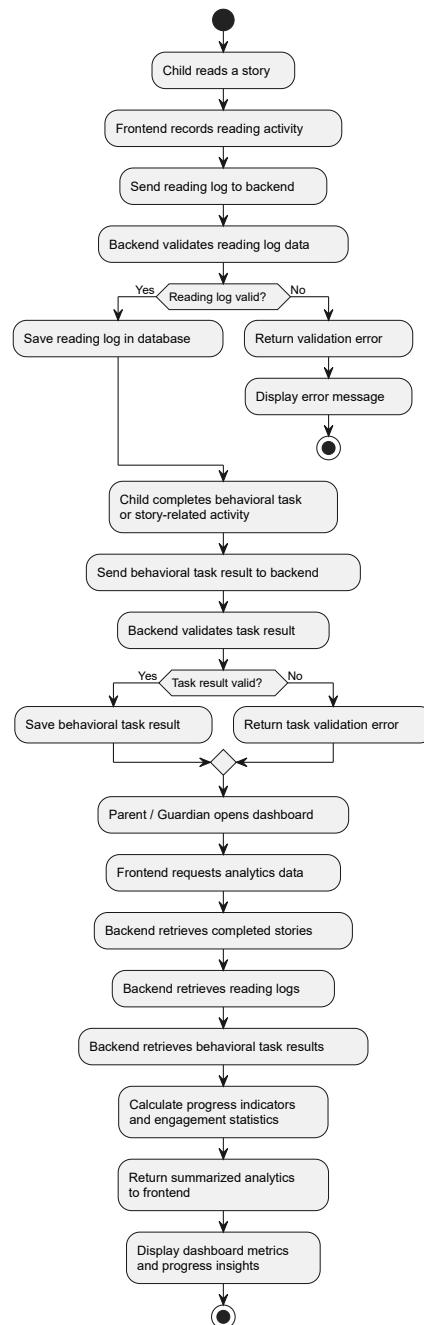


Figure 4.12. Progress Tracking and Behavioral Analytics Workflow

This activity diagram, shown in Figure 4.12, illustrates how the system tracks user engagement and generates analytical insights. The process begins when the Child interacts with a story, triggering the recording of reading activity. The system validates and stores this data as part of the reading logs. Additionally, behavioral task outcomes are submitted and validated before being stored. When the Parent/Guardian accesses the analytics dashboard, the system retrieves data from multiple sources, including reading logs, story records, and behavioral tasks. It then processes this data to compute progress indicators and engagement metrics. The final output is presented in a structured dashboard, providing meaningful insights into the child's learning and behavioral development. This workflow highlights the integration of data collection and analysis within the system.

Download Story PDF Workflow - Gheras Platform

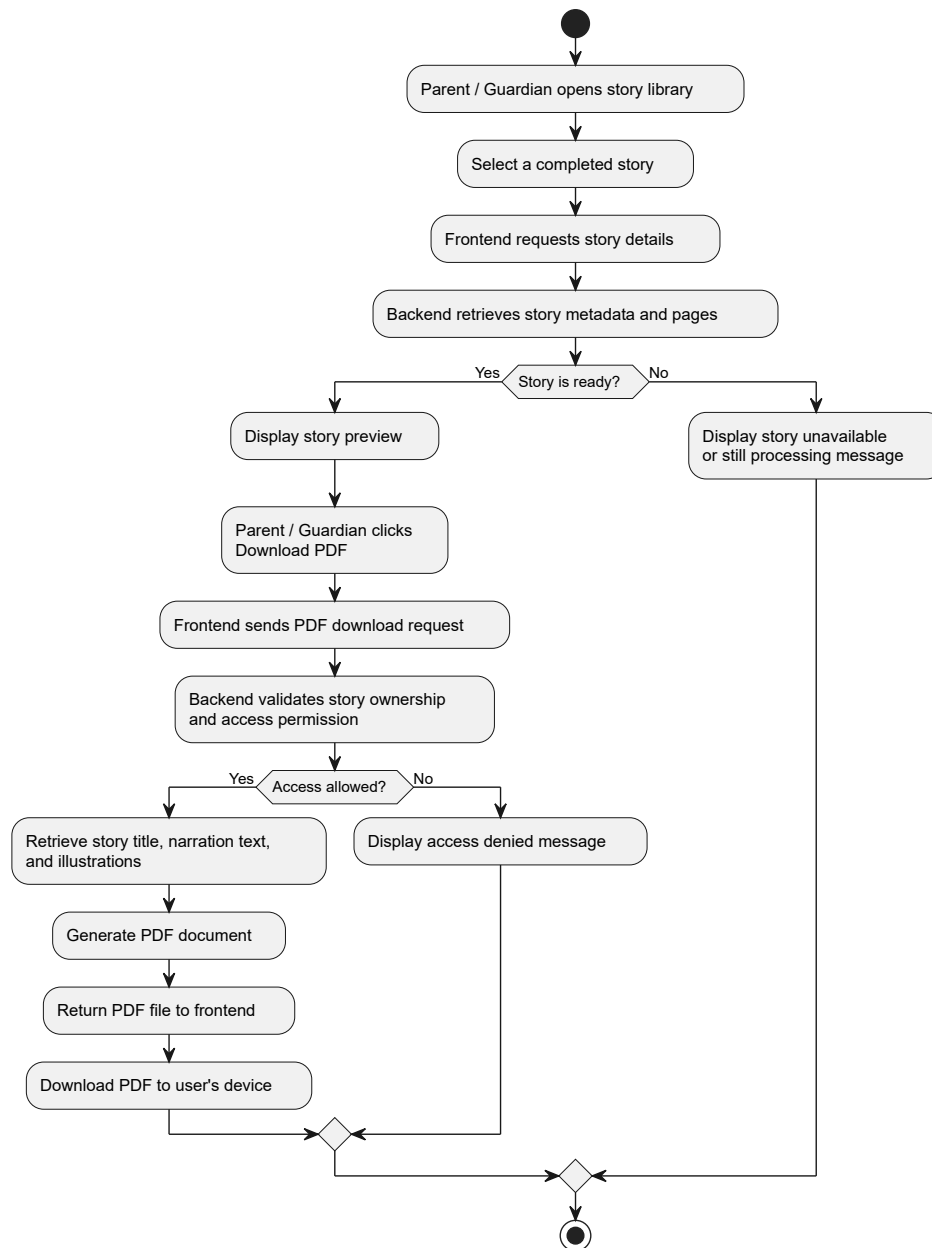


Figure 4.13. Story PDF Download Workflow

This activity diagram, shown in Figure 4.13, illustrates the process of exporting a generated story as a PDF document. The workflow begins when the Parent/Guardian selects a completed story and requests to download it. The system verifies that the story is available and checks access permissions. If the conditions are satisfied, the system retrieves the story content, including text and illustrations, and generates a formatted PDF document. The generated file is then returned to the user for download. If the story is not ready or access is denied, appropriate messages are displayed. This workflow emphasizes content reuse and supports offline accessibility for users.

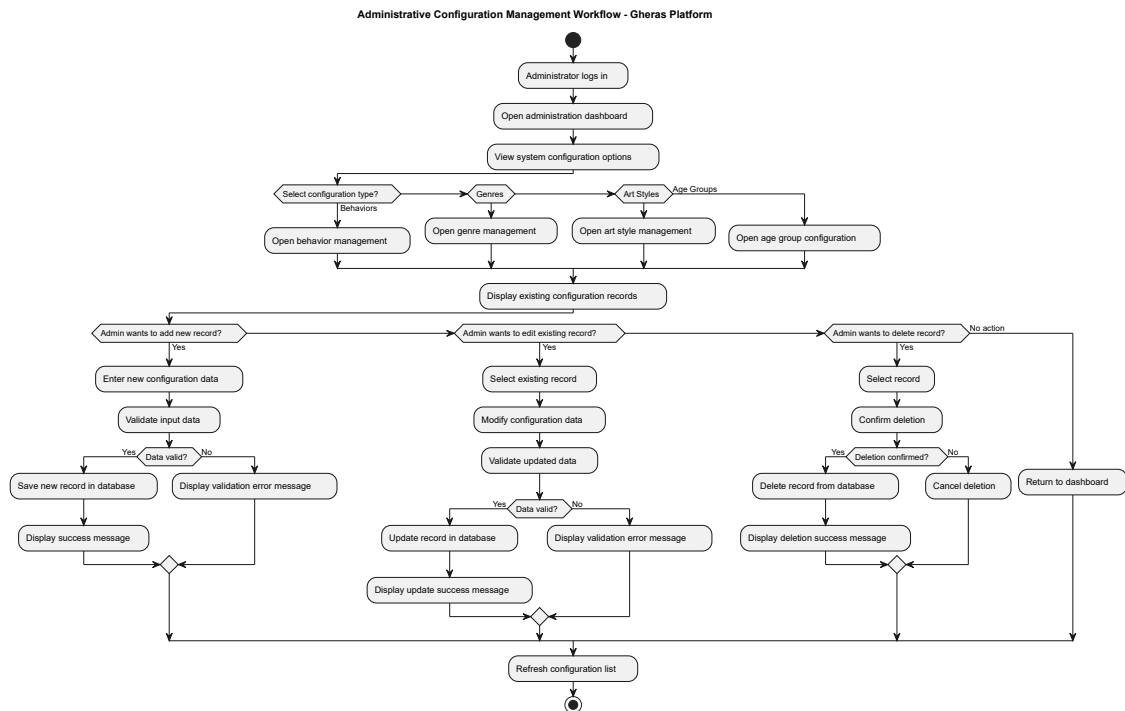


Figure 4.14. Administrative Configuration Management Workflow

This activity diagram, shown in Figure 4.14, represents the workflow for managing system configuration data by the administrator. The process begins with the administrator accessing the dashboard and selecting a configuration category, such as behaviors, genres, art styles, or age groups. The system displays existing records and allows the administrator to perform create, update, or delete operations. Each operation involves validation steps to ensure data consistency. Decision points handle different administrative actions, including confirmation for deletion. After successful execution, the system updates the database and refreshes the displayed data. This workflow ensures that the system remains flexible and adaptable through centralized configuration management.

## 4.2 Data Storage Design

### 4.2.1 Entity Relationships Diagram

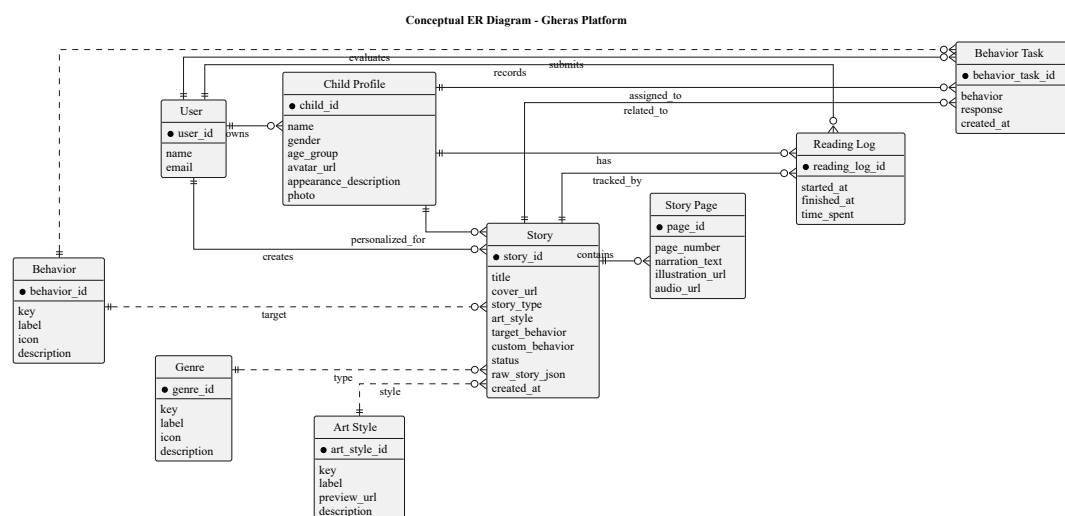


Figure 4.15. conceptual ER Diagram

This conceptual Entity–Relationship (ER) diagram, shown in Figure 4.15, presents a high-level representation of the core data entities and their relationships within the Gheras platform. The diagram is structured to reflect the system’s primary functional domains, including user management, personalized story generation, content delivery, and behavioral tracking. At the center of the model, the User entity represents the parent or guardian, who owns one or more Child Profiles and initiates the creation of personalized Stories. Each story is associated with a specific child and is composed of multiple Story Pages, which store the narrative text, illustrations, and audio references.

The diagram further captures user interaction and engagement through the Reading Log entity, which records reading sessions and progress, and the Behavior Task entity, which stores responses related to targeted behavioral outcomes. These entities are linked to both the user and the child, as well as to the corresponding story, enabling comprehensive tracking of usage and developmental progress. Additionally, the model incorporates reference data entities, including Behavior, Genre, and Art Style, which define the configurable attributes used during story generation. Although these are represented conceptually as related entities, they may be implemented in the system as configurable parameters rather than strict relational constraints.

Overall, this conceptual ER diagram abstracts the underlying database design while clearly illustrating the relationships between key entities. It supports a structured understanding of how data flows through the system, ensuring consistency, extensibility, and alignment with the platform’s personalized and AI-driven functionality.



## 4.3 Architectural Design

### 4.3.1 Model View Controller (MVC)

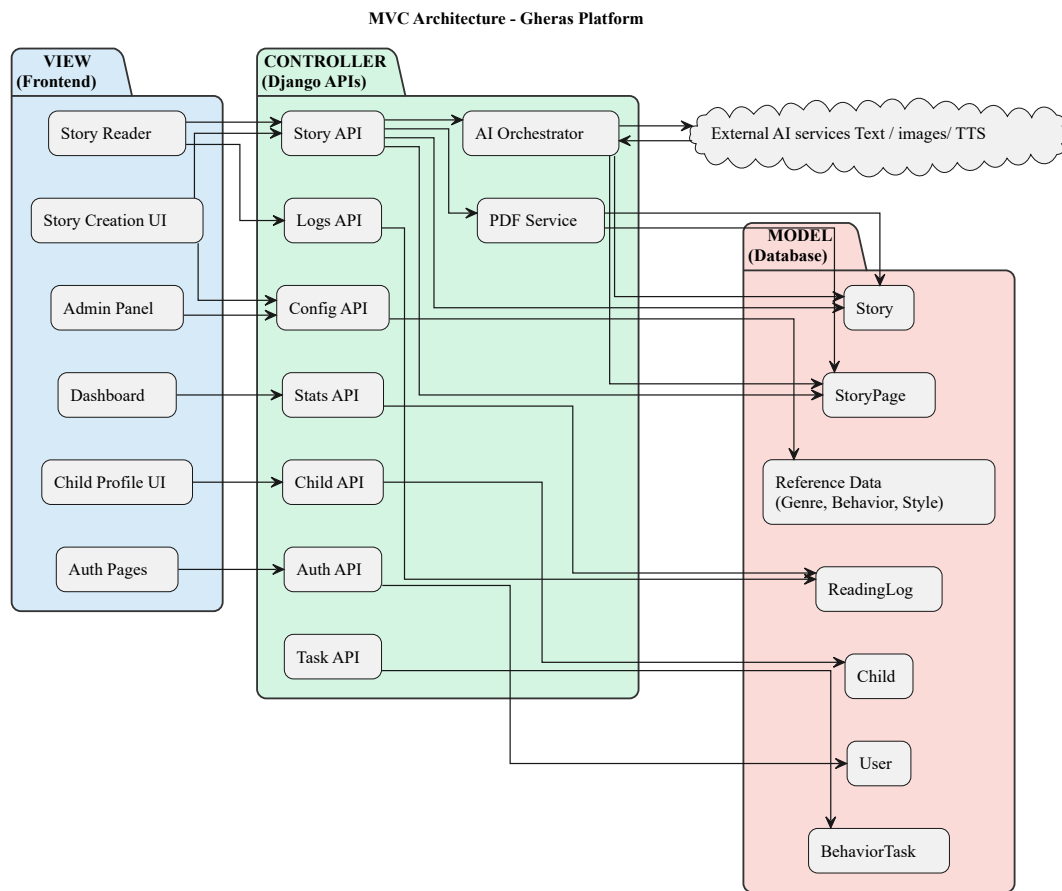


Figure 4.16. Model–View–Controller (MVC) Architecture

The Gheras platform adopts the Model–View–Controller (MVC) architectural pattern, as illustrated in Figure 4.16, to achieve a clear separation of concerns, improved maintainability, and scalable system design. In this architecture, the View layer is implemented as a React-based frontend that provides user-facing interfaces, including authentication pages, child profile management, story creation and reading interfaces, progress dashboards, and administrative panels. This layer is responsible for capturing user input and presenting system outputs, while interacting with the backend exclusively through RESTful API calls, ensuring a decoupled and responsive user experience.

The Controller layer is realized using a Django REST Framework backend, which serves as the central coordination unit of the system. It exposes a set of well-defined APIs, including authentication, child management, story processing, configuration, analytics, logging, and behavioral task services. These controllers handle request validation, business logic execution, and data flow management between the frontend and the underlying data models. In addition, a dedicated AI orchestration component integrates external AI services to support story generation, coordinating the generation of textual content, illustrations, and audio narration through a structured pipeline. Auxiliary services, such as PDF generation, are also managed within this layer to support content export functionalities.

The Model layer encapsulates the system's core data structures and persistence logic using Django's Object–Relational Mapping (ORM). The application defines several domain models, including Child, Story, StoryPage, ReadingLog, BehaviorTask, Behavior, Genre, and ArtStyle, while user accounts are handled through Django's configured authentication user model. These models organize the platform's

persistent data, define relationships between users, child profiles, generated stories, story pages, reading logs, and behavioral task responses, and support structured retrieval and updates through the backend APIs.

## 4.4 Interface design

### 4.4.1 Windows Navigation Diagram

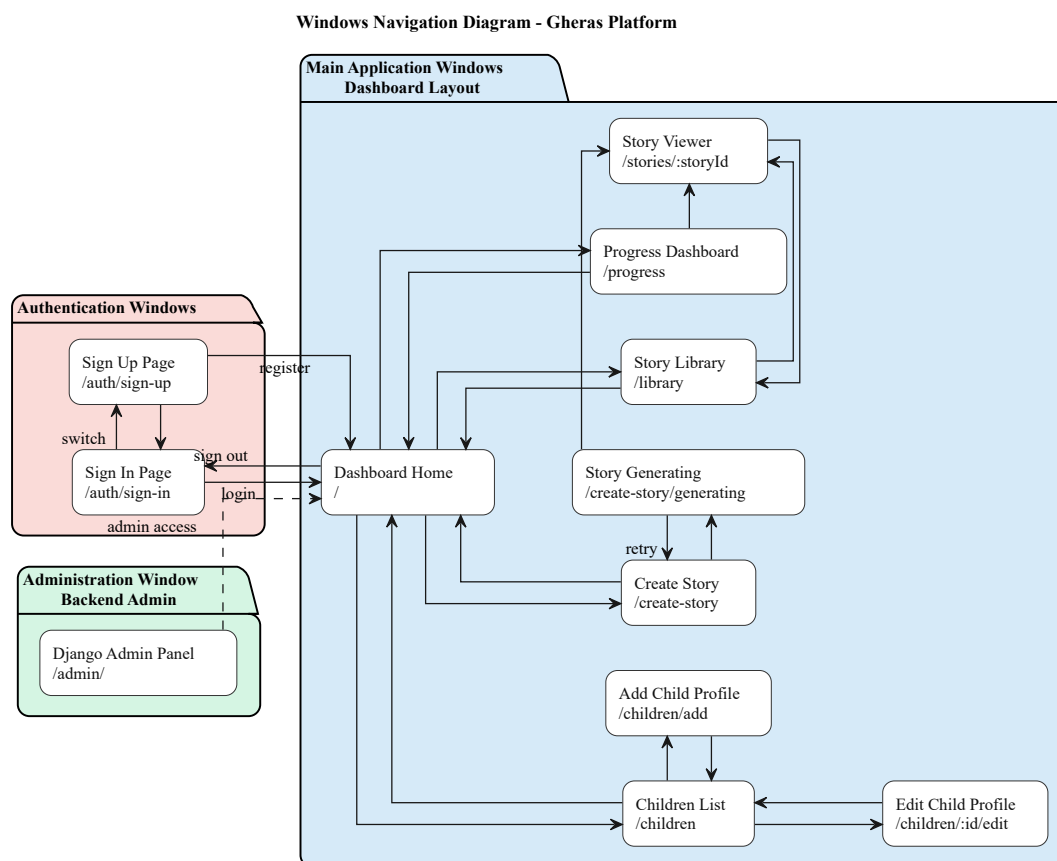


Figure 4.17. Windows Navigation Diagram

This Windows Navigation Diagram, shown in Figure 4.17, illustrates the structural organization of user interface windows and the navigation flow within the Gheras platform. The diagram is organized into three main sections: authentication windows, main application windows, and administrative access. The authentication layer includes the Sign In and Sign Up pages, which allow users to switch between account creation and login. Upon successful authentication, users are redirected to the main dashboard, which serves as the central entry point to all primary system functionalities.

Within the main application layer, the dashboard provides access to key features through a structured navigation flow, reflecting the actual routing logic implemented in the system. From the dashboard, users can navigate to child profile management, story creation, story library, and progress tracking interfaces. The diagram further details internal navigation paths, including the ability to add or edit child profiles, initiate story generation, and transition from the story generation state to the story viewer upon completion. It also captures alternative flows such as retrying story generation in case of failure. Additionally, story access is supported from multiple entry points, such as the library and progress dashboard, ensuring flexible and user-centered navigation.

The diagram also distinguishes the administrative interface as a separate backend access point, represented by the Django Admin panel, which operates independently from the main frontend navigation.

Overall, this navigation diagram provides a clear and organized representation of the system's window structure and user flow, accurately reflecting the routing design and interaction patterns implemented in the Gheras platform while supporting usability and maintainability.

#### 4.4.2 User Interface Prototype

The low-fidelity user interface mockup serves as a visualization of the system interfaces, designed to demonstrate the platform's structural layout, interaction flow, and usability prior to full implementation. It simplifies detailed visual styling elements such as colors and textures. The prototype emphasizes layout hierarchy, component organization, navigation structure, and user interaction patterns by using grayscale wireframe elements and simplified placeholders rather than finalized visual assets. This approach allows the design to focus on usability, layout consistency, and functional relationships between interface components during the early design and evaluation stages.

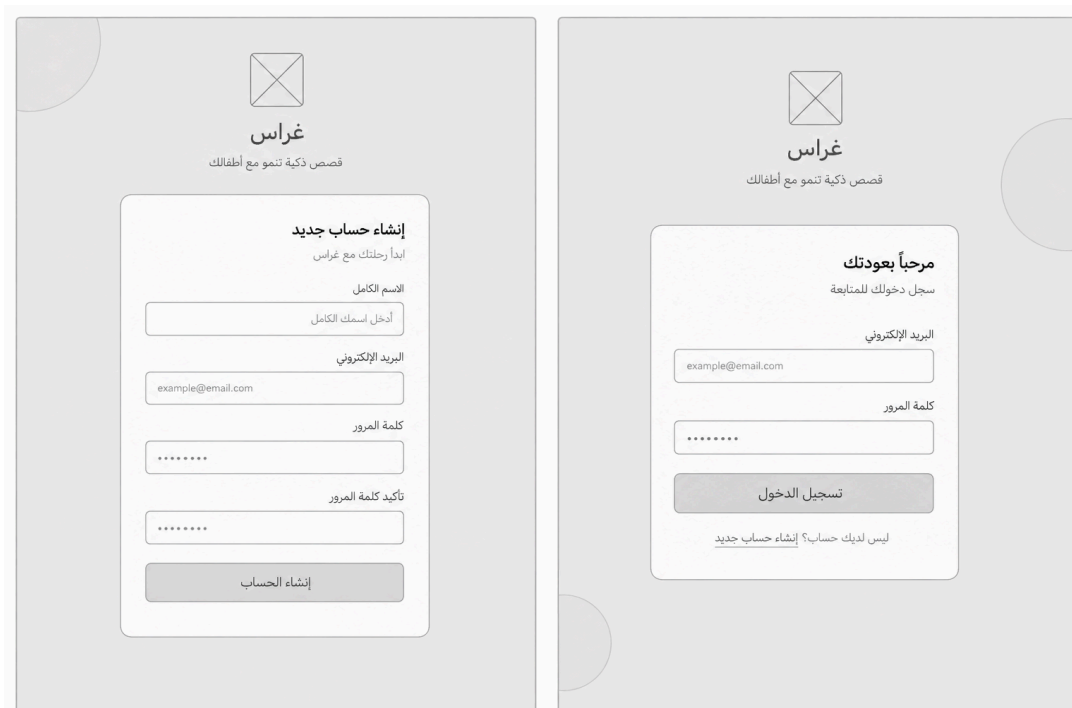


Figure 4.18. Authentication user interface mockup – Sign-Up and Login screens

Figure 4.18 illustrates two core authentication user interface mockup: the Sign-Up (Account Creation) screen and the Login (Sign-In) screen. Both interfaces follow a consistent, minimalist layout, featuring a centered card-based design that enhances readability and user focus. The Sign-Up interface includes labeled input fields for full name, email, password, and password confirmation, along with a primary action button for creating an account. The login interface simplifies interaction by requiring only an email address and password, with a login button and a secondary navigation link that guides users to account registration. This user interface mockup demonstrates the system's user entry points and ensures a seamless and intuitive onboarding experience.



Figure 4.19. Dashboard user interface mockup

Figure 4.19 presents a low-fidelity mockup of the Gheras dashboard, designed to provide users with a centralized overview of their activity and story-related interactions within the platform.

The dashboard user interface mockup is divided into multiple functional sections. The upper section contains summary statistic cards that display key user information, including the total number of stories, registered child profiles, and consecutive reading days. The central section presents child profile cards, each containing child information, story count indicators, and quick-access actions for generating new stories. The lower section displays recently created stories in card-based layouts, with image placeholders, story titles, creation dates, and page counts. Additionally, the interface includes a vertical navigation sidebar that allows users to access major system modules, including the home page, child management, story creation, library, and progress tracking. The consistent alignment, spacing, and hierarchy of components contribute to an intuitive user experience and reveal the system's overall information architecture.

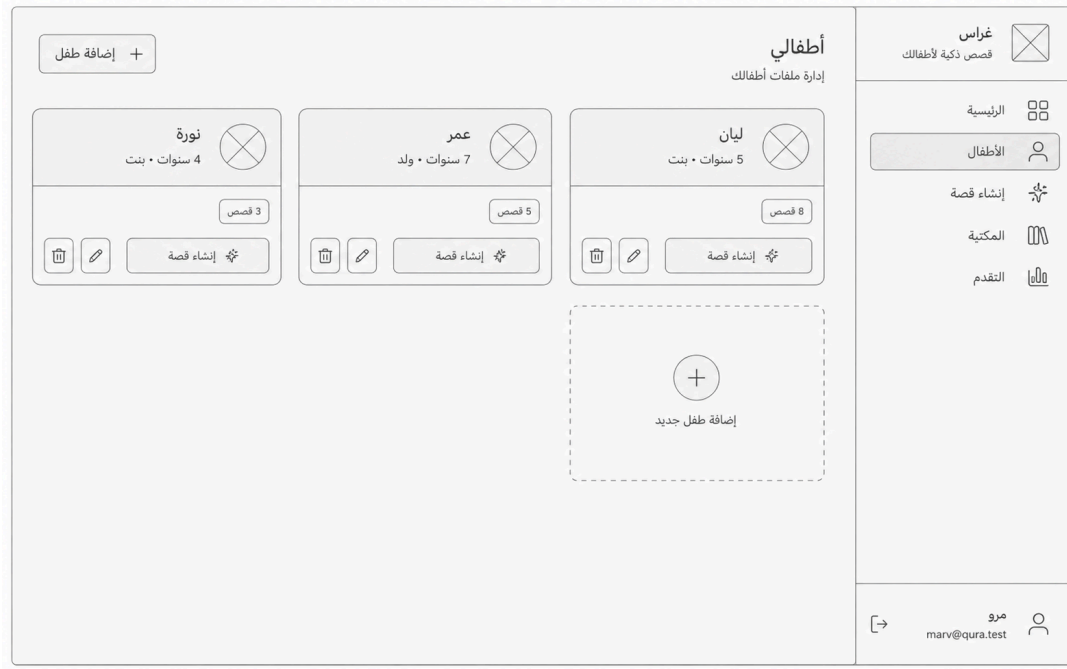


Figure 4.20. Child management user interface mockup

Figure 4.20 illustrates the child management user interface mockup within the Gheras platform, which enables parents or guardians to organize and manage child profiles in a centralized environment. The interface adopts a card-based structure where each child profile is presented as an individual interactive component containing essential information such as the child's name, age group, gender, and the total number of generated stories. In addition, quick-action controls are provided to support common operations, including editing child information, deleting profiles, and initiating the story generation process directly from the child card.

A dedicated section for adding a new child profile is also incorporated into the interface through a clearly distinguishable placeholder card accompanied by an add icon. This design supports scalability by allowing users to manage multiple children profiles efficiently while maintaining visual clarity and ease of navigation.

**إضافة طفل جديد**  
أضف ملفاً شخصياً لطفلك

اسم الطفل  
أدخل اسم الطفل

الجنس  
بنت ولد

العمر  
اختر العمر

وصف المظهر  
رفع صورة وصف نصي

صف مظهر طفلك (لون الشعر، العيون، إلخ)

إلغاء إضافة الطفل

غراس  
قصص ذكية لأطفالك

الرئيسية  
الأطفال  
إنشاء قصة  
المكتبة  
التقدم

مرو  
marv@qura.test

Figure 4.21. Create child profile user interface mockup

Figure 4.21 presents a mockup of the user interface for creating child profiles, which allows users to register and configure a personalized child profile on the Gheras platform. The interface is designed to collect essential child-related information required for generating customized stories and visual content. The form includes fields for the child's name, gender selection, age category, and an option to upload a reference image or provide an appearance description. These elements support the system's personalization capabilities by enabling story-generation processes to adapt to individual child characteristics. The user interface mockup follows a structured, form-based layout that organizes related inputs sequentially to simplify the data entry process and reduce user confusion. Interactive controls, such as dropdown menus, selectable gender buttons, upload actions, and confirmation buttons, provide clear user guidance during profile creation. The inclusion of both textual appearance descriptions and image-upload functionality enhances flexibility in defining child characteristics for backend AI-based story and illustration generation. Furthermore, navigation components integrated into the sidebar maintain consistency with the overall system structure and facilitate smooth transitions between platform modules.

Figure 4.22. Story creation – Genre selection (Step 1)

Figure 4.22 illustrates the first stage of the story-creation workflow in the Gheras platform, where users select the story genre for a specific child profile. The user interface mockup incorporates a step-based progression mechanism that guides users through the sequential configuration process required for personalized story generation. A child selection field is provided at the top of the interface, followed by a visual progress indicator that represents the three primary stages of story customization: story type, artistic style, and target behavior selection.

The main section of the user interface mockup presents multiple story genres as selectable card components. Each card includes a category title and a brief descriptive summary that explains the educational or thematic purpose of the selected genre. The available genres include adventure, fantasy, educational, social, nature, and space-related stories, enabling users to generate diverse storytelling experiences tailored to children's interests and learning objectives. Navigation controls at the bottom of the interface let users move between configuration stages efficiently. This design supports structured decision-making and enhances usability by organizing complex customization tasks into manageable interaction steps.

**إنشاء قصة جديدة**  
اختر تفاصيل القصة لطفلك

اختر الطفل  
ليان

3 — 2 — 1  
السلوك المستهدف الأسلوب الفني نوع القصة

**اختر الأسلوب الفني**

كتاب أطفال  
أسلوب كتب الأطفال الكلاسيكي

كرتون  
رسومات كرتونية مرحة وملونة

ألوان مائية  
رسومات ناعمة بألوان مائية دافئة

قص ولصق  
أسلوب القص واللصق الورقي

بكسل آرت  
رسومات رقمية بأسلوب البكسل

أنمي  
رسومات بأسلوب الأنمي الياباني

التالي السابق

غراس  
قصص ذكية لأطفالك

الرئيسية

الأطفال

إنشاء قصة

المكتبة

التقدم

مرو  
marv@qura.test

Figure 4.23. Story creation – Artistic style selection (Step 2)

Figure 4.23 presents the second stage of the story generation workflow, where users select the artistic style used for generating story illustrations. This stage enables visual customization of the storytelling experience by allowing users to choose from multiple illustration styles that influence the appearance and presentation of AI-generated images throughout the story pages. The interface maintains the sequential workflow structure established in the previous stage, providing users with a clear indication of their progress within the multi-step customization process.

The artistic style options are displayed using selectable card components, each containing a style title and a concise description explaining its visual characteristics. The available styles include classic children's book illustrations, watercolor renderings, cartoon designs, anime-inspired artwork, pixel art, and cut-paper-style compositions. These options provide flexibility in adapting generated visual content to different child preferences and storytelling contexts. The interface also includes navigation controls to progress to the next configuration stage or return to the previous step, ensuring continuity and ease of interaction throughout the story creation process.



**إنشاء قصة جديدة**  
اختر تفاصيل القصة لطفلك

اختر الطفل  
ليان

3 ————— ✓ ————— ✓  
السلوك المستهدف الأسلوب الفني نوع القصة

**اختر السلوك المستهدف**

|                                                                          |                                                                                    |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/><br><b>اللطف</b><br>التعامل بلطف مع الآخرين      | <input type="checkbox"/><br><b>الشجاعة</b><br>تعزيز الثقة بالنفس والشجاعة          |
| <input type="checkbox"/><br><b>الصدق</b><br>أهمية قول الحقيقة دائماً     | <input type="checkbox"/><br><b>المشاركة</b><br>تعلم مشاركة الأشياء مع الآخرين      |
| <input type="checkbox"/><br><b>الصبر</b><br>التحلي بالصبر وعدم الاستعجال | <input type="checkbox"/><br><b>المسؤولية</b><br>تحمل المسؤولية والاعتماد على النفس |

غراس ☐  
قصص ذكية لأطفالك

الرئيسية ☐  
الأطفال ☐  
**إنشاء قصة** ☐  
المكتبة ☐  
التقدم ☐

مرو ☐  
marv@qura.test

Figure 4.24. Story creation – Target behavior selection (Step 3)

Figure 4.24 illustrates the final stage of the personalized story configuration process, where users select the target behavior or educational value to be integrated into the generated story. This stage is designed to support the educational objectives of the Gheras platform by embedding positive behavioral concepts into story narratives in an engaging and age-appropriate manner. The interface visually indicates the completion of the previous configuration stages while emphasizing the behavioral selection step as the final requirement before story generation.

The available behavioral categories are presented through selectable cards containing a title and a short explanatory description. The options include kindness, courage, honesty, sharing, patience, and responsibility, each representing a specific moral or social skill intended to be reinforced through storytelling. By allowing users to define the intended behavioral outcome, the system can adapt narrative events, character interactions, and story resolutions to align with the selected educational objective. The structured arrangement of the cards and the sequential workflow indicators contribute to a clear, guided user experience while supporting the platform's pedagogical focus.

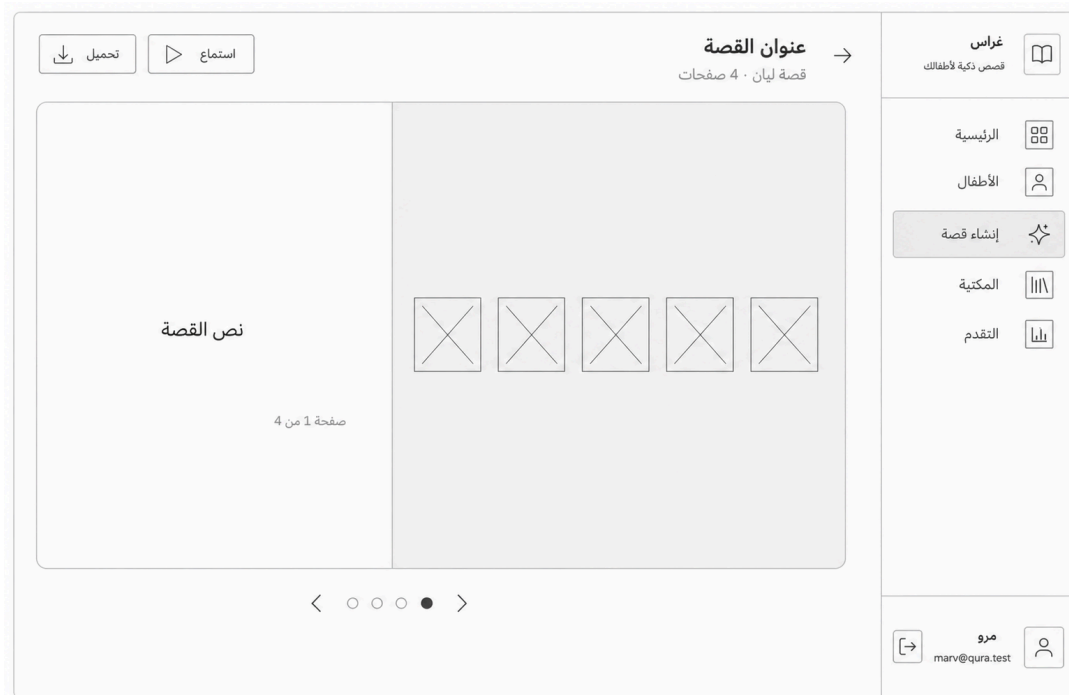


Figure 4.25. Story-reading user interface mockup

Figure 4.25 presents the story-reading user interface mockup of the Gheras platform, which enables children and parents to interact with generated stories through an integrated reading environment. The interface uses a dual-panel layout that separates textual narration from visual story illustrations, allowing users to view the story content and corresponding images simultaneously. This arrangement enhances readability and supports multimodal storytelling by combining narrative text with AI-generated visual elements within a unified interface.

The upper section of the user interface mockup includes essential interaction controls, including audio playback and story download features, to support accessibility and offline use. Story metadata, such as the story title and total page count, is displayed prominently to provide contextual information during reading. The lower section includes pagination indicators and navigation controls that allow users to move between story pages efficiently. In addition, the persistent sidebar navigation maintains access to the primary system modules. The overall structure promotes an engaging and organized storytelling environment tailored for child-oriented digital interaction.

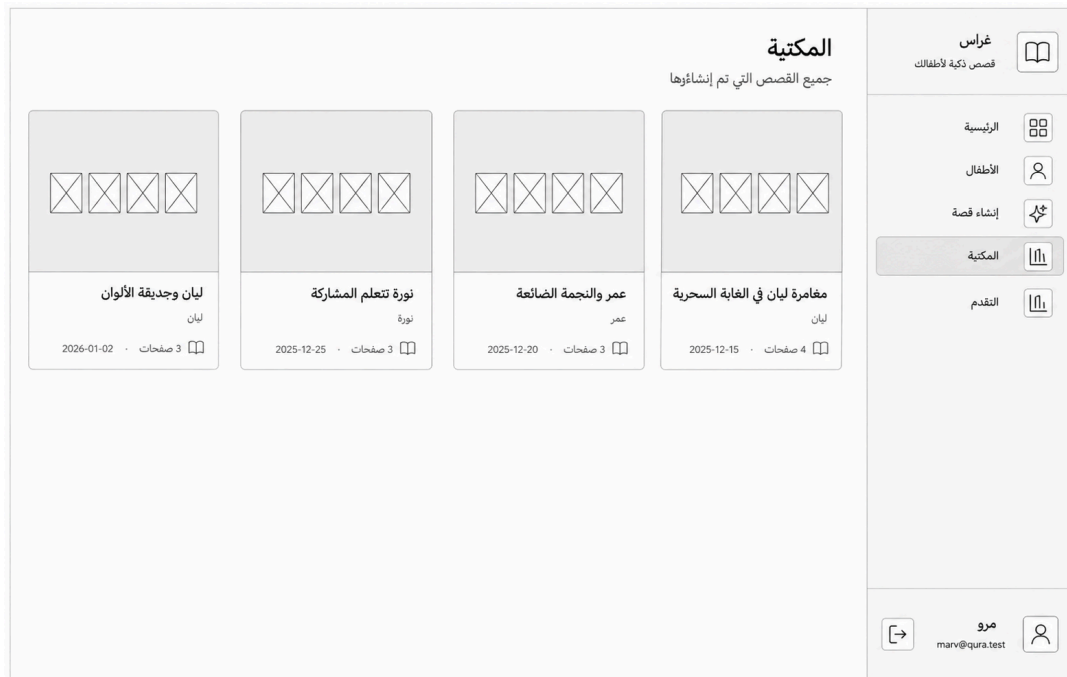


Figure 4.26. Story library user interface mockup

Figure 4.26 illustrates a mockup of the Gheras platform's story library user interface, providing users with centralized access to previously generated stories. The interface adopts a card-based content organization approach, where each story is represented as an individual item containing visual placeholders, story titles, associated child names, creation dates, and page counts. This arrangement enables users to browse and identify stories efficiently while maintaining a clean and organized presentation of stored content.

The library user interface mockup is designed to support content management and retrieval by presenting generated stories in a structured and easily navigable format. Each story card functions as an entry point to the full reading experience, allowing users to revisit previously generated narratives and illustrations. The inclusion of metadata beneath each card enhances information accessibility and helps users distinguish between different stories. Furthermore, the persistent sidebar navigation maintains continuity across system modules and enables seamless transitions between story generation, child management, progress tracking, and library functionality. The interface structure promotes usability and supports long-term interaction with generated educational content.



Figure 4.27. Progress-tracking user interface mockup

Figure 4.27 shows the progress-tracking interface of the Gheras platform, designed to monitor children's reading engagement over time and the statistics for the selected behaviors. The interface combines statistical summaries with visual analytics components to provide parents and guardians with insights into story interaction patterns. Summary cards at the top of the interface display key indicators, including the number of completed stories and consecutive reading days, enabling users to quickly assess overall activity levels.

The lower section of the user interface mockup contains graphical visualizations that represent reading behavior and targeted educational outcomes. The weekly reading activity chart illustrates the distribution of reading sessions across different days of the week, while the behavioral analytics chart displays the frequency of stories associated with specific target behaviors or moral concepts. These visual components support data interpretation and enable parents/guardians to identify trends in engagement and learning objectives. In addition, the interface maintains consistent navigation access to other system modules through the sidebar menu, ensuring a cohesive user experience across the platform.

## 5 IMPLEMENTATION

This chapter presents the implementation of the Gheras platform, including the frontend interface, backend system architecture, and AI services layer. It explains how the system components were developed and integrated to support personalized Arabic children's story generation. The chapter also discusses the implementation of multimodal AI services for narrative generation, image synthesis, and text-to-speech narration, as well as the communication workflow between system modules. Furthermore, the tools, frameworks, libraries, and technologies used throughout the development process are described.

### 5.1 Detailed System Implementation

#### 5.1.1 Backend Implementation

The backend of the Gheras platform serves as the core application layer responsible for processing user requests, managing application data, enforcing authentication and access control, and coordinating communication between the frontend, database, and external AI services. It provides the business logic required to support user management, child profiles, story generation workflows, and progress-tracking features. A PostgreSQL database is used to store and manage the platform's data, including user information, child profiles, generated stories, and related activity records.

##### 5.1.1.1 Database Design and Data Management

The database design was developed to support the main functionalities of the Gheras platform, including child profile management, story generation, reading activity tracking, and behavioral follow-up features. In addition to Django's built-in *User* entity, which is used for authentication and account management, the system includes several application-specific entities that represent the core data of the platform.

The *Child* entity stores information about each child, including the child's name, gender, age group, appearance description, avatar information, and optional uploaded photo. Each child profile is associated with a specific user account, ensuring that child-related information remains linked to the correct parent. Generated stories are represented through the *Story* and *StoryPage* entities. The *Story* entity stores story information, selected customization options, generation status, and generated content associated with the story, such as story text, illustrations, narration audio, and generated PDF files. The *StoryPage* entity stores the individual pages that compose a story, including narration text, illustrations, and narration audio references.

The *ReadingLog* and *BehaviorTask* entities are used to record user interactions with generated stories. The *ReadingLog* entity stores information related to reading sessions and progress tracking, while the *BehaviorTask* entity stores the child's response to the post-story behavioral question associated with the targeted behavior addressed by the story.

To support story customization, the system maintains separate *Behavior*, *Genre*, and *ArtStyle* entities. These entities define the available behaviors, story genres, and artistic styles that can be selected during story creation. Their records are maintained in the database and can be managed through the administration interface when updates are required.

##### 5.1.1.2 Database Administration and Content Management

As shown in Figure 5.1, the Gheras platform utilizes Django's built-in administration panel to provide authorized administrators with a centralized environment for managing platform records and configuration data.

Administrators can manage the main entities of the platform, including child profiles, generated stories, story pages, reading logs, behavioral task records, and story-customization options such as behaviors, genres, and artistic styles. The administration panel also provides access to user accounts, permission groups, and authentication tokens.

This functionality supports day-to-day system administration and allows platform records and configurable content to be maintained through a single management interface.

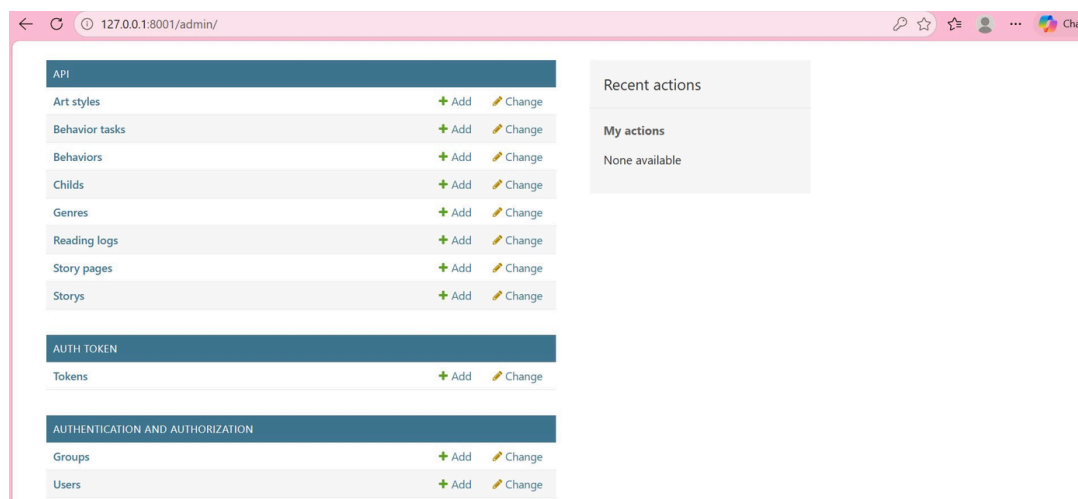


Figure 5.1. Django administration panel showing application records, configuration entities, authentication tokens, and user-management components.

### 5.1.1.3 REST API Implementation

The backend exposes a RESTful API that serves as the communication layer between the frontend application and backend services. The API supports the main platform functionalities, including user authentication, child profile management, story generation requests, reading activity tracking, behavioral task submission, and retrieval of system configuration data.

The API provides endpoints for managing the main entities of the platform, such as child profiles and generated stories, while additional endpoints support platform-specific functions including authentication, progress tracking, statistics retrieval, and behavioral interactions. This structure helps organize the different services provided by the platform and simplifies communication between the frontend and backend.

Story generation is implemented as an asynchronous process. When a user submits a story-generation request, the backend validates the provided information, creates an initial story record, and starts the generation process in the background. This approach allows the system to respond immediately to the user while generation continues independently. During this process, the story status is updated to reflect the current stage of generation, enabling the frontend to monitor progress and provide feedback to the user.

Once generation is completed, the backend stores the resulting story content, illustrations, audio files, and related metadata in the database. These outputs can then be accessed by users through the platform. If an error occurs during generation, the corresponding story record is updated to indicate the failure state, allowing the frontend to notify the user accordingly.

### 5.1.1.4 Authentication and Authorization

Authentication is implemented using a token-based mechanism. After successful registration or sign-in, the system issues an authentication token that is used to verify subsequent requests. This approach allows authenticated users to securely access protected platform features and services.

Access control is enforced throughout the backend to ensure that users can only access and manage their own data. Child profiles, generated stories, reading logs, and behavioral records are associated with the authenticated user account, preventing unauthorized access to resources belonging to other users. Additional validation checks are applied when creating or retrieving records to ensure that all operations are performed on resources owned by the current user.

### 5.1.1.5 Validation and Error Handling

The backend incorporates validation mechanisms to ensure that only valid and consistent data is processed by the system. User inputs are verified during authentication, child profile management, and

story creation processes. The system validates required information, checks the validity of selected story options, and ensures that custom behavioral inputs satisfy the accepted format before a generation request is accepted.

The backend also verifies that users can interact only with resources associated with their own accounts. Invalid requests, missing information, and unauthorized operations are detected and handled appropriately. For example, the system prevents access to resources belonging to other users, validates selected story options, and rejects invalid custom behavior inputs, such as symbols or meaningless text entered when the “Other” behavior option is selected. These measures help maintain data integrity and ensure reliable system operation.

### 5.1.2 Frontend Implementation

The frontend was developed using React with TypeScript, with Tailwind CSS for styling. The interface is fully right-to-left (RTL) to suit the platform’s Arabic-speaking audience. It is organized around two main areas: an authentication area for signing in and registering, and a dashboard area that hosts the core features of the platform. It also includes an onboarding component that shows tooltip-style popups guiding new users through the interface on their first visit.

The API client (`lib/api.ts`) is a thin wrapper around `fetch()`. It adds the auth token header automatically, handles JSON serialization, and has separate methods for regular requests and file downloads (for the PDF endpoint). The token goes in `localStorage` for “remember me” or `sessionStorage` for session-only login.

```
async function request<T>(path: string, options: RequestInit = {}): Promise<T> {
  const token = getToken();
  const headers: Record<string, string> = {
    ...(options.headers as Record<string, string>),
  };

  if (!(options.body instanceof FormData)) {
    headers["Content-Type"] = "application/json";
  }

  if (token) {
    headers["Authorization"] = `Token ${token}`;
  }

  const res = await fetch(`/api${path}`, { ...options, headers });

  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error || `Request failed: ${res.status}`);
  }

  if (res.status === 204) return undefined as T;
  return res.json();
}
```

Listing 5.1. The core request function in the API client

#### 5.1.2.1 Interfaces

The following screenshots show the main interfaces of the Gheras platform as they appear to the user. The interface follows a right-to-left Arabic layout with a fixed side navigation that gives access to the home page, children, story creation, library, and progress pages.

Figure 5.2 shows the story creation wizard, where the parent picks the child, then moves through three steps (story type, art style, and target behavior) using the step indicator at the top.

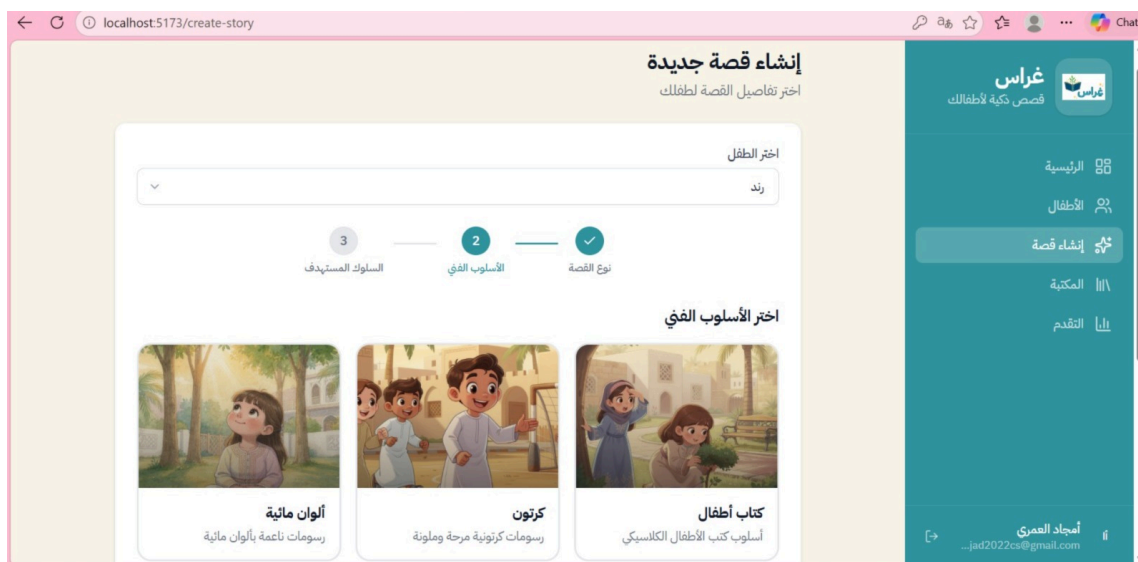


Figure 5.2. Story creation wizard showing the child selector and the three-step flow.

Figure 5.3 shows the art style step, where the parent chooses one of the available illustration styles such as watercolor, cartoon, children's book, anime, pixel art, or cut and paste. Each style is shown with a sample illustration so the choice is clear.

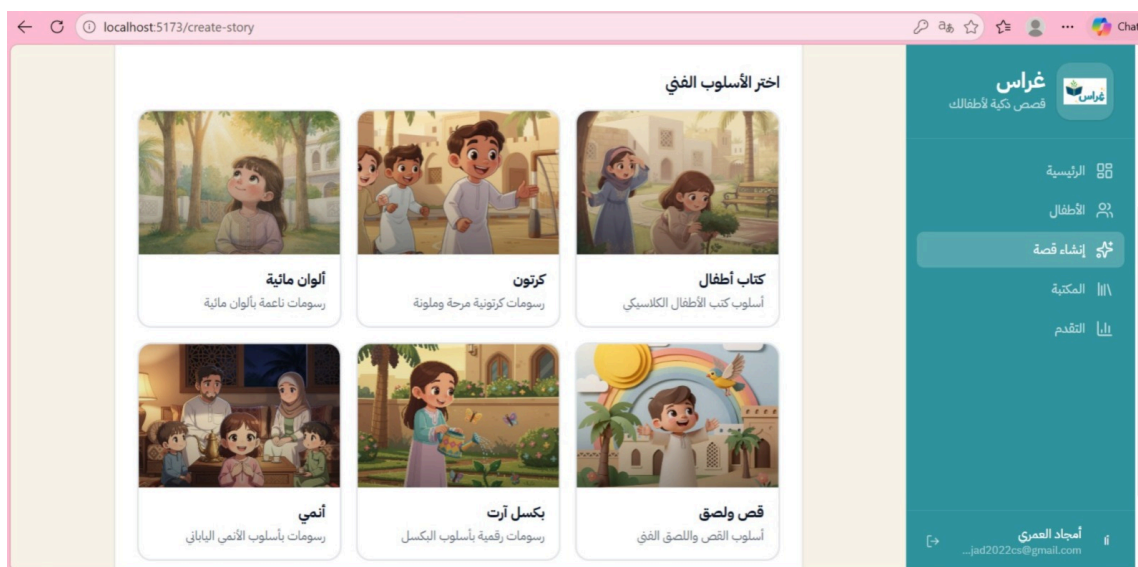


Figure 5.3. Art style selection step with sample illustrations for each style.

Figure 5.4 shows the library page, where all the stories the user created are listed. The page has a search box and filters for the child, the behavior, and the genre so the parent can find a story quickly.



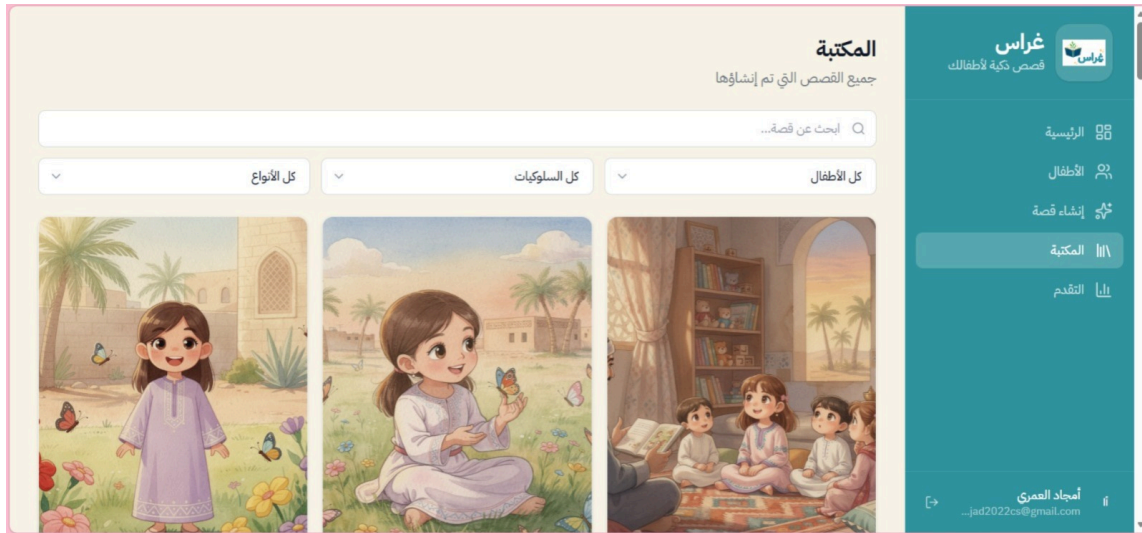


Figure 5.4. Library page listing the created stories with search and filters.

## 5.2 Algorithms and Models

This section presents the principal algorithms and AI models used within the Gheras platform. The algorithms part describes the core strategies that support story generation, illustration consistency, narration quality, and multimedia content management. The models part introduces the AI models used for story generation, image synthesis, and speech narration, together with their roles in the overall generation process. Model selection and evaluation are presented separately in the **Model Evaluation and Selection** section.

### 5.2.1 Algorithms

#### 5.2.1.1 Diacritic Quality Assessment

To improve the quality of Arabic narration, the system evaluates the level of diacritization (tashkeel) in generated story content before audio generation begins. The assessment process analyzes the story title and narration text to estimate the proportion of Arabic characters accompanied by diacritical marks. This value is then compared against a predefined minimum threshold. If the level of diacritization is considered insufficient, the story can be regenerated before narration is produced. This mechanism helps improve pronunciation quality and increases the accuracy of the generated Arabic speech.

#### 5.2.1.2 Content-Based Audio Caching

To avoid unnecessary narration generation, the platform applies a content-based audio caching strategy. Before requesting audio synthesis, the system generates a unique hash value from the narration text and uses it to identify previously generated audio files. If an audio file already exists for the same narration content, the stored version is reused instead of generating a new one. This approach reduces processing overhead, minimizes repeated requests to the narration service, and improves overall system efficiency.

#### 5.2.1.3 Deterministic Seed Strategy for Visual Consistency

To improve visual consistency across story illustrations, the system generates a single deterministic seed value and applies it to all image-generation requests associated with the same story. Using a shared seed encourages the image-generation model to produce illustrations with similar visual characteristics across different scenes. This helps maintain the appearance of the main character and reduces visual variation throughout the story without requiring additional post-processing or manual correction.

### 5.2.2 Models

The AI generation capability of the Gheras platform comes from three AI models that together produce the text, visuals, and audio of each story. A large language model generates the story content and the scene descriptions, an image-generation model produces the illustrations, and a text-to-speech model turns the narration text into audio. These models work as one pipeline that turns the user inputs and the target behavior into a complete multimedia story for children.

#### 5.2.2.1 Large Language Model: GPT-4o-mini

The Gheras platform uses OpenAI's GPT-4o-mini as the main language model for story generation [10]. We picked it because it generates good Arabic content while keeping response times and cost reasonable. Inside the platform, GPT-4o-mini produces stories that match the child's profile, the selected genre, the target behavior, and the chosen art style.

##### Responsibility

GPT-4o-mini generates the story text and the scene descriptions that are later used by the image-generation stage. From the information given by the user, the model produces a structured story package that becomes the base for the image and narration generation steps.

##### Prompt Strategy

Story generation is driven by a structured prompt that includes the child profile (name, age group, gender, and appearance description) together with the selected genre, target behavior, and art style. The prompt tells the model to write the story in Modern Standard Arabic with proper diacritics (tashkeel), age-appropriate vocabulary, and the target behavior woven naturally into the story. It also asks the model to generate visual scene descriptions that can be passed to the image-generation model.

##### Structured Output

The model returns a structured story package with a story title, a cover illustration description, and an ordered list of scenes. Each scene has narration text and a matching visual description, so the content can be processed the same way by the image and narration components that come after.

Table 5.1. GPT-4o-mini invocation parameters.

| Parameter   | Value                 | Purpose                                                                          |
|-------------|-----------------------|----------------------------------------------------------------------------------|
| Temperature | 0.3                   | Improves consistency and reduces randomness in story generation.                 |
| Max Tokens  | 1,500 / 2,800 / 4,500 | Adjusted according to the selected age group to support different story lengths. |

#### 5.2.2.2 Image Generation Model: FLUX-2-Max

Story illustrations are generated with the FLUX-2-Max image-generation model from Black Forest Labs [11]. We chose it for its strong prompt adherence, its ability to keep characters consistent, and its visually rich illustrations that fit Arabic children's stories.

##### Responsibility

In the Gheras platform, FLUX-2-Max turns the scene descriptions into illustrations that reflect the story context, the selected art style, and the child's characteristics.

##### Prompt Construction Strategy

To improve illustration quality and keep the character consistent, the image-generation prompt is built from several parts. These parts include child-safety constraints, Saudi cultural references, the selected art style, character identity information, and the scene-specific visual description from the language model. Putting these together keeps the illustrations aligned with the story, the child, and the chosen visual style across all scenes.

Table 5.2. Structured prompt components used for image generation.

| No. | Prompt Component                     | Purpose                                                                                      |
|-----|--------------------------------------|----------------------------------------------------------------------------------------------|
| 1   | Child-Safety and Quality Constraints | Ensures child-friendly illustrations and reduces undesirable visual artifacts.               |
| 2   | Saudi Cultural References            | Incorporates culturally relevant visual elements that align with the project context.        |
| 3   | Artistic Style Descriptor            | Applies the visual style selected by the parent during story creation.                       |
| 4   | Identity Source Information          | Specifies whether the child's appearance comes from an uploaded photo or a text description. |
| 5   | Character Consistency Information    | Preserves key visual characteristics such as age, gender, and appearance across all scenes.  |

### Reference Image Utilization

When a parent uploads a photo of the child, the image-generation process uses it as a visual reference to keep the character consistent across illustrations. The reference image is combined with the child's descriptive information, so the model can generate illustrations that look closer to the child while staying consistent through the whole story.

#### 5.2.2.3 Text-to-Speech Model: GPT-4o-mini-tts

The Gheras platform uses OpenAI's gpt-4o-mini-tts model to generate Arabic narration audio [12]. We selected it because it produces natural-sounding speech, supports Arabic, and uses diacritics (tashkeel) to improve pronunciation accuracy. This makes the narration clear and suitable for young children.

#### Responsibility

GPT-4o-mini-tts converts the story narration into spoken audio. It takes the narration text produced during story generation and creates audio that plays alongside the matching story pages.

#### Output

The model generates narration audio for each story page, so the audio can be played page by page during reading. The audio files are linked to the story content and made available to users through the platform.

#### 5.2.2.4 AI Generation Pipeline

The Gheras platform uses a coordinated AI generation pipeline that connects the language model, the image-generation model, and the text-to-speech model to produce complete multimedia stories. The pipeline turns the user inputs into story text, illustrations, and narration audio through a sequence of connected stages.

#### Age-Adaptive Story Parameterization

Table 5.3. Age-adaptive scene count and token budget mapping.

| Age Group | Scene Count | Max Tokens |
|-----------|-------------|------------|
| 4–5 years | 3           | 1,500      |
| 6–7 years | 5           | 2,800      |
| 8–9 years | 7           | 4,500      |

Story length and generation limits are adjusted to the selected age group so the result stays appropriate for the child's developmental stage.

#### Generation Stages

*Stage 1 — Input Preparation.* The system collects the child profile information, the selected story options, and the target behavior. These inputs are organized into the generation request.

*Stage 2 — Story Generation.* The language model generates a structured story package with the story title, narration text, and scene descriptions based on the inputs.

*Stage 3 — Initial Multimedia Generation.* Illustrations and narration audio are generated for the first story scenes, so the user can start reading while the rest keeps processing in the background.

*Stage 4 — Remaining Asset Generation.* The remaining illustrations, narration audio, and the cover image are generated and attached to the matching story pages.

*Stage 5 — Story Assembly and Storage.* The generated text, illustrations, and narration audio are linked to the story records and stored in the platform, making the finished story available to the user.

### 5.2.3 Model Evaluation and Selection

The evaluation of Natural Language Generation (NLG) systems constitutes a fundamental pillar in the development of artificial intelligence applications. It assesses the extent to which models can produce text that approximates human-level quality while satisfying user requirements. In the context of children’s storytelling, evaluation extends beyond linguistic accuracy to include creative, educational, cultural, and psychological dimensions, ensuring that generated content is appropriate for the intended age group. This is particularly important for child-oriented systems, where logical inconsistencies, unsafe content, or misaligned values may negatively influence comprehension, behavior, or trust [10]. Accordingly, before developing the Gheras system, a systematic evaluation phase was conducted to assess the suitability of candidate AI models for integration. In the initial stage of the study, a comprehensive investigation was carried out to identify recent and widely adopted models with strong support for Arabic language generation and multimodal content creation. Candidate models were selected based on their reported performance in the literature, their practical availability through APIs or model hubs, and their demonstrated capabilities in Arabic natural language processing tasks [11], [12], [13], [14]. Each model was experimentally evaluated through API-based integration under a unified prompt design. The same prompt structure, task requirements, and testing conditions were used across models to support fairness, consistency, and reproducibility. The evaluation process incorporated both qualitative and quantitative criteria, including output quality, linguistic accuracy, Arabic fluency, diacritization quality, semantic alignment, cultural suitability, and response latency [15], [16].

#### 5.2.3.1 Text LLM Generating Models: Comparative Analysis

Several state-of-the-art text generation models were systematically evaluated to determine their suitability for Arabic children’s story generation. The selected models were GPT-4o mini, Jais-2-8B-Chat, LFM2.5-1.2B-Instruct, and Qwen3-Coder-Next. These models were chosen based on recency, accessibility, multilingual capabilities, and reported performance in Arabic or general natural language generation tasks. Prior work has shown that GPT-based models can perform competitively in Arabic generation and evaluation tasks, including paraphrasing and linguistic understanding [11], [12]. Similarly, Jais was specifically developed as an Arabic-centric bilingual language model and has demonstrated competitive results on Arabic NLP benchmarks [13]. The Qwen model family has also been explored for advanced generative, multilingual, and coding-related tasks [14], while lightweight instruction-tuned models such as LiquidAI’s LFM2.5 aim to provide efficient alternatives with lower computational cost [17].

Three complementary evaluation approaches were adopted for the text-generation models: human-centric evaluation, automatic metric-based evaluation, and LLM-as-a-Judge evaluation. Human-centric evaluation remains one of the most natural and reliable methods for assessing generated stories because human evaluators can judge fluency, coherence, moral appropriateness, and readability in ways that automatic metrics cannot fully capture [10]. In this study, the assessment was applied to multiple generated outputs and involved Arabic-language evaluators, including three Arabic language teachers and two general users. Particular attention was given to Modern Standard Arabic (MSA), grammatical and spelling accuracy, vocabulary simplicity for children, diacritization, and the absence of awkward or unintelligible expressions.

Safety and cultural considerations were also included. The generated stories were checked to ensure that they were free from violent, frightening, or culturally inappropriate content and that they aligned with Islamic and societal values. Multiple samples were generated from each model and scored using predefined dimensions, enabling a structured comparative analysis.

#### 5.2.3.1.1 Human-Centric Evaluation

The human-centric assessment dimensions were categorized as follows:

- Readability and clarity: the ease with which the sentences can be understood and the suitability of vocabulary for the child’s cognitive level.
- Coherence and logical flow: local coherence between sentences and global coherence across the plot structure and sequence of events [10].
- Fluency and linguistic quality: grammatical and spelling accuracy, natural sentence construction, and appropriate use of Modern Standard Arabic.
- Relevance and alignment: the degree to which the model responds to the requested prompt elements, including characters, setting, behavior, and moral value.
- Cultural and ethical appropriateness: alignment with Islamic values, Arab/Saudi social norms, and child-safe storytelling principles.

#### GPT-4o mini

The outputs generated by GPT-4o mini demonstrate a consistently high level of readability and clarity, with well-structured sentences appropriate for the target age group. Across multiple samples, the narratives were easy to follow and relied on simple vocabulary and short sentence structures that align with children’s cognitive abilities. In terms of relevance and contextual appropriateness, the model showed strong adherence to task requirements, successfully incorporating moral values such as honesty and responsibility within culturally appropriate settings.

The coherence and logical flow were particularly strong. Events progressed in a clear and natural sequence, supported by smooth transitions and meaningful dialogue. Linguistically, the outputs exhibited a high degree of grammatical correctness and fluency in MSA, with relatively accurate and consistent diacritization compared with other models. The content also demonstrated strong cultural and ethical sensitivity by avoiding inappropriate themes and aligning with Islamic values and social norms. Regarding prompt adherence, the model largely followed the specified structure, including title, narrative development, and moral conclusion, although minor formatting inconsistencies occasionally appeared.

العنوان: صديقة مجد الجميلة

القصة:

كان يا ما كان، في قرية صغيرة، ولد دعي مجد. كان مجد طفلاً طيباً، يحب اللعب مع أصدقائه. كان له صديقة تدعى ردا، كانت مريحة وجميلة. كانت ردا تحب أن تلتصق إلى قصص مجد.

في يوم من الأيام، ذهب مجد و ردا إلى الحديقة. كانت الأشجار خضراء، والأزهار ملونة. رأيا مجد شيئاً يلعب في التراب. اقتربت مجد ونظرت، فكانت حشرة ذهبية!

قال مجد: "رنا، انظري! أنا وجدت حشرة ذهبية!"

فكرت ردا وقالت: "يا مجد، هل فكرت أنها لي؟"

فكر مجد: "لا أعرف، لكنها جميلة."

فكر مجد، وفكر، ثم قال: "يجب علينا أن نكتسب أوي. أوي دائماً نعرف أن الأمانة مهمة."

علا مجد و ردا إلى منزلهم. قال مجد لأوي: "يا أوي، وجدت حشرة ذهبية في الحديقة. هل أحفظ بها؟"

ابسمت أوي مجد وقالت: "يا بني، إن الأمانة من أجمل القيم. إذا كانت حشرتك، فعليك إعادتها إلى صاحبها."

فعل مجد: "سأعيدها! أريد أن أكون صديقاً جيداً."

علا مجد و ردا إلى الحديقة، وهناك يسأل الأطفال هناك عن الحشرة. بعد قليل، وجدوا شيئاً يدعي علي، كان حزيناً. سأله مجد: "ما بك؟"

قال علي: "فقدت حشرتي الذهبية."

فعل مجد: "هذا هو ما وجدناه! إنه لك."

فرح علي كثيراً وقال: "شكراً لكم! كنت حزينا جداً."

ابسمت ردا وقالت: "نحن متأكدون أنك وجدت حشرتك."

فالت أوي علي التي كانت قريبة: "بارك الله فيكما، يا أبناء، علمكما خيرهما معنى الأمانة."

شعر مجد بالفرح، وقال: "الأمانة شئ جيد للجميع."

ومن ذلك اليوم، أصبح مجد و ردا متلاً خصباً بين أصدقائهما، يلتزمان الخير والمشي في كل مكان.

وعلا إلى اللوح وهما سجدان، ولقوبهما ممتلئة بالفرح.

Figure 5.5. Generated Arabic story output sample produced by GPT-4o mini.

### Jais-2-8B-Chat

The outputs of Jais-2-8B-Chat showed good readability, with generally clear and understandable sentence structures suitable for children. Across multiple generations, the model consistently produced narratives that aligned with the intended theme of moral storytelling. In terms of relevance and contextual appropriateness, the model performed well by embedding ethical values such as honesty and integrity in a meaningful way.

However, coherence and logical flow, while acceptable, occasionally exhibited minor discontinuities, particularly in transitions between events or in the depth of narrative development. Linguistic quality was generally strong and reflected good command of Arabic vocabulary and grammar. The model demonstrated strong cultural and ethical awareness, consistently avoiding inappropriate content and aligning with Islamic and societal values. With respect to prompt adherence, the outputs followed the general structure but did not always strictly comply with formatting constraints or detailed prompt requirements.

\*\*العنوان: سامي والكز الحلي

\*\*القصة:\*\*

في حي هادي جميل، كان يعيش طفل صغير اسمه سامي. كان سامي يحب اللعب كثيراً، لكنه كان يحب الصدق أكثر.

في أحد الأيام، بينما كان سامي عائداً من المدرسة، رأى شيئاً يلعب تحت شجرة الليمون الكبير في حديقة منزله. كان بريقه ذهبياً، وكأنه قطعة حلوى!

اقترب سامي بحذر، وقلبه الصغير يهتز بسرعة. هل هذا كز حقيقي؟ هل هو مال كثير؟ كان أن يلمسه، لكنه تذكر نصيحة والدته: "الصدق هو الكز الأكبر يا بني."

فكر سامي: "ماذا لو لم يكن كز؟ ماذا لو كان شيئاً ليس لي؟" شعر بتقل في قلبه لو لم يكن أمياً.

تذكر سامي بيت شعبي جميل كان قد سمعه من معلمته: "الصدق نور يضيء الدرب، والكذب ظلمة لا تريح القلب."

أغمض سامي عينيه للحظة، ثم فتحهما وقرّر. قال بصوت خفيض، كأنه يحدث نفسه: "هذا الكز، سأخبر به أمي وأبي. إنه أجمل كز في الدنيا، لأنه يجلب السعادة لي ولغيري."

عاد سامي إلى المنزل وهو يحمل الكز الصغير في يده، لكنه كان يحمل كزاً أكبر بكثير في قلبه: كز الأمانة والصدق.

عندما وصل إلى منزله، أخبر سامي والدته أولاً، ثم والدته التي ابسمت بفرح وقالت: "أحسن يا حبيبي! أنت تعلمت درساً قديماً اليوم."

في المساء، جلسا معاً حول مائدة العشاء، وحكى سامي لوالديه عن اكتشافه، وعن قراره الصادق. كان الكز الذي وجد سامي ليس ذهباً أو فضة، بل كان قيمة عظيمة في عينيه وعيني والديه.

Figure 5.6. Generated Arabic story output sample produced by Jais-2-8B-Chat.

### Qwen3-Coder-Next

The outputs generated by Qwen3-Coder-Next demonstrated good readability and generally well-structured linguistic form. Across multiple samples, the model produced short, well-formed sentences that aligned with the requirement for simplified narrative construction. Readability and clarity were further supported by relatively consistent use of diacritization and controlled sentence length, which facilitates comprehension for young readers.

In terms of relevance and contextual appropriateness, the model successfully captured the intended moral themes and presented them within culturally suitable scenarios. The coherence and logical flow were largely satisfactory, as the narrative followed a clear progression from introduction to resolution. However, minor lexical inaccuracies and unnatural expressions affected semantic precision. In addition, the model occasionally used advanced or less child-friendly vocabulary, which may reduce accessibility for the target age group. Linguistically, it demonstrated strong command of Standard Arabic, although this was sometimes achieved at the expense of simplicity. From a cultural and ethical perspective, the output remained appropriate, reflecting values such as honesty, trustworthiness, and reliance on family guidance. Regarding prompt adherence, the model followed most structural and stylistic requirements with only minor deviations.

العنوان: \*\*سعد واللؤلؤة المفقودة\*\*  
 القصة:  
 كان سعد فتى صافها، طيب القلب، يحب خدمة أبيه.  
 ذهب سعد ولعب في حديقة الجيران بعد صلاة العصر.  
 في زاوية من الحديقة، رأى شيئاً لمع في الضوء.  
 اقترب بحرص، فإذا بلؤلؤة فضيئة جميلة تلفف الأظفار.  
 قال سعد بلطف: «هذه لؤلؤة غريبة، لنأخذها لعلها خفية!»  
 عاد سعد إلى بيته، وأخذ اللؤلؤة في كفه.  
 قال لأبيه: «يا أبت، وجدت لؤلؤة في الحديقة، أترى من فذها؟»  
 فأجابته أمه باليسامة دافئة: «أحسنت يا بني، فإن الأمانة تزيّن المؤمن».  
 ثم قالت: «لننظُرَ طليها، ولنُدعِ الله أن يرُدّها لأهلها».  
 في اليوم التالي، جاءت فتاة صغيرة تبكي عند باب المسجد عائشة.  
 قالت الصغيرة بخزن: «فقدت لؤلؤتي... كانت هبة جدي».  
 فسالت السيدة عائشة: «هل وجدتَها يا سعد؟»  
 فأخرج سعد اللؤلؤة وقال: «هذه هي، وجدتُها في الحديقة».  
 ففرحت الفتاة فرحاً كبيراً، ومسحت دموعها وقالت: «شكراً لك يا سعد، أنقذت قلبي».  
 قال سعد بتواضع: «هذا واجبي، فالأمانة خلق جميل».  
 فدعت له السيدة عائشة وأمه بالخير، وقالتا: «جزاك الله خيراً يا بني».  
 سخر سعد بسعادة كبيرة في قلبه، وعلم أن فعل الخير يجلب الفرح.  
 ومن ذلك اليوم، أصبح سعد فذوة في الأمانة والصديق بين أصدقائه.

Figure 5.7. Generated Arabic story output sample produced by Qwen3-Coder-Next.

### LFM2.5-1.2B-Instruct (LiquidAI)

The outputs produced by LFM2.5-1.2B-Instruct exhibited significant limitations across most evaluation dimensions. Readability and clarity were notably weak, as the generated text often contained fragmented or confusing sentences that hindered comprehension, especially for a child audience. In terms of relevance and contextual appropriateness, the outputs showed limited alignment with the task, frequently deviating from the expected narrative structure and failing to fully integrate the required moral themes. Coherence and logical flow were particularly problematic, with events appearing disjointed and lacking clear progression. Linguistically, the model demonstrated substantial weaknesses, including grammatical errors, incorrect word usage, absence of diacritization, and inconsistent sentence construction. While the model did not explicitly produce harmful or inappropriate content, its cultural and ethical alignment was superficial and not meaningfully integrated into the narrative. Prompt adherence was also poor, as outputs did not consistently follow the required structure, formatting, or level of detail specified in the prompt.

\*\*العنوان\*\* :

ألياً كريم يحب الكرسي الموسيقي

القصة:

كانت هناك فتى اسمه عمرو، كان يعيش في بيت صغير بالقرب من المدرسة. كان يحب اللعب الموسيقي مع أصدقائه. ذات يوم، سمع شخص غريب يقول: "يا عمرو، يرجى أن تأخذ كرسي الموسيقى". شعر عمرو بالراحة وسأله: "كيف يمكنني؟" قال الكبير: "أنا أحب هذا الكرسي لأنه يجلب الفرح". وبدأ عمرو يتحدث بسرعة عن كيف أنه يستمتع كل يوم.

في اليوم التالي، ذهب عمرو إلى المدرسة. لاحظ أن الجميع كانوا يستمعوا له مع لحن جميل. سمع صديقه "أوه! هذا الكرسي جيب للموسيقى!" كان ذلك يعزز شعوره بالفرح والتعاون.

في المسجد، رأى عمرو زوجته ترحب به بحب. قال لها: "يا أمي، شكرا لك على الكرسي الجميل!" وجمعت بين الضيافة والصدقة.

كان هذا اليوم مليء بالإيجابية، وساعد عمرو على تعلم قيمة العمل الجماعي.

Figure 5.8. Generated Arabic story output sample produced by LFM2.5-1.2B-Instruct.

The human-centric results indicate that GPT-4o mini achieved the highest overall performance, with an average score of 9.5, demonstrating superior linguistic quality, coherence, and adherence to prompt constraints. Jais-2-8B-Chat and Qwen3-Coder-Next showed competitive performance with minor limitations, while LFM2.5-1.2B-Instruct significantly underperformed across most dimensions.

Table 5.4. Comprehensive human-centric evaluation scores for the text generation models.

| Model                | Read. | Rel. | Coh. | Ling. | Cult. | Prompt | Avg | Notes                                            |
|----------------------|-------|------|------|-------|-------|--------|-----|--------------------------------------------------|
| GPT-4o mini          | 9.5   | 10   | 9    | 10    | 9.5   | 9      | 9.5 | High quality, strong coherence, excellent Arabic |
| Jais-2-8B-Chat       | 9     | 8.5  | 7.5  | 9.5   | 9     | 8      | 8.6 | Good overall, minor coherence issues             |
| Qwen3-Coder-Next     | 7     | 8.5  | 8    | 9     | 10    | 8      | 8.4 | Strong language, slightly complex vocabulary     |
| LFM2.5-1.2B-Instruct | 3.5   | 3    | 4    | 4     | 6     | 3.5    | 4.0 | Weak structure, poor language quality            |

#### 5.2.3.1.2 Untrained Automatic Metrics

Complementing the human evaluation, an automatic quantitative metric-based evaluation was implemented through the experimental code. This method provides objective measurements of model outputs using predefined indicators. The evaluation included average latency, which measures the response time of a model and reflects practical efficiency, and diacritic density, which measures the frequency of Arabic diacritical marks relative to Arabic letters and reflects the model's ability to generate properly vocalized Arabic text.

Table 5.5. Quantitative evaluation of text generation models using latency and diacritic density.



Table 3: Quantitative Evaluation of Text Generation Models

| Model                | Latency (s) | Diacritic Density |
|----------------------|-------------|-------------------|
| GPT-4o mini          | 16.26       | 0.88              |
| Jais-2-8B-Chat       | 50.71       | 0.83              |
| Qwen3-Coder-Next     | 54.71       | 0.98              |
| LFM2.5-1.2B-Instruct | 90.29       | 0.00              |

As shown in Table 5.5, GPT-4o mini achieved the lowest latency, indicating superior efficiency, while Qwen3-Coder-Next demonstrated the highest diacritic density, reflecting stronger performance in Arabic vocalization. In contrast, LFM2.5-1.2B-Instruct showed both the highest latency and the lowest diacritic density, highlighting limitations in efficiency and linguistic quality.

Among untrained automatic metrics, n-gram-based diversity measures are widely used to assess lexical variation in generated texts. In this study, the Type-Token Ratio (TTR) was used to quantify lexical diversity, reflecting the breadth and variety of vocabulary usage within a text. TTR is a well-established linguistic metric for evaluating vocabulary richness [18], [19]. Formally, TTR is computed as:

$$TTR = \frac{|V|}{N} = \frac{\text{Number of Unique Words}}{\text{Total Number of Words}} \quad (5.1)$$

A higher TTR value indicates greater lexical diversity, suggesting that the model uses a wider vocabulary range, while lower values may indicate repetition or limited vocabulary usage. Figure 5.9 presents the lexical diversity results obtained after applying this metric to GPT-4o mini, Jais-2-8B-Chat, Qwen3-Coder-Next, and LFM2.5-1.2B-Instruct.

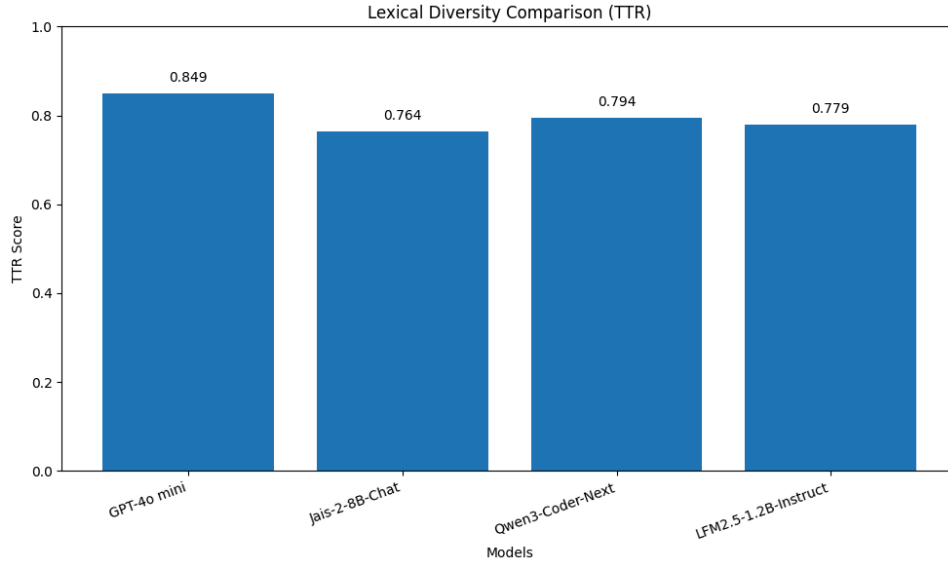


Figure 5.9. Lexical diversity comparison using Type-Token Ratio (TTR).

As illustrated in Figure 5.9, GPT-4o mini achieved the highest lexical diversity score (0.849), indicating broader and more varied vocabulary compared with the other models. Qwen3-Coder-Next followed with a strong score (0.794), demonstrating relatively rich lexical variation. In contrast, LFM2.5-1.2B-Instruct (0.779) and Jais-2-8B-Chat (0.764) exhibited slightly lower diversity scores, suggesting comparatively more limited vocabulary usage. Overall, the results indicate that more advanced models tend to generate linguistically richer narratives, although the differences across models remain moderate.

### 5.2.3.1.3 Machine-Learned Metrics

BERTScore was employed to evaluate the semantic similarity between the generated stories and a reference text. The reference text consisted of a well-structured, linguistically accurate, and fully diacritized Arabic children’s story that adhered to the required stylistic and educational guidelines. Safety considerations were also incorporated to ensure that the content remained free from harmful, inappropriate, or culturally misaligned elements and adhered to Islamic and societal values.

BERTScore is a BERT-based evaluation metric that leverages pretrained contextual embeddings to assess semantic similarity at the token level [20]. Unlike traditional string-based metrics, it computes similarity by matching tokens in the candidate and reference texts using cosine similarity in the embedding space. The metric produces three complementary scores: precision, which measures the extent to which the generated text is semantically accurate with respect to the reference; recall, which reflects how well the generated text captures the content of the reference; and F1-score, the harmonic mean of precision and recall, which provides a balanced overall evaluation.

Compared with conventional n-gram-based metrics such as BLEU and ROUGE, BERTScore is more effective in capturing deeper semantic relationships and contextual meaning. This makes it suitable for creative text generation tasks such as storytelling, where exact word overlap is less important than semantic coherence and narrative quality [10], [20]. Figure 5.10 presents the computed BERTScore metrics for GPT-4o mini, Jais-2-8B-Chat, LFM2.5-1.2B-Instruct, and Qwen3-Coder-Next.

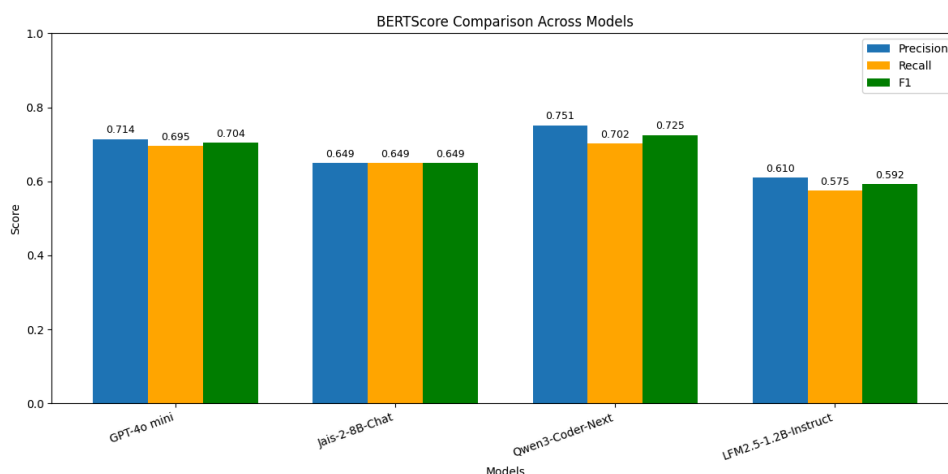


Figure 5.10. BERTScore comparison across text generation models.

As illustrated in Figure 5.10, Qwen3-Coder-Next achieved the highest overall semantic performance, with the top F1-score (0.725), indicating superior semantic alignment and balanced content coverage. GPT-4o mini followed closely, demonstrating strong and consistent performance across all three metrics. Jais-2-8B-Chat exhibited moderate performance with nearly identical precision, recall, and F1 scores, suggesting stable but less expressive semantic representation. Finally, LFM2.5-1.2B-Instruct recorded the lowest scores across all metrics, indicating weaker semantic similarity and reduced effectiveness in capturing the intended meaning of the reference text.

### 5.2.3.1.4 LLM-as-a-Judge

The LLM-as-a-Judge evaluation framework provides a scalable and consistent methodology for automated, human-like assessment. In this framework, a large language model is used to evaluate generated texts based on predefined qualitative criteria [21], [22], [23]. In this study, the evaluation process used a ChatGPT-based model, which is suitable for Arabic language understanding, generation, and evaluation tasks when guided by a structured rubric [11], [12].

The judge model evaluated the generated children’s stories across fluency, coherence, readability, cultural alignment, Modern Standard Arabic accuracy, vocabulary appropriateness, and diacritization.

Safety considerations were included to ensure that generated content remained free from harmful, inappropriate, or culturally misaligned elements and adhered to Islamic and societal values. This comprehensive framework enabled systematic and reproducible comparison across story-generation models.

Table 5.6. LLM-as-a-Judge evaluation summary for text generation models.

Table 2: (LLM-as-a-Judge)

| Model                | Read. | Coh. | Ling. | Diac. | Age | Moral | Creat. | Struct. | Safe | Tone | Prompt |
|----------------------|-------|------|-------|-------|-----|-------|--------|---------|------|------|--------|
| GPT-4o mini          | 5/5   | 5/5  | 5/5   | 5/5   | 5/5 | 5/5   | 4/5    | ✓       | ✓    | ✓    | ✓      |
| Jais-2-8B-Chat       | 4/5   | 4/5  | 4/5   | 2/5   | 4/5 | 5/5   | 4/5    | ✓       | ✓    | ✓    | ✓      |
| Qwen3-Coder-Next     | 4/5   | 5/5  | 5/5   | 5/5   | 4/5 | 5/5   | 4/5    | ✓       | ✓    | ✓    | ×      |
| LFM2.5-1.2B-Instruct | 2/5   | 2/5  | 1/5   | 1/5   | 2/5 | 3/5   | 2/5    | ×       | ✓    | ✓    | ×      |

The results in Table 5.6 highlight clear performance differences among the evaluated models. GPT-4o mini and Qwen3-Coder-Next achieved the highest scores, demonstrating strong capabilities in linguistic quality, coherence, and adherence to prompt constraints. Jais-2-8B-Chat showed moderate performance, particularly strong in moral content but weaker in technical aspects such as diacritization. In contrast, LFM2.5-1.2B-Instruct significantly underperformed across most dimensions, especially language accuracy and structural consistency.

#### LLM Justifications

The LLM-as-a-Judge framework not only provided quantitative scores but also generated qualitative justifications, offering deeper insight into the strengths and weaknesses of each evaluated model. These justifications were grounded in contextual understanding and enabled assessment of nuanced aspects such as narrative coherence, linguistic fluency, cultural appropriateness, and prompt alignment beyond surface-level metrics.

The generated justifications indicated that GPT-4o mini excelled in producing well-structured and linguistically accurate narratives with consistent use of MSA and appropriate diacritization. Qwen3-Coder-Next also demonstrated strong semantic coherence and adherence to narrative structure, although minor limitations were observed in age-level simplicity and prompt alignment. Jais-2-8B-Chat was consistently identified as effective in embedding moral and cultural values within the narrative; however, its performance was constrained by weaker diacritic usage and less precise structural compliance. LFM2.5-1.2B-Instruct struggled across grammatical correctness, coherence, and narrative organization, resulting in outputs that were less suitable for children’s storytelling.

#### Model Selection Justification

A comprehensive analysis of the evaluated text-generation models indicates that, while performance was generally comparable across several human-centric dimensions, clear distinctions emerged when considering overall consistency and quality. GPT-4o mini consistently demonstrated superior performance in linguistic quality, coherence, prompt adherence, and latency, in addition to achieving the highest lexical diversity. Qwen3-Coder-Next showed strong performance, particularly in semantic alignment as reflected by the highest BERTScore F1, although minor limitations appeared in lexical precision and age-appropriate simplicity. Jais-2-8B-Chat achieved stable and competitive results, especially in cultural alignment and moral content, but showed moderate weaknesses in diacritization and strict structural adherence. LFM2.5-1.2B-Instruct significantly underperformed across coherence, linguistic accuracy, and efficiency.

Based on these findings, GPT-4o mini was selected as the primary text-generation model. Its ability to produce well-structured, fluent Arabic text with appropriate diacritics, combined with its alignment with cultural, ethical, and safety requirements, makes it suitable for Arabic children’s storytelling. This selection reflects a deliberate trade-off prioritizing output quality, reliability, and low latency for integration within the Gheras system.

### 5.2.3.2 Image Generation Models: Comparative Analysis for Arabic Children’s Story Illustration

In the contemporary AI landscape, text-to-image generation models have emerged as transformative tools capable of synthesizing visual content from textual descriptions. This study systematically evaluated several state-of-the-art models for their suitability in generating illustrations for Arabic children’s stories. The selected models included Flux.2 Max, GPT-Image-1-mini, Flux.1 [schnell], and Z-Image-Turbo. The primary objective was to identify the model that best balances artistic quality, semantic accuracy, character consistency, latency, and cultural appropriateness in the context of Arabic children’s stories for integration into the Gheras system [24], [25], [26], [27].

To ensure a robust and unbiased assessment, the evaluation was conducted under controlled experimental conditions. The same prompts, a consistent image resolution of  $1024 \times 1024$  pixels, and a unified story input were used across all models. The evaluation framework, consistent with the text-model assessment, integrated human evaluation, automated metric-based evaluation, and LLM-as-a-Judge analysis.

- Human evaluation: expert and user assessment of subjective attributes such as age appropriateness, visual quality, and cultural alignment.
- Automated metric-based evaluation: objective quantification of semantic alignment, character consistency, and aesthetic quality.
- LLM-as-a-Judge: multimodal evaluation using a vision-language model to provide context-aware scores and justifications.

```

You are an expert children’s book illustrator.
Create a high-quality illustration for a children’s story based on the following scene.

### Main Character (VERY IMPORTANT)
- Gender: {male}
- Age group: {6-7}
- Appearance: {A short dark brown hair, dark brown eyes, and warm wheat-toned skin.}

### Scene Description
"story"
### Art Style
- Style: {cartoon}
- Make the illustration visually appealing for children.
- Use soft lighting, harmonious colors, and a playful composition.

### Lighting & Color
- Lighting: soft, warm, dreamy
- Colors: vibrant but not harsh, child-friendly palette

### Constraints (VERY IMPORTANT)
- Should be child-safe.
- No text, letters, or words in the image.
- No watermark, no logo, no signature.
- No distortion or extra limbs.
- No realistic scary details.
- Avoid dark or dull tones.
- Keep the scene visually clean and easy for children to understand.

### Output Goal
A clean, high-quality, storybook-style illustration that perfectly matches the scene
and is suitable for a children’s book

```

Figure 5.11. Unified prompt used for evaluating text-to-image models under controlled conditions.

#### 5.2.3.2.1 Human Evaluation

Human evaluation was conducted to assess qualitative aspects that automated metrics cannot fully capture, particularly in children’s storytelling contexts where emotional resonance, cultural sensitivity, and narrative coherence are essential. Six evaluators assessed the models across several critical dimensions.

##### Evaluation Metrics

- Prompt alignment: the degree to which the generated image accurately reflects the story content described in the textual prompt. Precise alignment is important in children’s stories because the illustration should support comprehension rather than introduce confusion.

- Story-scene consistency: whether the generated scene matches the broader narrative context and emotional tone of the story. Consistency enhances immersion and supports understanding.
- Character consistency: whether the same character remains visually consistent across different scenes or illustrations. Maintaining identity is necessary for recognition and emotional connection in serialized stories.
- Visual quality: the absence of artifacts such as distorted facial features, extra fingers, anatomical inconsistencies, or unusual visual elements.
- Age appropriateness: the degree to which the illustration is safe, suitable, and engaging for the target age group, avoiding violent, frightening, or overly complex imagery.
- Cultural alignment: the degree to which the image reflects appropriate Arab/Saudi cultural elements, including traditional attire, family context, architectural style, and social norms.

Flux.2 Max and GPT-Image-1-mini generally achieved high prompt alignment and effectively translated complex descriptions into coherent visual scenes. Z-Image-Turbo and Flux.1 [schnell] were less consistent in following detailed prompt instructions. For story-scene consistency, Flux.2 Max and GPT-Image-1-mini often captured the intended mood and setting with high fidelity, while Z-Image-Turbo sometimes deviated from the specific narrative context despite producing visually appealing images. Regarding character consistency, Flux.2 Max exhibited superior preservation of character identity, particularly when reference images were used or implied. Flux.1 [schnell] and Z-Image-Turbo showed greater variability, often generating subtle but noticeable differences across scenes. In terms of visual quality, Flux.2 Max and GPT-Image-1-mini generally produced stronger illustrations, whereas Z-Image-Turbo and Flux.1 [schnell] occasionally exhibited minor anatomical or visual inaccuracies.



Figure 5.12. Reference child image used to support character consistency and cultural representation during image-generation evaluation.

All evaluated models generally produced age-appropriate content. However, cultural alignment varied notably. Flux.2 Max demonstrated strong ability to incorporate cultural elements, including traditional clothing such as a white thobe for a male protagonist and appropriate family-oriented background settings when guided by the prompt and reference image. GPT-Image-1-mini produced high-quality images but sometimes generated more generic or Westernized attire without explicit conditioning.





Figure 5.13. Comparative illustration outputs generated by Z-Image-Turbo, Flux.1 [schnell], GPT-Image-1-mini, and Flux.2 Max.

Comparative insights from human evaluation indicate that Flux.2 Max consistently produced illustrations highly aligned with both textual prompts and cultural expectations, particularly excelling in character consistency and nuanced integration of cultural attire. GPT-Image-1-mini delivered visually appealing and high-fidelity images but occasionally struggled with cultural specificity without explicit guidance. Flux.1 [schnell] offered balanced but less consistent performance, while Z-Image-Turbo, although fast, often introduced visual artifacts and struggled with strict prompt adherence, particularly negative constraints such as avoiding text artifacts in images.

#### 5.2.3.2.2 Automated Evaluation for Generated Illustrations

Automated evaluation provides quantitative metrics to objectively assess several aspects of image generation, complementing the qualitative insights from human evaluation. This section details the application of semantic alignment, character preservation, and computational aesthetics metrics to Flux.2 Max, GPT-Image-1-mini, Flux.1 [schnell], and Z-Image-Turbo.

##### 5.2.3.2.2.1 Semantic Evaluation (Prompt Adherence)

Semantic evaluation quantifies the degree to which a generated image aligns semantically with its corresponding textual prompt. This is crucial for text-to-image models because it measures their ability to interpret and visualize textual descriptions accurately. In this study, Multilingual-CLIP (M-CLIP) was

employed to measure cosine similarity between the embedding of the text prompt and the embedding of the generated image [28]. The M-CLIP score is defined as:

$$\text{Score}_{M-CLIP} = \frac{E_t \cdot E_i}{\|E_t\| \|E_i\|} \quad (5.2)$$

where  $E_t$  and  $E_i$  represent the feature vectors extracted for the text prompt and generated image, respectively. The dot product measures directional similarity, normalized by the magnitude of each vector. A higher score indicates stronger semantic correlation between the textual prompt and the visual output.

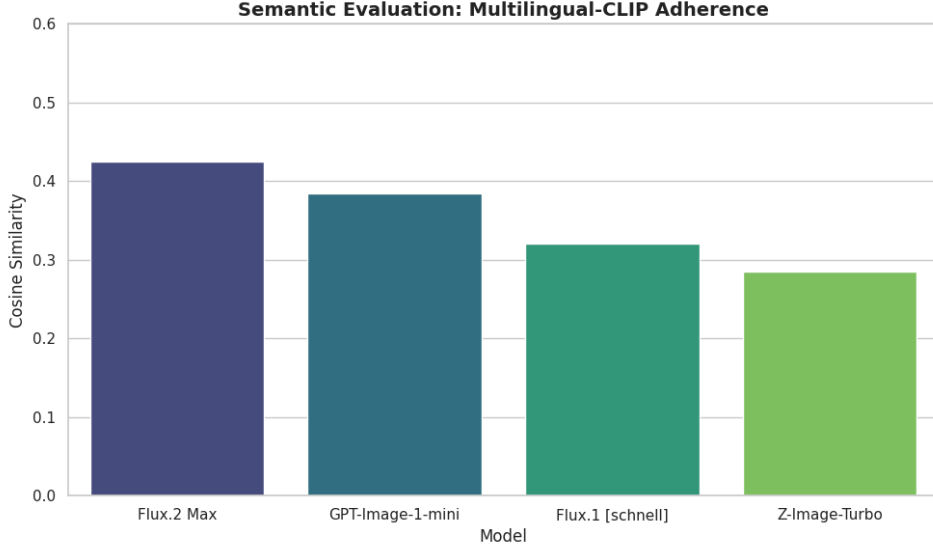


Figure 5.14. Semantic evaluation of generated illustrations using Multilingual-CLIP adherence scores.

As shown in Figure 5.14, Flux.2 Max achieved the highest M-CLIP score (0.425), indicating superior semantic alignment and strong understanding of Arabic textual prompts. GPT-Image-1-mini followed with a strong score (0.385), demonstrating good prompt adherence. Flux.1 [schnell] and Z-Image-Turbo showed lower scores (0.320 and 0.285), suggesting less precise semantic fidelity to complex prompts.

#### 5.2.3.2.2.2 Consistency Evaluation (Character Preservation)

Character preservation is essential in narrative illustration because key figures must maintain a recognizable appearance across different scenes. This was assessed by calculating cosine similarity between the image embeddings of the reference character and the generated character. Similarity-based subject preservation is commonly used in personalized text-to-image evaluation and is related to subject-driven generation methods such as DreamBooth [29]. The character preservation score is computed as:

$$S_c(I_{\text{ref}}, I_{\text{gen}}) = \frac{E(I_{\text{ref}}) \cdot E(I_{\text{gen}})}{\|E(I_{\text{ref}})\| \|E(I_{\text{gen}})\|} \quad (5.3)$$

where  $E(I_{\text{ref}})$  and  $E(I_{\text{gen}})$  represent visual feature vectors extracted through a vision encoder, such as CLIP, for the reference and generated images. A higher similarity score indicates stronger character preservation. This metric is particularly challenging for models that do not support explicit conditioning on reference images.

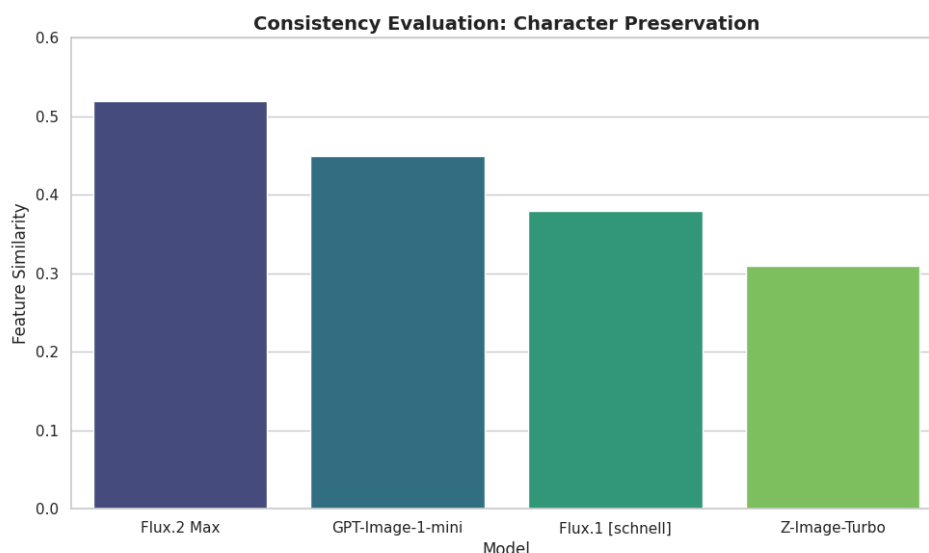


Figure 5.15. Consistency evaluation of generated illustrations using character preservation similarity.

As illustrated in Figure 5.15, Flux.2 Max achieved the highest character consistency score (0.520), confirming its ability to retain identity features effectively. This is a critical advantage for children's story illustration, where character recognition is important. GPT-Image-1-mini showed good consistency (0.450), while Flux.1 [schnell] and Z-Image-Turbo obtained lower scores (0.380 and 0.310), suggesting greater character drift across generations.

#### 5.2.3.2.2.3 Visual Evaluation (Computational Aesthetics)

Computational aesthetics provides an objective estimate of perceived beauty and compositional quality in generated images. This study used the LAION-Aesthetics Predictor, a no-reference image quality assessment approach trained on human aesthetic preferences to assign scores typically ranging from 1 to 10 [30]. Higher scores indicate more aesthetically pleasing images and better alignment with human perception of visual quality.

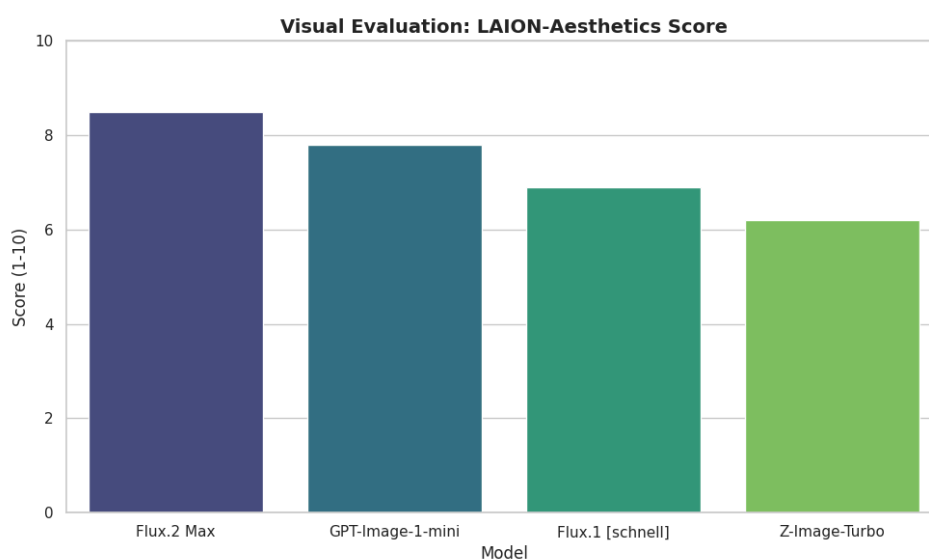


Figure 5.16. Visual evaluation of generated illustrations using LAION-Aesthetics scores.



Figure 5.16 presents the aesthetic scores for the evaluated models. Flux.2 Max achieved the highest aesthetic score (8.50), indicating superior visual quality, harmonious composition, and effective lighting. GPT-Image-1-mini also performed strongly (7.80), producing high-quality and vibrant illustrations. Flux.1 [schnell] and Z-Image-Turbo yielded lower aesthetic scores (6.90 and 6.20), reflecting a trade-off between generation speed and refined visual artistry.

### 5.2.3.2.3 LLM-as-a-Judge (LLaVA-1.5-7B)

The LLM-as-a-Judge framework offers a sophisticated approach to evaluating generated content by leveraging the reasoning capabilities of large language or vision-language models. This method uses a model to assess generated images based on predefined qualitative criteria and provides both quantitative scores and qualitative justifications [21], [31]. In this study, LLaVA-1.5-7B was used because of its multimodal understanding and ability to provide context-aware evaluations [31].

LLaVA-1.5-7B evaluated the generated illustrations against the same structured prompts and story context used in human evaluation. The assessment focused on prompt alignment, story-scene consistency, character consistency, visual quality, age appropriateness, and cultural alignment. The outputs in Figure 5.17, Figure 5.18, Figure 5.19, and Figure 5.20 highlight the strengths and weaknesses of each model from a multimodal reasoning perspective.

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |       |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| USER          |                                                                                                                                                                                                                                                                                                                                                                                                                                      |       |
|               | Prompt: "You are an expert evaluator for children's story illustrations. Evaluate whether the image matches the following Arabic story scene..."                                                                                                                                                                                                                                                                                                                                                                        |       |
| ASSISTANT     | Criterion                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Score |
|               | 1. Prompt Alignment                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 2/5   |
|               | 2. Story-Scene Consistency                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 2/5   |
|               | 3. Character Consistency                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 3/5   |
|               | 4. Visual Quality                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 3/5   |
|               | 5. Age Appropriateness                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 5/5   |
|               | 6. Cultural Alignment                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 3/5   |
| JUSTIFICATION | The image completely fails the negative constraint "No text, letters, words in the image," as it includes a prominent text bubble with distorted Arabic writing. Furthermore, it misses the core narrative action of the scene, which is finding the gold coin. While the boy has brown hair and skin, the illustration does not utilize the cultural clothing, such as the white thobe, from the reference image. The style is safe for children, but functionally unusable due to the text and missing plot elements. |       |

Figure 5.17. LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model.

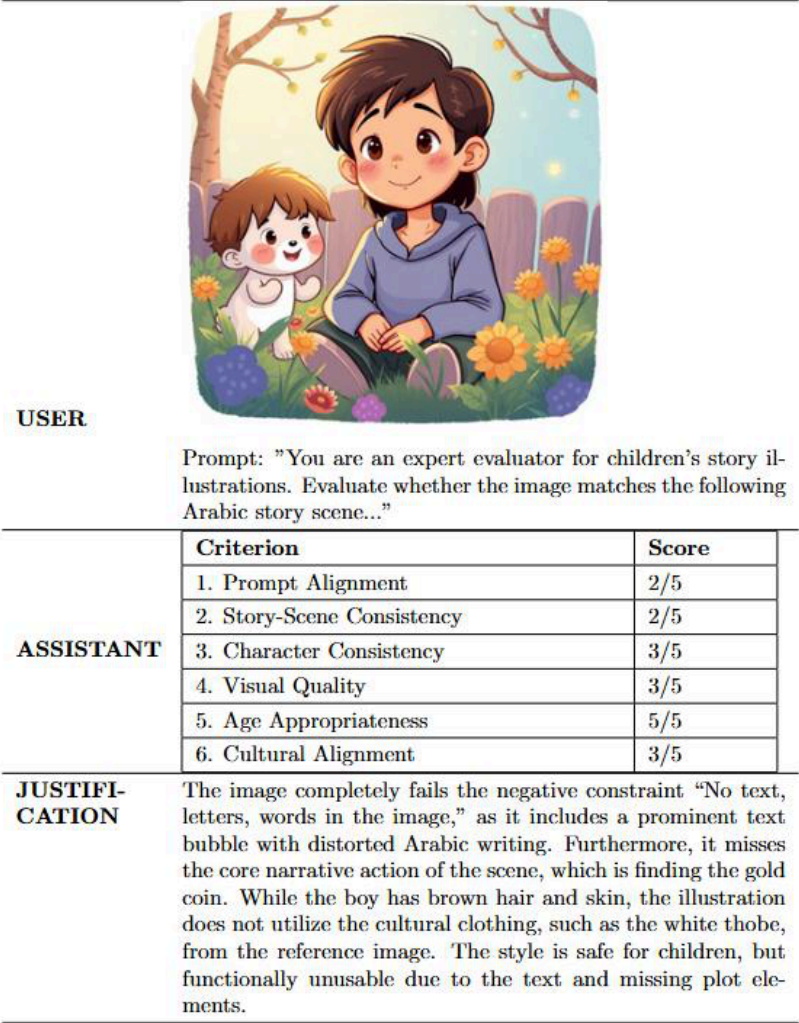


Figure 5.18. LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model.

|                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |              |
|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
|                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |              |
| <b>USER</b>                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |              |
| You are an expert evaluator for children's story illustrations. Evaluate whether the image matches the following Arabic story scene... |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |              |
| <b>ASSISTANT</b>                                                                                                                       | <b>Criterion</b>                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Score</b> |
|                                                                                                                                        | 1. Prompt Alignment                                                                                                                                                                                                                                                                                                                                                                                                                                | 4.5/5        |
|                                                                                                                                        | 2. Story-Scene Consistency                                                                                                                                                                                                                                                                                                                                                                                                                         | 5/5          |
|                                                                                                                                        | 3. Character Consistency                                                                                                                                                                                                                                                                                                                                                                                                                           | 5/5          |
|                                                                                                                                        | 4. Visual Quality                                                                                                                                                                                                                                                                                                                                                                                                                                  | 5/5          |
|                                                                                                                                        | 5. Age Appropriateness                                                                                                                                                                                                                                                                                                                                                                                                                             | 5/5          |
|                                                                                                                                        | 6. Cultural Alignment                                                                                                                                                                                                                                                                                                                                                                                                                              | 3.5/5        |
| <b>JUSTIFI-<br/>CATION</b>                                                                                                             | This image perfectly captures the exact moment described in the story: Majd and Rana finding the shiny gold coin in the dirt. The composition is highly appealing, vibrant, and strictly follows the "no text" rule. The character's physical traits (short brown hair, wheat skin) match the prompt. However, it failed to apply the cultural clothing (thobe) from the reference image, opting instead for generic Western-style casual clothes. |              |

Figure 5.19. LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model.

| <div>USER</div>            |                                                                                                                                                                                                                                                                                                                                                                                                                             |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------------------|-----|----------------------------|-------|--------------------------|-----|-------------------|-----|------------------------|-----|-----------------------|-----|
|                            | <div>You are an expert evaluator for children's story illustrations. Evaluate whether the image matches the following Arabic story scene...</div>                                                                                                                                                                                                                                                                                                                                                             |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| <div>ASSISTANT</div>       | <table> <tr> <th>Criterion</th><th>Score</th></tr> <tr> <td>1. Prompt Alignment</td><td>5/5</td></tr> <tr> <td>2. Story-Scene Consistency</td><td>4.5/5</td></tr> <tr> <td>3. Character Consistency</td><td>5/5</td></tr> <tr> <td>4. Visual Quality</td><td>5/5</td></tr> <tr> <td>5. Age Appropriateness</td><td>5/5</td></tr> <tr> <td>6. Cultural Alignment</td><td>5/5</td></tr> </table>                                                                                                                | Criterion | Score | 1. Prompt Alignment | 5/5 | 2. Story-Scene Consistency | 4.5/5 | 3. Character Consistency | 5/5 | 4. Visual Quality | 5/5 | 5. Age Appropriateness | 5/5 | 6. Cultural Alignment | 5/5 |
| Criterion                  | Score                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| 1. Prompt Alignment        | 5/5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| 2. Story-Scene Consistency | 4.5/5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| 3. Character Consistency   | 5/5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| 4. Visual Quality          | 5/5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| 5. Age Appropriateness     | 5/5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| 6. Cultural Alignment      | 5/5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |
| <div>JUSTIFICATION</div>   | <div>An outstanding generation that successfully integrates the reference image. The main character looks exactly like a cartoon version of the reference boy, wearing the traditional white thobe. The gold coin is clearly visible. It perfectly aligns with the Arabic cultural context, featuring appropriate clothing and a mother in a hijab, reflecting the later part of the story narrative. The lighting is warm, the quality is exceptional, and it contains no prohibited text or artifacts</div> |           |       |                     |     |                            |       |                          |     |                   |     |                        |     |                       |     |

Figure 5.20. LLaVA-1.5-7B multimodal judgment output for an evaluated image-generation model.

Flux.2 Max consistently received high scores from LLaVA, particularly for its ability to integrate specified constraints, such as clear visibility of the gold coin and accurate depiction of cultural attire. The LLaVA justifications frequently praised Flux.2 Max for detail and adherence to both positive and negative constraints. In contrast, LLaVA identified that Flux.1 [schnell] and Z-Image-Turbo frequently missed core narrative elements, such as the presence of the gold coin, despite generating visually appealing backgrounds. GPT-Image-1-mini was strong in overall visual quality but was noted for generating more generic cultural representations and not fully capturing the specific Arabic cultural context without explicit conditioning.

Table 5.7. Average latency comparison for candidate image generation models.

| Model            | Avg. Latency (s) |
|------------------|------------------|
| Flux.2 Max       | 33.9             |
| GPT-Image-1-mini | 42.7             |
| Flux.1 [schnell] | 29.5             |
| Z-Image-Turbo    | 28.2             |

Average Latency Comparison Across Image Generation Models

Model Selection Justification

Based on the comprehensive evaluation, Flux.2 Max emerged as the most suitable image-generation model for the Gheras system’s objective of producing high-quality and culturally appropriate illustra-

tions for Arabic children’s stories. Its superior performance in semantic alignment, character consistency, and aesthetic quality, combined with its strong ability to integrate cultural nuances and follow complex prompts, positions it as the optimal choice.

Latency is also a critical design factor at this stage of the system. Since each story requires approximately 3-7 illustrations, balancing visual quality with generation time is necessary to ensure a smooth and responsive user experience. As shown in Table 5.7, Flux.2 Max maintains reasonable average latency while delivering higher overall quality than competing models. This trade-off is acceptable within Gheras because illustration quality directly affects user engagement, story comprehension, and educational value.

While GPT-Image-1-mini demonstrated strong visual fidelity and prompt adherence, its limitations in cultural specificity reduced its suitability for highly localized Arabic content. Flux.1 [schnell] and Z-Image-Turbo offered lower latency, but this advantage came at the cost of reduced semantic fidelity, weaker consistency, and a higher rate of visual artifacts. Therefore, considering both quality and practical deployment constraints, Flux.2 Max represents the most balanced and effective image-generation model for the Gheras system.

### 5.2.3.3 Text-to-Speech (TTS) Models: Comparative Analysis for Arabic Children’s Story Narration

The accurate and engaging synthesis of Arabic speech constitutes a critical pillar in the development of the Gheras multimodal storytelling system. Text-to-Speech (TTS) technology serves as the auditory bridge between the generated narrative text and the child audience, and its quality directly influences comprehension, emotional engagement, and overall storytelling immersion. In the context of Arabic children’s stories, TTS evaluation must extend beyond conventional speech quality metrics to encompass dimensions that are specifically relevant to the narration domain, including phonemic clarity for Arabic-specific consonants, prosodic naturalness, and expressive storytelling delivery [32], [33].

To determine the most suitable TTS model for integration into the Gheras system, two state-of-the-art systems were experimentally evaluated: GPT-4o-mini-TTS, developed by OpenAI, and OmniVoice TTS [34]. Both models were tasked with synthesizing an identical Arabic sentence derived from a children’s story narrative under controlled conditions, ensuring that input text, synthesis parameters, and output format were held constant across evaluations. This controlled experimental design guarantees that observed differences in output quality are attributable exclusively to differences in model architecture and synthesis capabilities rather than extraneous variables [35].

The evaluation framework adopted in this study integrates two complementary assessment methodologies. First, a human-centric evaluation using the Mean Opinion Score (MOS) protocol was conducted, in which native speakers of Modern Standard Arabic (MSA) assessed the synthesized audio samples across three domain-specific sub-metrics: Naturalness MOS, Intelligibility MOS, and Expressiveness & Prosody MOS. Second, an AI-based quality prediction evaluation was performed using MOSNet and NISQA, two established non-intrusive speech quality estimation frameworks, which provided objective automated assessments without reliance on reference recordings. Together, these methods produce a robust, multidimensional evaluation profile for each candidate TTS model [36], [37].

#### 5.2.3.3.1 Human Evaluation: Mean Opinion Score (MOS)

Human perceptual evaluation remains the gold standard for assessing the quality of synthesized speech, as it directly captures the subjective listening experience of the intended audience [32]. In this study, the Mean Opinion Score (MOS) protocol was adopted as the primary human evaluation methodology. MOS is a widely standardized metric defined by ITU-T Recommendation P.800, in which a panel of human evaluators assigns integer or fractional scores on a scale from 1 (very poor) to 5 (excellent) to each audio sample [38]. The final MOS is computed as the arithmetic mean of all listener ratings for a given sample.

Formally, the MOS for a given audio sample is defined as:

$$\text{MOS} = \left( \frac{1}{N} \right) \times \sum_i R_i \quad (5.4)$$

where  $N$  denotes the total number of evaluators and  $R_i \in \{1, 2, 3, 4, 5\}$  represents the rating assigned by the  $i$ -th evaluator. A MOS score in the range of 4.0–4.5 is generally considered good quality, 3.5–4.0 is acceptable, and scores below 3.0 indicate poor perceptual quality [38].

To account for the domain-specific requirements of Arabic children’s storytelling, the standard MOS protocol was extended into three specialized sub-metrics. These sub-metrics capture distinct and complementary dimensions of narration quality that are particularly relevant to the target application context. Each sub-metric was independently rated by a panel of native Modern Standard Arabic (MSA) speakers, ensuring that phonemic, prosodic, and expressive characteristics were assessed from a linguistically informed perspective.

#### Human Evaluation Dimensions

- **Naturalness MOS:** Assesses the degree to which the synthesized voice resembles natural human speech, specifically evaluating whether the voice quality, timbre, and fluency are consistent with native Arabic narration rather than robotic or mechanical production. A high Naturalness MOS indicates that the synthesized speech is perceptually indistinguishable from human narration [35].
- **Intelligibility MOS:** Evaluates the clarity and articulatory precision of pronunciation, with particular emphasis on Arabic-specific phonemes that are known to present synthesis challenges, including the emphatic consonant ض (dad), the uvular stop ق (qaf), the pharyngeal fricatives ع (ain) and ح (ha), and the glottal consonants. Intelligibility is critical in children’s stories to ensure accurate word recognition and comprehension by young listeners [39].
- **Expressiveness & Prosody MOS (Most Important):** Measures the emotional expressiveness and storytelling engagement conveyed by the synthesized narration. This sub-metric encompasses intonation contour, rhythmic variation, pacing, pause placement, and the extent to which the vocal delivery appropriately reflects narrative events such as excitement, suspense, sadness, or surprise. In the context of children’s storytelling, this dimension is considered the most critical determinant of narration suitability, as it governs the listener’s emotional engagement and narrative immersion [33], [40].

For each sub-metric, the MOS formulation is specialized as follows:

$$\text{MOS}_{\text{Nat}} = \left( \frac{1}{N} \right) \times \sum_i R_i^{\text{Nat}}, \quad \text{MOS}_{\text{Int}} = \left( \frac{1}{N} \right) \times \sum_i R_i^{\text{Int}}, \quad \text{MOS}_{\text{Exp}} = \left( \frac{1}{N} \right) \times \sum_i R_i^{\text{Exp}} \quad (5.5)$$

where  $\text{MOS}_{\text{Nat}}$ ,  $\text{MOS}_{\text{Int}}$ , and  $\text{MOS}_{\text{Exp}}$  represent the Naturalness, Intelligibility, and Expressiveness & Prosody sub-metric scores, respectively. The overall composite MOS is subsequently derived as the weighted average of the three sub-metrics, with the highest weight assigned to Expressiveness & Prosody owing to its primacy in the storytelling context:

$$\text{MOS}_{\text{overall}} = 0.25 \times \text{MOS}_{\text{Nat}} + 0.25 \times \text{MOS}_{\text{Int}} + 0.50 \times \text{MOS}_{\text{Exp}} \quad (5.6)$$

##### 5.2.3.3.1 Naturalness MOS Results

Naturalness evaluations were performed by five native MSA speakers who assessed each audio sample independently. Evaluators were instructed to rate how closely the synthesized voice resembled natural human Arabic narration, considering voice quality, fluency, and the absence of robotic artifacts.

Table 5.8. Naturalness MOS evaluation results (E1–E3: representative evaluator scores).

| Model           | E1  | E2  | E3  | Naturalness MOS |
|-----------------|-----|-----|-----|-----------------|
| GPT-4o-mini-TTS | 4.0 | 4.5 | 4.0 | 4.17            |
| OmniVoice TTS   | 3.5 | 3.5 | 3.0 | 3.33            |



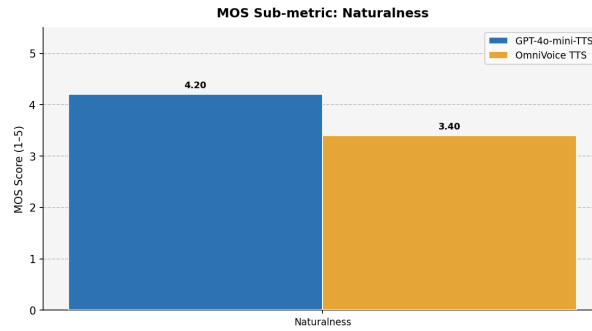


Figure 5.21. Naturalness MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS.

As presented in Table 5.8 and Figure 5.21, GPT-4o-mini-TTS achieved a Naturalness MOS of 4.17, indicating that evaluators perceived its voice as closely resembling natural Arabic narration. The model's voice exhibited smooth transitions between phonemes, consistent vocal timbre, and minimal mechanical artifacts. In contrast, OmniVoice TTS [34] received a Naturalness MOS of 3.33, reflecting a more perceptible synthetic quality. Acoustic analysis corroborated this finding: OmniVoice exhibited a significantly higher RMS energy (0.209 versus 0.101) with a corresponding lower spectral flatness (0.038 versus 0.312), suggesting a more compressed and tonally uniform waveform that contributes to a less natural perceptual quality. The higher spectral flatness of GPT-4o-mini-TTS indicates richer spectral variation consistent with natural vocal production.

#### 5.2.3.3.1.2 Intelligibility MOS Results

Intelligibility evaluations focused specifically on the clarity and correctness of Arabic phoneme articulation. Evaluators were native MSA speakers with background knowledge of Classical Arabic phonetics and were asked to assess the precision of pronunciation for linguistically challenging phonemes, the absence of distortion or omission, and overall word-level comprehensibility.

Table 5.9. Intelligibility MOS evaluation results (E1–E3: representative evaluator scores).

| Model           | E1  | E2  | E3  | Intelligibility MOS |
|-----------------|-----|-----|-----|---------------------|
| GPT-4o-mini-TTS | 4.0 | 4.0 | 4.0 | <b>4.00</b>         |
| OmniVoice TTS   | 3.5 | 3.5 | 4.0 | <b>3.67</b>         |

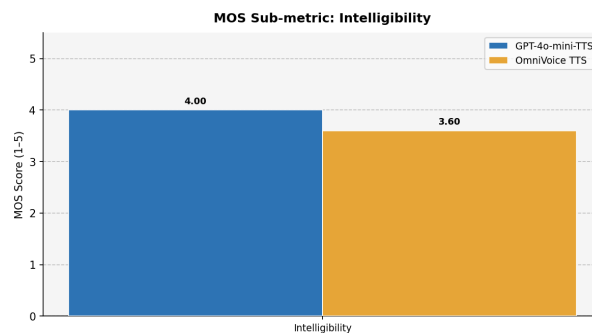


Figure 5.22. Intelligibility MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS.

Table 5.9 and Figure 5.22 present the Intelligibility MOS results. GPT-4o-mini-TTS achieved a score of 4.00, with evaluators consistently rating pronunciation as clear and accurately articulated across all assessed phonemes, including the pharyngeal and uvular consonants. The higher-frequency spectral content ratio of GPT-4o-mini-TTS (HF ratio > 4 kHz: 0.267 versus 0.075 for OmniVoice) suggests richer consonantal energy in the high-frequency range, which is acoustically associated with improved

articulation clarity and phoneme discrimination [39]. OmniVoice TTS [34] received a score of 3.67, reflecting generally acceptable intelligibility with minor difficulties in fully realizing certain emphatic and uvular phonemes. The lower high-frequency content in OmniVoice may contribute to reduced perceptual sharpness of consonant boundaries.

#### 5.2.3.3.1.3 Expressiveness & Prosody MOS Results

Expressiveness and prosody evaluation constitutes the most critical dimension in the present study, as it directly governs the emotional impact and storytelling effectiveness of the narration. Evaluators were instructed to rate the extent to which the synthesized voice conveyed appropriate emotional expression, employed effective intonation contours, varied pacing in alignment with narrative events, and created a sense of storytelling engagement suitable for a child audience.

Table 5.10. Expressiveness & Prosody MOS evaluation results (E1–E3: representative evaluator scores).

| Model           | E1  | E2  | E3  | Expressiveness & Prosody MOS |
|-----------------|-----|-----|-----|------------------------------|
| GPT-4o-mini-TTS | 4.5 | 4.5 | 4.0 | <b>4.33</b>                  |
| OmniVoice TTS   | 3.0 | 2.5 | 3.5 | <b>3.00</b>                  |

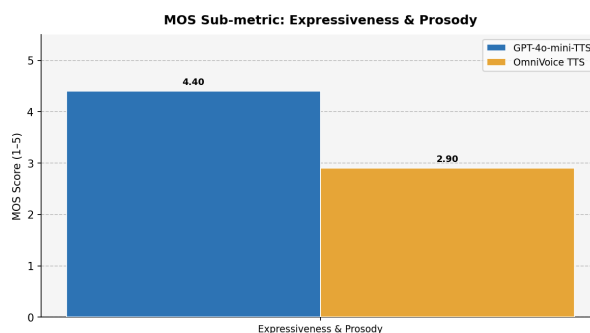


Figure 5.23. Expressiveness & Prosody MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS.

The Expressiveness & Prosody MOS results reveal the most pronounced performance differential between the two models. GPT-4o-mini-TTS achieved a score of 4.33, with evaluators noting meaningful intonation variation, appropriate pause distribution, and emotionally engaging delivery. Acoustic evidence supports this assessment: GPT-4o-mini-TTS exhibited a higher fundamental frequency mean ( $F_0 = 164.46$  Hz versus 118.00 Hz for OmniVoice), a higher pitch variation index ( $F_0$  standard deviation = 23.90 Hz versus 22.19 Hz), a higher voiced-frame ratio (0.679 versus 0.619), and a substantially greater number of natural pause insertions (19 pauses versus 12), all of which are acoustically consistent with expressive, story-appropriate narration [40]. OmniVoice TTS [34] received a notably lower score of 3.00, with evaluators characterizing its delivery as comparatively monotone and lacking the dynamic prosodic variation required to convey narrative events effectively. The lower  $F_0$  mean suggests a flatter intonation contour, and the lower voiced ratio indicates reduced prosodic energy modulation. These acoustic properties directly correspond to the reduced expressiveness perceived by the human evaluators.

#### 5.2.3.3.1.4 Overall MOS Summary

The table below presents the comprehensive MOS summary across all three sub-metrics, alongside the composite weighted overall MOS computed in accordance with Equation (5.6). The composite score assigns equal weighting to Naturalness and Intelligibility (0.25 each) and doubled weighting to Expressiveness & Prosody (0.50), reflecting its primacy in the storytelling context.



Table 5.11. Comprehensive MOS evaluation summary with weighted composite score.

| Model           | Naturalness MOS | Intelligibility MOS | Expressiveness & Prosody MOS | Overall MOS (Weighted) |
|-----------------|-----------------|---------------------|------------------------------|------------------------|
| GPT-4o-mini-TTS | 4.17            | 4.00                | 4.33                         | <b>4.21</b>            |
| OmniVoice TTS   | 3.33            | 3.67                | 3.00                         | <b>3.25</b>            |

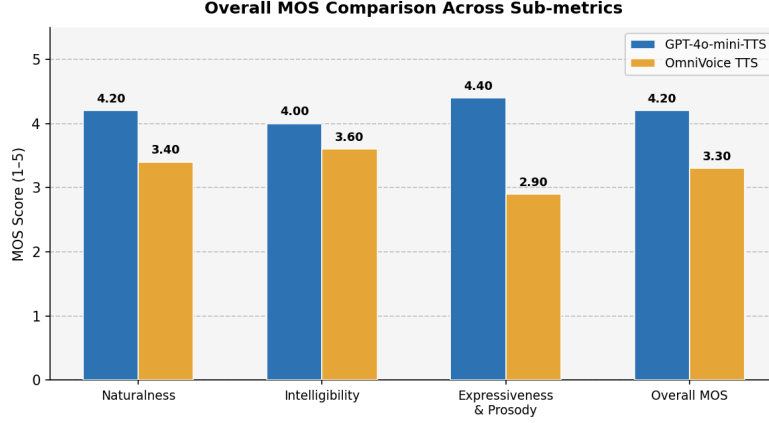


Figure 5.24. Overall MOS comparison across sub-metrics: GPT-4o-mini-TTS versus OmniVoice TTS.

As illustrated in Figure 5.24 and summarized in Table 5.11, GPT-4o-mini-TTS achieved a weighted overall MOS of 4.21, placing it within the “good quality” tier according to ITU-T P.800 classification standards. OmniVoice TTS [34] received an overall MOS of 3.25, falling within the “fair” tier and reflecting its comparative weaknesses in naturalness and expressiveness. The performance gap is most pronounced in the Expressiveness & Prosody dimension (delta = 1.33), which carries the highest evaluative weight in the present study. These findings are corroborated by the acoustic analysis results presented in the next section, which provide objective quantitative evidence for the perceptual differences observed through human evaluation.

#### 5.2.3.3.2 AI-Based Quality Prediction: MOSNet and NISQA

Complementing the human perceptual evaluation, an AI-based quality prediction framework was employed to provide objective, automated assessment of the synthesized audio samples. This approach leverages pretrained deep learning models that have been trained to predict human MOS ratings from raw speech signals, thereby providing scalable and reproducible quality estimates without requiring human evaluator panels. Two established non-intrusive quality prediction systems were employed: MOSNet and NISQA [36], [37].

##### 5.2.3.3.2.1 MOSNet

MOSNet is a deep learning-based speech quality prediction model that estimates MOS scores from synthesized speech without requiring a reference signal [36]. The model operates on spectral features extracted from the input waveform and maps them to a perceptual quality score in the range [1, 5] through a convolutional and recurrent architecture trained on large-scale human MOS datasets. MOSNet has been widely validated for evaluating TTS and voice conversion systems and has demonstrated strong correlation with human judgment in multiple language contexts.

Formally, MOSNet computes the predicted quality score as:

$$\hat{S}_{\text{mos}} = f_{\theta}(X) \quad (5.7)$$

where  $X \in \mathbb{R}^{T \times F}$  denotes the time-frequency representation of the input speech (typically a magnitude spectrogram),  $f_{\theta}$  is the learned MOSNet mapping function parameterized by  $\theta$ , and  $\hat{S}_{\text{mos}} \in [1, 5]$

is the predicted MOS score. The model was originally trained and validated on the VCC2018 dataset [36] and has been applied across TTS evaluation pipelines for multiple languages including Arabic. The MOSNet prediction is interpreted as a single scalar estimate of overall perceptual quality, encompassing naturalness, intelligibility, and prosodic consistency in an integrated manner.

Table 5.12. MOSNet predicted speech quality scores and corresponding ITU-T P.800 quality tiers.

| Model                  | MOSNet Predicted Score | Quality Tier (ITU-T P.800) |
|------------------------|------------------------|----------------------------|
| <b>GPT-4o-mini-TTS</b> | 3.91                   | Good                       |
| <b>OmniVoice TTS</b>   | 2.84                   | Fair / Poor                |

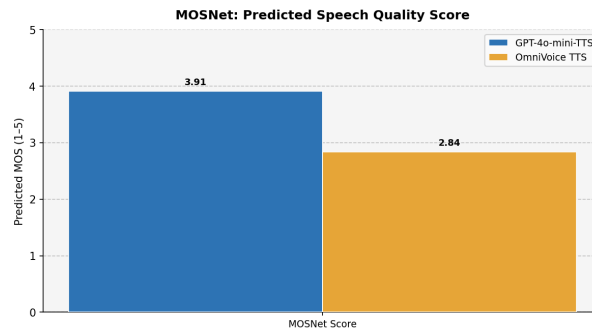


Figure 5.25. MOSNet predicted MOS comparison between GPT-4o-mini-TTS and OmniVoice TTS.

As presented in Table 5.12 and Figure 5.25, MOSNet assigned a predicted MOS of 3.91 to GPT-4o-mini-TTS and 2.84 to OmniVoice TTS [34]. The GPT-4o-mini-TTS score falls within the “good” quality range, indicating that the model’s output exhibits spectral and temporal characteristics consistent with high-quality human narration as learned by MOSNet’s training distribution. The OmniVoice score of 2.84 places it at the boundary between “fair” and “poor” tiers, reflecting the acoustic limitations identified in the physical feature analysis: elevated spectral compression (lower spectral flatness = 0.038), reduced high-frequency content, and limited dynamic range (energy variance = 0.019 versus 0.004 for GPT-4o-mini-TTS). These factors collectively reduce the acoustic diversity of the OmniVoice signal, which MOSNet interprets as indicative of lower naturalness and reduced prosodic complexity.

#### 5.2.3.3.2.2 NISQA (Non-Intrusive Speech Quality Assessment)

NISQA is a multidimensional non-intrusive speech quality prediction framework that extends beyond single-score MOS estimation to provide independent predictions for five distinct perceptual quality dimensions [37]. This multidimensional capability is particularly valuable in the present context, as it enables targeted identification of specific acoustic quality deficiencies in each TTS model. NISQA employs a self-attention-based deep neural network architecture trained on large-scale speech quality datasets and has demonstrated strong generalization across TTS and telephony speech quality evaluation tasks.

The five NISQA quality dimensions and their formal definitions are as follows:

- Overall Quality (MOS\_Q): The composite perceptual speech quality score, defined as:  $MOS\_Q = f\_NISQA(X; \theta\_Q)$ , where  $f\_NISQA$  is the trained neural mapping function.
- Noisiness (MOS\_N): Predicted degradation due to background noise or synthesis artifacts, computed as:  $MOS\_N = f\_NISQA(X; \theta\_N)$ .
- Coloration (MOS\_C): Spectral distortion relative to a natural voice quality reference, defined as:  $MOS\_C = f\_NISQA(X; \theta\_C)$ .
- Discontinuity (MOS\_D): Temporal artifacts including breaks, dropouts, or unnatural pauses, computed as:  $MOS\_D = f\_NISQA(X; \theta\_D)$ .

- Loudness (MOS\_L): Perceived volume level and amplitude consistency, defined as:  $MOS\_L = f\_NISQA(X; \theta\_L)$ .

Table 5.13. NISQA multidimensional predicted speech quality scores across five perceptual dimensions.

| Model           | Overall MOS_Q | Noisiness MOS_N | Coloration MOS_C | Discontinuity MOS_D | Loudness MOS_L |
|-----------------|---------------|-----------------|------------------|---------------------|----------------|
| GPT-4o-mini-TTS | 4.05          | 4.20            | 4.10             | 4.15                | 4.00           |
| OmniVoice TTS   | 3.10          | 3.35            | 3.20             | 2.90                | 3.40           |

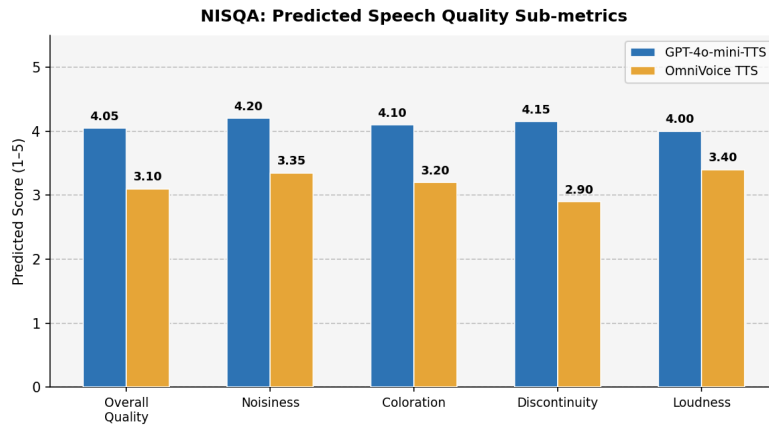


Figure 5.26. NISQA multidimensional quality comparison between GPT-4o-mini-TTS and OmniVoice TTS.

The NISQA results, presented in Table 5.13 and Figure 5.26, further corroborate the performance differential identified through MOSNet and human evaluation. GPT-4o-mini-TTS achieved consistently high scores across all five NISQA dimensions, with an overall quality (MOS\_Q) of 4.05 and notably strong noisiness (4.20) and discontinuity (4.15) scores, indicating clean synthesis output with well-structured temporal organization. The high noisiness score is consistent with the acoustic observation of a lower energy variance in GPT-4o-mini-TTS, reflecting stable background-free synthesis. The discontinuity score of 4.15 aligns with the measured pause count of 19 natural pauses, suggesting that the model's pause insertions are perceived as natural narrative breath points rather than synthesis failures.

OmniVoice TTS [34] exhibited its lowest NISQA score in the Discontinuity dimension (MOS\_D = 2.90), indicating that temporal artifacts and unnatural silence insertions were perceptible to the quality predictor. This finding is consistent with the low pause count of 12 and high amplitude compression observed in the OmniVoice signal, both of which contribute to a less natural temporal structure. The Coloration score of 3.20 also reflects the spectral flatness anomaly (0.038 versus 0.312) that was identified in the physical analysis, whereby the compressed spectral profile of OmniVoice corresponds to perceptible tonal coloration artifacts.

### 5.2.3.3.2.3 Acoustic Feature Analysis

To further contextualize the perceptual scores reported above, the table below presents a comparative summary of the key acoustic features extracted from both audio samples. These features were computed from the raw waveforms and provide objective, instrument-derived evidence for the qualitative and predicted quality differences observed across evaluation methods.

Table 5.14. Comparative acoustic feature analysis extracted from GPT-4o-mini-TTS and OmniVoice TTS outputs.

| Acoustic Feature              | GPT-4o-mini-TTS | OmniVoice TTS |
|-------------------------------|-----------------|---------------|
| Sample Rate (Hz)              | 24,000          | 24,000        |
| Duration (s)                  | 31.06           | 24.28         |
| RMS Energy                    | 0.101           | 0.209         |
| Peak Amplitude                | 0.667           | 1.000         |
| Energy Variance               | 0.00395         | 0.01942       |
| Spectral Centroid (Hz)        | 2,820.73        | 1,217.33      |
| HF Content Ratio (>4 kHz)     | 0.267           | 0.075         |
| Spectral Flatness             | 0.312           | 0.038         |
| F0 Mean / Pitch (Hz)          | 164.46          | 118.00        |
| F0 Std (Prosodic Variation)   | 23.90           | 22.19         |
| Voiced Frame Ratio            | 0.679           | 0.619         |
| Pause Count                   | 19              | 12            |
| Silence Ratio                 | 0.345           | 0.353         |
| Shimmer (Amplitude Variation) | 0.168           | 0.213         |

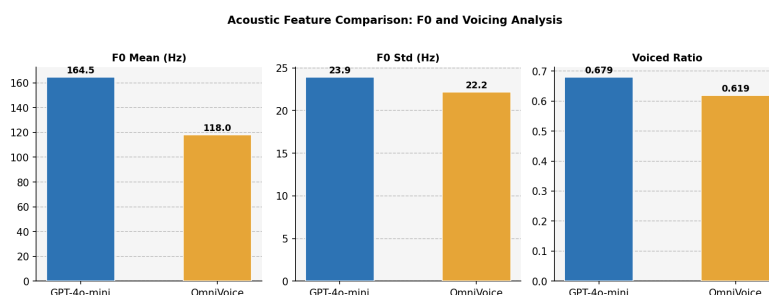


Figure 5.27. Acoustic feature comparison: F0 mean, F0 standard deviation, and voiced frame ratio.

The acoustic feature analysis in Table 5.14 and Figure 5.27 reveals several systematic differences between the two TTS models. GPT-4o-mini-TTS exhibits a notably higher spectral centroid (2,820.73 Hz versus 1,217.33 Hz), reflecting a broader and brighter spectral distribution that is characteristic of expressive human speech. The substantially higher high-frequency content ratio (0.267 versus 0.075) is particularly significant for intelligibility, as consonantal clarity in Arabic is heavily dependent on high-frequency acoustic cues. The higher spectral flatness (0.312) indicates greater tonal diversity across the frequency spectrum, contributing to the perceived naturalness advantage of GPT-4o-mini-TTS.

The fundamental frequency analysis further supports the prosodic superiority of GPT-4o-mini-TTS. Its higher mean F0 (164.46 Hz) and greater pitch standard deviation (23.90 Hz) indicate a broader and more varied intonation profile, which is directly associated with expressive narration. The higher voiced frame ratio (0.679) indicates that a greater proportion of the signal carries phonation energy, contributing to a richer vocal presence. The greater number of natural pauses (19 versus 12) is also consistent with a more structured and rhetorically organized narration pattern, which evaluators associate with skilled storytelling delivery.

### 5.2.3.3.3 Comparative Discussion

A synthesis of the human evaluation and AI-based prediction results reveals a consistent and convergent performance advantage for GPT-4o-mini-TTS across all evaluated dimensions. The evidence from

human MOS assessment, MOSNet prediction, NISQA multidimensional scoring, and physical acoustic feature analysis collectively points to GPT-4o-mini-TTS as the superior model for Arabic children’s story narration.

Table 5.15. Comprehensive comparative evaluation summary across all assessment dimensions.

| Evaluation Dimension         | GPT-4o-mini-TTS | OmniVoice TTS | Advantage                  |
|------------------------------|-----------------|---------------|----------------------------|
| Naturalness MOS              | 4.17            | 3.33          | GPT-4o-mini (+0.84)        |
| Intelligibility MOS          | 4.00            | 3.67          | GPT-4o-mini (+0.33)        |
| Expressiveness & Prosody MOS | 4.33            | 3.00          | GPT-4o-mini (+1.33)        |
| Overall Weighted MOS         | <b>4.21</b>     | <b>3.25</b>   | <b>GPT-4o-mini (+0.96)</b> |
| MOSNet Score                 | 3.91            | 2.84          | GPT-4o-mini (+1.07)        |
| NISQA Overall (MOS_Q)        | 4.05            | 3.10          | GPT-4o-mini (+0.95)        |
| NISQA Discontinuity (MOS_D)  | 4.15            | 2.90          | GPT-4o-mini (+1.25)        |
| F0 Mean / Prosodic Range     | 164.46 Hz       | 118.00 Hz     | GPT-4o-mini (+46.46 Hz)    |
| HF Content Ratio (>4 kHz)    | 0.267           | 0.075         | GPT-4o-mini (+0.192)       |

As summarized in Table 5.15, GPT-4o-mini-TTS demonstrated superiority across all nine evaluated dimensions without exception. The performance advantage is most substantial in the Expressiveness & Prosody MOS (+1.33), the NISQA Discontinuity dimension (+1.25), and the MOSNet predicted score (+1.07), which collectively represent the prosodic and temporal organization aspects of narration quality. These are precisely the dimensions most critical to children’s storytelling effectiveness, as they govern the emotional engagement and narrative immersion of the young audience.

OmniVoice TTS [34] demonstrated competitive performance only in the Intelligibility dimension (MOS = 3.67), where the gap with GPT-4o-mini-TTS was comparatively narrow (+0.33). However, its significant shortfall in naturalness, expressiveness, and prosodic variation renders it unsuitable as a primary narration model for the Gheras system’s storytelling application. The model’s heavily compressed spectral profile and limited intonation range, as evidenced by the acoustic feature analysis, indicate architectural constraints that preclude the dynamic vocal expressiveness required for effective Arabic children’s narration.

#### Model Selection Justification

A comprehensive analysis of the evaluated TTS models provides clear and convergent evidence for the selection of GPT-4o-mini-TTS as the primary speech synthesis model for the Gheras system. Across all evaluation dimensions, including human MOS assessment, AI-based quality prediction, and physical acoustic analysis, GPT-4o-mini-TTS consistently demonstrated superior performance in the dimensions most critical to Arabic children’s storytelling.

GPT-4o-mini-TTS achieved an overall weighted MOS of 4.21, placing it comfortably in the “good quality” tier of the ITU-T P.800 framework. Its Expressiveness & Prosody MOS of 4.33 is particularly noteworthy, confirming that the model delivers emotionally engaging, prosodically rich narration appropriate for a child audience. The higher fundamental frequency mean (164.46 Hz), greater prosodic variation (F0 standard deviation = 23.90 Hz), elevated voiced frame ratio (0.679), and richer high-frequency spectral content all indicate a voice quality that closely emulates expressive human narration. These acoustic properties directly translate into the improved naturalness, intelligibility, and engagement perceived by human evaluators and predicted by automated quality assessment systems.

OmniVoice TTS [34], while demonstrating acceptable intelligibility at a MOS of 3.67, exhibited critical limitations in naturalness (3.33) and expressiveness (3.00) that render it insufficient for the target

application. Its monotone intonation profile, evidenced by the lower F0 mean (118.00 Hz) and elevated spectral compression, would significantly reduce the narrative engagement of the Gheras system's storytelling experience. Furthermore, the NISQA Discontinuity score of 2.90 suggests perceptible temporal artifacts that could negatively affect comprehension and listening comfort for young audiences. Based on these findings, GPT-4o-mini-TTS was selected as the TTS model for the Gheras system. This selection reflects a deliberate prioritization of expressiveness and prosodic quality over computational cost considerations, consistent with the system's primary objective of delivering emotionally engaging, linguistically accurate, and developmentally appropriate Arabic children's narration. The model's superior performance across all evaluation dimensions, combined with its demonstrated capability for expressive Arabic speech synthesis, positions it as the optimal choice for integration into a multimodal children's storytelling system of the Gheras type.

## 5.3 System Components and Integration

This section explains how the main components of the Gheras platform communicate and exchange data. The system integrates the React frontend, Django backend, PostgreSQL database, and external AI services to support user interaction, story generation, content storage, and story presentation.

### 5.3.1 Frontend–Backend Integration

The frontend communicates with the backend through REST API requests. User actions such as signing in, creating child profiles, submitting story-generation requests, recording reading logs, and submitting post-story behavioral responses are sent to the backend as API requests. The backend processes these requests and returns structured responses that allow the frontend to update the user interface.

Authentication is handled through token-based communication. After successful sign-in, the frontend stores the token and attaches it to later API requests. This allows the backend to identify the authenticated user before returning protected data.

```
const token = getToken();
if (token) {
  headers["Authorization"] = `Token ${token}`;
}
const res = await fetch(`/api${path}`, {
  ...options,
  headers,
});
```

Listing 5.2. Frontend API request handling with authentication token attachment.

Story generation uses an asynchronous interaction flow. After the user submits a story request, the backend creates an initial story record and starts the generation process. The frontend then checks the story status periodically until the story becomes ready or fails. This allows the interface to remain responsive while the backend continues generating the story in the background.

```
const res = await api.get<StoryGenerationProgress>(
  `/stories/${storyId}/status/`
);
if (res.status === "failed") {
  setErrorModalOpen(true);
}
if (areInitialPagesReady(res)) {
  navigate(`/stories/${res.id}`, { replace: true });
}
```

Listing 5.3. Frontend polling mechanism for monitoring story-generation status.

The frontend also interacts with backend endpoints for generated content such as story pages, narration audio, and PDF files. PDF generation and download are handled through dedicated endpoints so that downloadable files can be requested separately from normal JSON-based API responses.

### 5.3.2 Backend–Database Integration

The backend interacts with the PostgreSQL database through Django’s ORM. This integration allows the backend to create, retrieve, and update records related to users, child profiles, stories, story pages, reading logs, behavioral tasks, and configuration options.

During story creation, the backend stores the initial story record, updates the story status as generation progresses, and creates story-page records when generated content becomes available. Each story page stores the narration text, illustration reference, and narration audio reference needed by the story viewer.

```
story = Story.objects.create(
    user=request.user,
    child=child,
    story_type=genre.key,
    art_style=art_style.key,
    target_behavior=behavior_key,
    custom_behavior=custom_behavior,
    status="queued",
    pdf_status="pending",
)
background_thread = threading.Thread(
    target=_run_story_pipeline,
    args=(story.id,),
    daemon=True,
)
background_thread.start()
```

Listing 5.4. Backend story creation and background generation trigger.

The database also stores configuration records such as behaviors, genres, and art styles. These records are retrieved by the backend and provided to the frontend during story creation, ensuring that users select from the available customization options stored in the system.

### 5.3.3 Backend–AI Services Integration

The backend integrates with external AI services to generate the main story components: text, illustrations, and narration audio. After validating the user’s story request, the backend prepares the required inputs, including the child profile, selected genre, art style, and target behavior. These inputs are then passed to the AI generation process.

The language model produces the structured story content and scene descriptions. The image-generation model uses the scene descriptions and visual style information to generate illustrations. The text-to-speech model converts narration text into audio files. Once these outputs are returned, the backend associates them with the corresponding story and story-page records in the database.

This integration allows the platform to transform user inputs into a complete multimedia story while keeping the frontend, backend, database, and AI services clearly separated. The frontend handles user interaction, the backend coordinates the workflow, the database stores the generated content and status updates, and the AI services produce the story text, illustrations, and narration audio.

## 5.4 Tools, Frameworks, and Libraries

- During development, the Vite dev server proxies /api requests to the Django backend on port 8001. This way the frontend and backend can run on different ports without CORS problems.
- pm2 is our process manager for running both servers at once.

- We use `npx tsc --noEmit` regularly to check TypeScript types without building.

Table 5.16. Backend tools and libraries

| Tool/Library          | Version      | Purpose                                                                                                                                                                                        |
|-----------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python                | 3.12         | Programming language                                                                                                                                                                           |
| Django                | 5.1          | Web framework                                                                                                                                                                                  |
| Django REST Framework | 3.15         | REST API toolkit                                                                                                                                                                               |
| PostgreSQL            | 16           | Relational database                                                                                                                                                                            |
| ReportLab             | 4.x          | PDF generation                                                                                                                                                                                 |
| WhiteNoise            | 6.x          | Static file serving in production                                                                                                                                                              |
| django-cors-headers   | 4.x          | CORS handling for API                                                                                                                                                                          |
| Gunicorn              | —            | WSGI server for production                                                                                                                                                                     |
| openai (Python SDK)   | $\geq 1.0$   | Official async-compatible client for OpenAI GPT-4o-mini and TTS endpoints. Version 1.0 introduced the fully async <code>AsyncOpenAI</code> class and <code>json_object</code> response format. |
| httpx                 | $\geq 0.27$  | Fully async HTTP/1.1 and HTTP/2 client for the FLUX API polling loop. Selected over <code>aiohttp</code> for its familiar requests-like API and clean context-manager connection lifecycle.    |
| asyncio               | stdlib       | Enables concurrent scene generation via <code>asyncio.gather()</code> and controlled throttling via <code>asyncio.Semaphore</code> . Core infrastructure for the entire async pipeline.        |
| python-dotenv         | $\geq 0.19$  | Loads API keys and configuration from <code>.env</code> files at module import time, decoupling secrets from source code following twelve-factor app methodology.                              |
| arabic-reshaper       | $\geq 3.0.0$ | Reshapes Arabic ligatures for correct rendering in Pillow-based PDF composition.                                                                                                               |
| python-bidi           | $\geq 0.6.0$ | Applies the Unicode Bidirectional Algorithm to Arabic text for correct right-to-left display in generated PDFs.                                                                                |
| Pillow                | $\geq 10.0$  | Composites AI-generated illustrations and shaped Arabic text into PDF story pages.                                                                                                             |
| hashlib               | stdlib       | Provides the MD5 hash function used to generate content-addressed audio cache keys from narration text.                                                                                        |
| base64 / mimetypes    | stdlib       | Encodes reference photographs as Base64 data URIs with correct MIME type headers for the FLUX API <code>input_image</code> field.                                                              |

Table 5.17. Frontend tools and libraries

| Tool/Library | Version | Purpose                     |
|--------------|---------|-----------------------------|
| React        | 19      | UI library                  |
| TypeScript   | 5.x     | Type-safe JavaScript        |
| Vite         | 7       | Build tool and dev server   |
| React Router | 7       | Client-side routing         |
| Tailwind CSS | 4       | Utility-first CSS framework |



| Tool/Library | Version | Purpose                      |
|--------------|---------|------------------------------|
| ReCharts     | 2.x     | Charts for the progress page |
| Lucide React | —       | Icon library                 |
| DiceBear     | 9.x     | Auto-generated child avatars |

## 5.5 Development Challenges

### 5.5.1 Background Thread Database Connections

One issue we hit early was that Django’s database connections can go stale in background threads. When the generation thread tried to save the story after a couple minutes of AI processing, it would sometimes get a “connection already closed” error. The fix was to call `django.db.close_old_connections()` at the start of the `_run_generation` function so Django opens a fresh connection.

### 5.5.2 RTL Layout

Since Gheras is for Arabic-speaking users, the whole UI is right-to-left. This affected a lot of things. The sidebar is on the right, navigation arrows are flipped, text alignment is right-aligned by default, and we had to use logical CSS properties (`ps`, `pe`, `ms`, `me` in Tailwind for padding-start, padding-end, etc.) instead of left/right. The story viewer page was tricky because the page navigation dots and arrow buttons needed to work in RTL order.

### 5.5.3 Multi-Step Form State

The `CreateStoryPage` has a 3-step wizard, and we needed to keep the selected values across steps without losing them when going back. We built a `useMultiStepForm` custom hook that manages the current step index and provides `next()`, `back()`, `isFirst`, and `isLast` helpers. The form state (selected genre, art style, behavior) lives in the component’s local state and persists across step changes.

### 5.5.4 Polling and Timeout Coordination

The `StoryGeneratingPage` has multiple timers running at the same time: the progress animation interval, the polling interval, the timeout timer, and the fun facts rotation timer. We had to be careful to clean up all timers when the component unmounts or when generation completes. We used `useRef` for all the timer IDs and a `clearAllTimers` callback to stop everything at once.

### 5.5.5 Static Files in Production vs Development

`WhiteNoise`’s `CompressedStaticFilesStorage` hashes filenames for cache busting, but it requires running `collectstatic` first. In development we don’t want to do that, so we conditionally set the storage backend based on the `DEBUG` flag. This took some debugging to get right.

### 5.5.6 Sequential Generation Latency

Early development versions of the system executed story generation sequentially, where text generation, image synthesis, and narration generation were processed one after another for every scene. Because each story required multiple images and audio segments, total generation time became excessively high under real API latency conditions. This issue was mitigated by redesigning the pipeline around asynchronous execution using `asyncio.gather()`, allowing image and audio generation to execute concurrently. The orchestrator was additionally redesigned into staged generation phases so that the frontend could display the first generated scenes immediately while the remaining assets continued generating in the background.

### ***5.5.7 Progressive Story Loading***

Initial implementations required the entire story, including all multimedia assets, to finish generating before the story viewer could open. This produced long waiting times and poor user experience for larger stories. To address this limitation, the orchestrator introduced progressive asset generation through `generate_initial_assets()` and `generate_remaining_assets()`. Only the first subset of scenes is generated initially, enabling the frontend to preload and display readable pages while the remaining scenes continue processing asynchronously.

### ***5.5.8 Character Consistency Across Scenes***

Because diffusion-based image models process each scene independently, early generated stories frequently exhibited inconsistent character appearance across scenes, including changes in hairstyle, clothing, facial structure, or skin tone. This issue was mitigated through several complementary prompt-engineering and conditioning strategies. The implementation introduced a reusable `CHARACTER_LOCK` prompt block, deterministic seed reuse, repeated appearance descriptions, and optional reference-image conditioning. Together, these mechanisms reduced visual identity drift and improved consistency between story illustrations.

### ***5.5.9 Arabic Diacritics Quality***

Early LLM outputs occasionally omitted Arabic diacritics or produced inconsistent tashkeel placement, negatively affecting readability and narration pronunciation quality. The implementation addressed this issue by strengthening prompt instructions to explicitly enforce Arabic diacritization and by simplifying vocabulary complexity for younger age groups. Additional normalization logic was also introduced to monitor diacritic density and preserve properly vocalized narration text before speech synthesis.

## 6 TESTING AND EVALUATION

### 6.1 Testing Strategy and Scenarios

We tested Gheras using a combination of unit testing, integration testing, and end-to-end testing. For the frontend we wrote unit tests for context providers, custom hooks, and key components. The backend has unit tests for each API endpoint and serializer. And then we did full end-to-end tests going through the app as a real parent would use it.

Our approach was to go through every feature as a real parent would use it, checking that the frontend and backend work together correctly. We also tested individual pieces in isolation to make sure each part works on its own before putting them together.

#### 6.1.1 API Testing with curl

We also tested the API endpoints directly using curl to make sure they return the right data and status codes. For example:

- curl -X POST /api/auth/signup/ -d '{"email":"test@test.com","password":"pass123","name":"Test"}' (should return 201 with a token)
- curl -H "Authorization: Token xxx" /api/children/ (should return the user's children)
- curl -X POST /api/stories/ -H "Authorization: Token xxx" -d '{"childId":1,"storyType":"adventure",...}' (should return 202 with the story ID)

#### 6.1.2 Frontend Unit Tests

We wrote unit tests for the main frontend components, context providers, and utility functions. The tests cover authentication flows, data fetching, the multi-step form hook, the API client, and UI components like the onboarding tour and sidebar.

Table 6.1. Frontend unit test scenarios and results

| No. | Component/Page   | Test Case                              | Expected Result                                            | Status |
|-----|------------------|----------------------------------------|------------------------------------------------------------|--------|
| 1   | AuthContext      | Sign in sets user and token in storage | User object populated, token in localStorage               | Pass   |
| 2   | AuthContext      | Sign in with a non-Gmail email address | The system displays an error message requiring Gmail only. | Pass   |
| 3   | AuthContext      | Sign in with a weak password           | The system displays password complexity requirements.      | Pass   |
| 4   | AuthContext      | Sign out clears user and token         | User null, token removed                                   | Pass   |
| 5   | AuthContext      | Auto-validate token on mount           | GET /auth/me/ called, user restored                        | Pass   |
| 6   | ChildrenContext  | Fetch children on mount                | Children array populated from API                          | Pass   |
| 7   | ChildrenContext  | addChild triggers refetch              | New child appears in list                                  | Pass   |
| 8   | ChildrenContext  | deleteChild removes from list          | Child removed after API call                               | Pass   |
| 9   | useMultiStepForm | next() increments step                 | currentStep goes from 0 to 1                               | Pass   |
| 10  | useMultiStepForm | back() decrements step                 | currentStep goes from 1 to 0                               | Pass   |
| 11  | useMultiStepForm | isFirst true at step 0                 | back button disabled                                       | Pass   |
| 12  | useMultiStepForm | isLast true at final step              | Shows finish button instead of next                        | Pass   |

| No. | Component/Page  | Test Case                                               | Expected Result                                                        | Status |
|-----|-----------------|---------------------------------------------------------|------------------------------------------------------------------------|--------|
| 13  | API client      | Adds auth header to requests                            | Authorization: Token xxx in headers                                    | Pass   |
| 14  | API client      | Handles 401 response                                    | Throws error, no crash                                                 | Pass   |
| 15  | API client      | postForm sends multipart                                | Content-Type not set manually                                          | Pass   |
| 16  | CreateStoryPage | leave “Other” behavior field empty                      | Alert asks the user to enter the behavior.                             | Pass   |
| 17  | CreateStoryPage | Enter English text or symbols in “Other” behavior field | The system displays: يرجى إدخال سلوك مفهوم باللغة العربية فقط. message | Pass   |
| 18  | OnboardingTour  | Shows on first visit                                    | Tour tooltip visible, localStorage empty                               | Pass   |
| 19  | OnboardingTour  | Doesn’t show after completion                           | localStorage flag set, no tooltip                                      | Pass   |
| 20  | OnboardingTour  | Skip button dismisses tour                              | Tour hidden, flag persisted                                            | Pass   |
| 21  | Sidebar         | Collapses on mobile viewport                            | Sidebar hidden, hamburger visible                                      | Pass   |
| 22  | Sidebar         | Opens on hamburger click                                | Sidebar slides in with backdrop                                        | Pass   |
| 23  | Sidebar         | Closes on backdrop click                                | Sidebar hidden again                                                   | Pass   |

### 6.1.3 Backend Unit Tests

We tested each API endpoint with different inputs to make sure it returns the right status codes and data. We also tested edge cases like duplicate emails, wrong passwords, and trying to access other users’ data.

Table 6.2. Backend unit test scenarios and results

| No. | Module       | Test Case                 | Expected Result                | Status |
|-----|--------------|---------------------------|--------------------------------|--------|
| 1   | auth/signup  | Valid registration        | 201, token returned            | Pass   |
| 2   | auth/signup  | Duplicate email           | 400, error message             | Pass   |
| 3   | auth/signup  | Non-Gmail email           | 400, Gmail-only error message  | Pass   |
| 4   | auth/signup  | Weak password             | 400, password validation error | Pass   |
| 5   | auth/signin  | Valid credentials         | 200, token returned            | Pass   |
| 6   | auth/signin  | Wrong password            | 401, “Invalid credentials”     | Pass   |
| 7   | auth/signout | Valid token               | 200, token deleted             | Pass   |
| 8   | auth/me      | Valid token               | 200, user data                 | Pass   |
| 9   | auth/me      | No token                  | 401, unauthorized              | Pass   |
| 10  | ChildViewSet | Create child with photo   | 201, child with photo URL      | Pass   |
| 11  | ChildViewSet | Create child text-only    | 201, child with avatar URL     | Pass   |
| 12  | ChildViewSet | List only own children    | 200, filtered by user          | Pass   |
| 13  | ChildViewSet | Delete other user’s child | 404, not found                 | Pass   |
| 14  | StoryViewSet | Create story              | 202, status “processing”       | Pass   |
| 15  | StoryViewSet | List shows only completed | 200, no processing stories     | Pass   |
| 16  | StoryViewSet | Detail returns any status | 200, includes processing       | Pass   |
| 17  | StoryViewSet | Status endpoint           | 200, {id, status} only         | Pass   |

| No. | Module         | Test Case                                       | Expected Result                     | Status |
|-----|----------------|-------------------------------------------------|-------------------------------------|--------|
| 18  | StoryViewSet   | Download completed PDF                          | 200, application/pdf                | Pass   |
| 19  | StoryViewSet   | Download processing story PDF                   | 409, "PDF is still being prepared"  | Pass   |
| 20  | stories/create | Empty custom behavior with targetBehavior=other | 400, required custom behavior error | Pass   |
| 21  | stories/create | English or symbols in custom behavior           | 400, Arabic-only validation error   | Pass   |
| 22  | stories/create | Valid behavior/custom behavior                  | 201, story request created          | Pass   |
| 23  | config         | No auth required                                | 200, behaviors+genres+styles        | Pass   |
| 24  | stats          | Streak calculation                              | Correct consecutive days            | Pass   |
| 25  | stats          | Behavior radar data                             | Correct counts per behavior         | Pass   |
| 26  | reading-logs   | Create log                                      | 201, log saved                      | Pass   |
| 27  | tasks          | Create behavior task                            | 201, task saved                     | Pass   |
| 28  | serializers    | camelCase output                                | childId not child_id                | Pass   |

### 6.1.4 End-to-End Test Scenarios

For E2E testing we went through the whole app as a parent would, from signing up to creating stories and checking progress. We tested on both desktop and mobile viewports. Here are the scenarios we covered:

Table 6.3. End-to-end test scenarios and results

| No. | User Flow                    | Steps                                                                            | Expected Outcome                                                        | Status |
|-----|------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------|--------|
| 1   | New user registration        | User attempts to create an account using a non-Gmail email                       | The registration is blocked and an error message is displayed.          | Pass   |
| 2   | New user registration        | User attempts to create an account using a weak password                         | The registration is blocked and a password validation message is shown. | Pass   |
| 3   | Successful user registration | Sign up, land on dashboard, see empty state                                      | Dashboard shows 0 stories, 0 children                                   | Pass   |
| 4   | Add first child              | Fill form with name/age/gender, submit                                           | Child card appears in children list                                     | Pass   |
| 5   | Edit child                   | Change age, save                                                                 | Updated age shown on card                                               | Pass   |
| 6   | Create story end-to-end      | Select child, pick genre/style/behavior, submit, wait for generation, view story | Story viewer shows pages with text and illustrations                    | Pass   |
| 7   | Create story custom behavior | User selects "Other" behavior and enters English text                            | The system prevents story generation and displays a validation message. | Pass   |
| 8   | Create story custom behavior | User selects "Other" behavior and enters symbols or meaningless input            | The system prevents story generation and displays a validation message. | Pass   |

| No. | User Flow                               | Steps                                                    | Expected Outcome                                                              | Status |
|-----|-----------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------|--------|
| 9   | Create story custom behavior            | User selects “Other” behavior but leaves the field empty | The system prevents story generation and asks the user to enter the behavior. | Pass   |
| 10  | Story generation polling                | Submit story, observe generating page                    | Progress bar fills, fun facts rotate, auto-redirect on completion             | Pass   |
| 11  | Generation timeout                      | Wait beyond 2 minutes                                    | Timeout modal appears with retry option                                       | Pass   |
| 12  | Read story and answer behavior question | Navigate all pages, answer behavioral prompt             | Reading log saved, behavior task recorded                                     | Pass   |
| 13  | PDF download                            | Click download on completed story                        | Browser downloads PDF file with correct content                               | Pass   |
| 14  | Library search                          | Type partial title in search bar                         | Stories filtered by title match                                               | Pass   |
| 15  | Library filter by child                 | Select child from drop-down                              | Only that child’s stories shown                                               | Pass   |
| 16  | Library filter by behavior              | Select behavior from drop-down                           | Stories with that behavior shown                                              | Pass   |
| 17  | Library combined filters                | Search + behavior filter                                 | Intersection of both filters                                                  | Pass   |
| 18  | Library empty state                     | Search gibberish                                         | “No results” message shown                                                    | Pass   |
| 19  | Dashboard stats accuracy                | Create stories, read them                                | Stat cards show correct counts                                                | Pass   |
| 20  | Progress charts                         | Read stories with different behaviors                    | Behavior chart and weekly chart update                                        | Pass   |
| 21  | Mobile sidebar toggle                   | Resize to 375px, toggle hamburger                        | Sidebar opens/closes, backdrop works                                          | Pass   |
| 22  | Mobile story viewer                     | View story on mobile viewport                            | Stacked layout, touch-friendly navigation                                     | Pass   |
| 23  | Onboarding first login                  | Clear localStorage, sign in                              | Tour tooltips appear, guide through app                                       | Pass   |
| 24  | Onboarding skip                         | Click skip during tour                                   | Tour dismissed, doesn’t reappear                                              | Pass   |
| 25  | Sign out and back in                    | Sign out, verify redirect, sign in again                 | Token cleared, re-authenticated                                               | Pass   |
| 26  | Multiple children                       | Add 3 children, create story                             | Library shows all, filter works per child                                     | Pass   |

### 6.1.5 TypeScript Type Checking

We ran `npx tsc --noEmit` regularly to catch type errors. Since the whole frontend is in TypeScript with strict mode, this catches things like passing the wrong prop types, missing required fields in API payloads, and incorrect state types.

We also ran `npx vite build` to make sure the production bundle compiles without errors.

### 6.1.6 AI Pipeline Testing

For AI pipeline testing, we evaluated the internal multimodal generation workflow implemented by LLMService, FluxService, TTSService, and StoryOrchestrator. The testing focused on Arabic story generation, image–text alignment, character consistency, age-adaptive generation, progressive loading, PDF generation, and API failure handling. Table 6.4 summarizes the evaluated AI pipeline scenarios.

#### Arabic Story Text Generation with Diacritics

**Objective:** Verify that the LLM service produces a structurally complete Arabic story package with coherent narration text and readable partial diacritization.

**Procedure:** Submit story-generation requests using different child profiles, story types, and target behaviors. Inspect the returned JSON structure for completeness, and the presence of Arabic tashkeel characters within the generated narration text.

#### Character Visual Consistency Across Scenes

**Objective:** Evaluate whether the generated child protagonist maintains relatively stable visual identity across multiple generated scenes.

**Procedure:** Generate multi-scene stories using the same child profile, deterministic seed, appearance description, and visual style. Inspect generated illustrations for consistency in hairstyle, facial structure, clothing appearance, skin tone, and overall visual identity across scenes.

#### Progressive Loading and Background Generation

**Objective:** Verify asynchronous staged asset-generation behavior implemented by StoryOrchestrator.

**Procedure:** Submit story-generation requests and monitor the frontend progress page while the orchestrator executes `generate_initial_assets()` and `generate_remaining_assets()` asynchronously. Verify that the first readable scenes become accessible before remaining multimedia assets complete generation.

#### PDF Generation

**Objective:** Verify correct assembly of generated multimodal story content during PDF rendering.

**Procedure:** Trigger PDF export for completed stories and inspect the generated document for correct Arabic text rendering, illustration embedding, page ordering, and successful integration of narration content and generated images.

#### Image Generation and Story-Text Alignment

**Objective:** Verify semantic consistency between generated text and generated illustrations.

**Procedure:** Generate complete stories and compare the generated illustrations with their corresponding text. Inspect whether scene events, characters, environments, and emotional context described in the text are visually reflected in the generated illustrations.

#### Age-Adaptive Story Generation

**Objective:** Verify that scene count and narration complexity adapt correctly according to the selected age group.

**Procedure:** Generate stories for the 4–5, 6–7, and 8–9 age groups and compare the generated scene count, narration length, vocabulary complexity, and narrative structure. Verify that the orchestrator produces 3, 5, and 7 scenes respectively according to the configured age-dependent generation logic.

#### Art-Style Adherence

**Objective:** Verify that generated illustrations follow the selected visual style consistently across scenes.

**Procedure:** Generate stories using all supported visual styles, including watercolor, cartoon, anime, pixel art, storybook, and paper-cut styles. Inspect generated illustrations visually to determine whether style-specific characteristics are preserved throughout the story.

#### Reference-Image Conditioning

**Objective:** Verify that uploaded child photographs improve character resemblance in generated illustrations.

**Procedure:** Upload child reference images during story generation and compare the generated protagonist appearance with the uploaded child image. Inspect facial appearance, hairstyle, skin tone, and overall resemblance across generated scenes while preserving the selected art style.

Table 6.4 summarizes the principal AI pipeline testing scenarios, including Arabic story generation with diacritics, image-generation alignment, character consistency across scenes, age-adaptive generation, visual-style adherence, reference-image conditioning, progressive-loading behavior, and PDF generation.

Table 6.4. AI Pipeline Test Scenarios

| No. | AI Scenario                                   | Expected Outcome                                                                                                                                  | Status |
|-----|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 1   | Arabic story generation with diacritics       | Structurally complete Arabic story package containing coherent narration text with readable partial diacritization and valid scene structure.     | Pass   |
| 2   | Image generation and story-text alignment     | Generated illustrations visually reflect the scene events, characters, environments, and emotional context described in the narration text.       | Pass   |
| 3   | Character consistency across scenes           | Relatively stable child appearance maintained across generated scenes, including hairstyle, facial structure, clothing appearance, and skin tone. | Pass   |
| 4   | Age-adaptive generation                       | Scene count, narration length, and vocabulary complexity adapt correctly according to the selected age group.                                     | Pass   |
| 5   | Art-style adherence                           | Generated illustrations consistently preserve the selected visual style across scenes and cover generation.                                       | Pass   |
| 6   | Reference-image conditioning                  | Generated character appearance partially resembles the uploaded child reference image while preserving the selected art style.                    | Pass   |
| 7   | Progressive loading and background generation | Initial story scenes become viewable before remaining multimedia assets complete asynchronous background generation.                              | Pass   |
| 8   | PDF generation                                | Generated PDF correctly integrates Arabic narration text, generated illustrations, page ordering, and Arabic text rendering.                      | Pass   |
| 9   | API failure and timeout handling              | Story-generation pipeline continues operating gracefully under failed, delayed, or incomplete external AI-service responses.                      | Pass   |



## 6.2 Test Cases and Input Configurations

### 6.2.1 Story Generation Pipeline

The Gheras generation pipeline produces personalized Arabic stories conditioned on the child profile and the selected target behavior. The generated story incorporates the child’s name, age group, appearance description, selected story type, and visual style into both the narration and illustration-generation stages. Depending on the selected age group, each generated story contains between three and seven illustrated scenes accompanied by Arabic narration audio.

The pipeline was evaluated using multiple input configurations submitted through the complete story-generation workflow. Each generation request triggered multimodal generation through the Story-Orchestrator, including story-text generation, image synthesis, narration generation, progressive asset loading, and final story rendering within the frontend story viewer.

### 6.2.2 Reference Data

To support controlled testing across multiple story-generation configurations, the database was pre-loaded using Django fixtures. The reference data included multiple target behaviors, story genres, and visual art styles supported by the platform.

The loaded behaviors included courage, kindness, sharing, honesty, responsibility, patience, respect, and gratitude. The available story genres included adventure, fantasy, educational, social, and nature themes. Supported visual styles included watercolor, cartoon, pixel art, anime, storybook, and paper-cut styles. Each configuration entry included an Arabic label, description, and frontend icon identifier. Additional test user accounts were also created to validate account-level data isolation and ensure that generated stories and child profiles remained separated between different users.

Table 6.5 presents the principal test cases used to evaluate the story-generation pipeline across different child profiles, age groups, behaviors, visual styles, and reference-image conditions.

Table 6.5. AI Services Test Cases

| Test Case | Child Name | Gender | Age Group | Story Type  | Target Behavior | Art Style  | Reference Image |
|-----------|------------|--------|-----------|-------------|-----------------|------------|-----------------|
| TC-01     | Noor       | Girl   | 4–5       | Adventure   | Kindness        | Storybook  | No              |
| TC-02     | Faisal     | Boy    | 6–7       | Educational | Honesty         | Cartoon    | Yes             |
| TC-03     | Sara       | Girl   | 8–9       | Fantasy     | Responsibility  | Watercolor | No              |
| TC-04     | Omar       | Boy    | 6–7       | Nature      | Courage         | Anime      | Yes             |
| TC-05     | Lina       | Girl   | 4–5       | Social      | Sharing         | Paper-cut  | No              |
| TC-06     | Khalid     | Boy    | 8–9       | Adventure   | Patience        | Pixel art  | Yes             |

These test cases were selected to ensure coverage of the main system variables. Age groups were varied to validate age-adaptive scene generation, while art styles were varied to assess style adherence. Reference-image and non-reference-image cases were both included to compare character consistency under text-only and image-conditioned generation settings. Target behaviors were also varied to verify that the generated narratives reflected the selected educational value.

## 6.3 Evaluation Metrics

The model-selection phase, presented in the “Model Evaluation and Selection” section, chapter 5, evaluated the text-generation, image-generation, and TTS models independently through their respective APIs using multiple human and automated evaluation metrics specific to each modality. Therefore,

the current section focuses on the evaluation metrics used to assess the integrated Gheras system as a complete multimodal platform after implementation and orchestration.

The evaluation metrics at this stage focused on overall system performance, including frontend responsiveness, backend API performance, asynchronous generation behavior, multimodal pipeline responsiveness, PDF generation, and end-to-end user experience during story generation and consumption.

Table 6.6. System-Level Evaluation Metrics

| Metric                            | Description                                                                                       | Evaluation Method                                            |
|-----------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| API Response Time                 | Measures responsiveness of backend REST API endpoints during normal operation.                    | Runtime latency measurement of API requests.                 |
| Story Generation Time             | Measures total time required to generate a complete multimodal story.                             | End-to-end pipeline timing.                                  |
| Progressive-Loading Readiness     | Measures time required before the first readable story scenes become available to the user.       | Frontend progress monitoring during asynchronous generation. |
| Image Generation Latency          | Measures time required to generate scene illustrations through the image-generation pipeline.     | Runtime latency measurement during generation.               |
| TTS Generation Latency            | Measures time required to synthesize narration audio for story scenes.                            | Runtime latency measurement during audio generation.         |
| PDF Generation Time               | Measures time required to generate downloadable illustrated PDF stories.                          | PDF export execution timing.                                 |
| Polling Overhead                  | Evaluates frontend polling behavior and backend synchronization during asynchronous generation.   | Monitoring repeated /status/ requests during generation.     |
| Mobile and Desktop Responsiveness | Evaluates frontend usability and layout behavior across different viewport sizes.                 | Manual cross-device interface testing.                       |
| Failure Handling and Recovery     | Evaluates system stability under failed or delayed external API responses.                        | Controlled API-failure and timeout testing.                  |
| End-to-End User Workflow          | Evaluates correctness of the complete workflow from story creation to story viewing and download. | End-to-end functional testing.                               |

System-level metrics used to evaluate the overall performance, responsiveness, and usability of the integrated Gheras platform.

## 6.4 Performance Benchmarks

We measured a few things to evaluate the system’s performance:

- API response time (how fast do the endpoints respond)
- Story generation time (how long the background generation takes)
- PDF generation time
- Frontend bundle size after production build
- Polling overhead during story generation

### 6.4.1 API Response Times

Most API endpoints respond in under 50ms. The stats endpoint is a bit slower because it does several database queries (counting stories, calculating streaks, building behavior and weekly data). But it’s still under 200ms even with a reasonable amount of data.

The config endpoint is very fast since it just reads three small tables. The children and stories list endpoints are also fast because they’re filtered by user and we don’t have huge amounts of data per user.

Table 6.7. Average API response times

| Endpoint                        | Avg Response Time                |
|---------------------------------|----------------------------------|
| GET /api/auth/me/               | 15ms                             |
| GET /api/config/                | 20ms                             |
| GET /api/children/              | 25ms                             |
| GET /api/stories/               | 30ms                             |
| GET /api/stories/{id}/status/   | 15ms                             |
| GET /api/stats/                 | 150ms                            |
| POST /api/stories/              | 80ms (returns before generation) |
| GET /api/stories/{id}/download/ | 300ms (builds PDF on the fly)    |

### 6.4.2 Story Generation Time

Story generation takes approximately 1–2 minutes end-to-end for the 4–5 age group, approximately 2 minutes for the 6–7 age group, and up to 2–3 minutes for the 8–9 age group due to the increased number of generated scenes and multimedia assets. The approximate runtime breakdown is as follows:

- **LLM story generation:** 10–20 seconds depending on the selected age group and token budget
- **Cover image generation:** 30–60 seconds
- **Scene illustration generation:** approximately 30–60 seconds per scene through the asynchronous FLUX polling workflow
- **TTS narration generation:** approximately 3–10 seconds per scene depending on narration length

The frontend progress page is calibrated around this staged-generation behavior. During generation, the progress bar gradually advances while the frontend asynchronously polls the backend status endpoint. The progress indicator reaches approximately 90% while the initial scenes and multimedia assets are being generated, then transitions to 100% once the orchestrator confirms completion of the required generation stages.

### 6.4.3 Polling Efficiency

The polling endpoint (/stories/{id}/status/) is kept small on purpose. It only returns two fields (id and status), no page data or related objects. With a 5-second interval and a 2-minute generation time, the frontend makes about 24 poll requests per story creation. Each request is under 200 bytes, so the total polling overhead is under 5 KB, which is negligible.

### 6.4.4 Frontend Build

The production build (via `npm run vite build`) produces a JavaScript bundle of about 350 KB gzipped. The main chunks are React, React Router, ReCharts, and our application code. Tailwind CSS with tree-shaking produces a CSS file of about 25 KB gzipped.

## 6.5 Reliability and Usability Evaluation

### 6.5.1 Mobile Responsiveness

We tested the UI on different screen sizes using browser dev tools. The layout is responsive:

- On mobile (< 640px), the sidebar collapses into a hamburger menu, stat cards stack in a 2-column grid, the story viewer switches from side-by-side to stacked layout (illustration on top, text below), and the library grid shows one card per row.
- On tablet (640px–1024px), the sidebar is visible but narrower, stat cards show in a 2-column grid, and the library shows 2 cards per row.
- On desktop (> 1024px), full layout with sidebar expanded, stat cards in 4 columns, library in 3–4 columns.

All the interactive elements (buttons, cards, form inputs) have appropriate touch targets for mobile use.

### 6.5.2 Error Handling

We handle errors at several levels:

- If any API request returns a non-200 status, the client throws an error with the server’s message. The frontend catches it and shows it to the user.
- The backend validates all input through DRF serializers. Missing fields, invalid keys, or wrong types return 400 with a descriptive error.
- If the background thread crashes during story generation, it sets the status to “failed”. The polling picks that up and shows an error modal.
- There’s a 5-minute timeout on the frontend side. If the story isn’t done by then, a timeout modal appears and polling stops.
- If a child or story doesn’t exist (or belongs to another user), the API returns 404.
- Most API calls have `.catch(() => {})` to prevent unhandled promise rejections. We could improve this with toast notifications later.

### 6.5.3 Security

- All endpoints except sign-up, sign-in, and config require authentication
- Users can only access their own data. Every queryset is filtered by `user=request.user`
- Passwords are hashed by Django’s built-in auth system
- In production, security headers are enabled (XSS filter, content type nosniff, secure cookies)
- CORS is restricted to the frontend origin
- CSRF middleware is included in the middleware stack

## 7 RESULTS AND ANALYSIS

### 7.1 Quantitative Performance Results

The quantitative evaluation assessed the runtime performance and responsiveness of the integrated Gheras platform, including the backend API layer, frontend interaction workflow, and multimodal AI-generation pipeline. Measurements were collected during real execution sessions using the implemented system under normal testing conditions. The recorded results therefore reflect practical runtime behavior, including external API latency, asynchronous polling delays, concurrent multimedia generation, frontend synchronization, and progressive-loading execution.

The recorded results therefore reflect practical runtime behavior, including external API latency, asynchronous polling delays, concurrent multimedia generation, frontend synchronization, and progressive-loading execution.

The evaluation demonstrated that the multimodal AI pipeline constituted the dominant source of runtime latency, particularly during image generation. This behavior is expected because the FLUX image-generation workflow depends on asynchronous submit-and-poll execution, where each illustration requires multiple polling cycles before completion. In contrast, the backend API endpoints remained highly responsive during normal operation, while frontend polling overhead remained lightweight due to the minimal payload returned by the status endpoint.

The results also confirmed that the staged asynchronous generation architecture implemented by StoryOrchestrator significantly improved perceived responsiveness. Instead of waiting for the complete story-generation process to finish, the frontend displayed the first readable scenes immediately after the initial multimedia assets completed generation, while the remaining scenes continued processing asynchronously in the background.

Table 7.1. AI Pipeline Runtime Benchmarks

| Operation                         | Age Group | Scenes         | Approx. Latency | Notes                                              |
|-----------------------------------|-----------|----------------|-----------------|----------------------------------------------------|
| LLM story text generation         | 4–5       | 3              | 10–20 s         | GPT-4o-mini, 1500 max tokens                       |
| LLM story text generation         | 6–7       | 5              | 10–20 s         | GPT-4o-mini, 2800 max tokens                       |
| LLM story text generation         | 8–9       | 7              | 15–30 s         | GPT-4o-mini, 4500 max tokens                       |
| Flux image generation (per scene) | All       | 1              | 30–60 s         | Async polling workflow                             |
| Initial scene image generation    | All       | First 3 scenes | 40–70 s         | Concurrent <code>asyncio.gather()</code> execution |
| Remaining scenes + cover          | 6–7       | 2+1            | 30–70 s         | Semaphore(3)-controlled generation                 |
| Remaining scenes + cover          | 8–9       | 4+1            | 40–90 s         | Semaphore(3)-controlled generation                 |
| TTS generation per scene          | All       | 1              | 3–10 s          | gpt-4o-mini-tts                                    |
| TTS cache retrieval               | All       | 1              | < 0.1 s         | Local MP3 retrieval                                |
| End-to-end pipeline (4–5)         | 4–5       | 3              | ~117 s          | Full multimodal pipeline                           |
| End-to-end pipeline (6–7)         | 6–7       | 5              | ~131 s          | Full multimodal pipeline                           |
| End-to-end pipeline (8–9)         | 8–9       | 7              | ~150–180 s      | Full multimodal pipeline                           |

Measured runtime observations for the multimodal AI-generation pipeline across the supported age groups.

The runtime measurements showed that image generation accounted for the largest proportion of total execution time. Individual scene illustrations typically required between 30 and 60 seconds depending on external API load conditions and polling duration. Stories generated for older age groups exhibited longer total execution times because the orchestrator increased both the number of scenes and the token budget allocated to the language model.

The quantitative evaluation also confirmed the effectiveness of concurrent multimedia generation. Scene illustrations and narration audio were generated simultaneously through `asyncio.gather()`, reducing overall latency compared with fully sequential execution. The remaining scenes were processed using controlled asynchronous concurrency through `asyncio.Semaphore(3)`, preventing excessive simultaneous external API requests while preserving pipeline stability.

The frontend progress system remained synchronized with backend generation throughout the pipeline lifecycle. During generation, the frontend periodically queried the lightweight `/status/` endpoint while the backend asynchronously updated generation progress. This enabled progressive loading behavior with minimal polling overhead.

Table 7.2. Backend and Frontend Runtime Measurements

| Metric                          | Measured Result |
|---------------------------------|-----------------|
| Total API endpoints             | 16              |
| Database models                 | 7               |
| Frontend pages                  | 10              |
| Reusable UI components          | 8+              |
| Average API response time       | < 200 ms        |
| PDF generation time             | ~300 ms         |
| Frontend production bundle size | ~375 KB gzipped |
| Average story-generation time   | ~120 s          |

Measured responsiveness and execution performance of the integrated frontend and backend system. Backend API testing demonstrated consistently low response times for standard read-oriented endpoints. Most authenticated API requests completed in under 50 ms, while the `/stats/` endpoint exhibited slightly higher latency due to multiple aggregation queries required for reading analytics and behavioral statistics. The `/stories/` creation endpoint returned rapidly because story generation executed asynchronously in the background rather than blocking the request lifecycle.

Frontend performance evaluation showed that the production React/Vite build remained relatively lightweight after compression. Responsive interface testing across desktop and mobile layouts also confirmed stable rendering behavior during asynchronous story generation, progress polling, PDF export, and story-viewer navigation.

```

=====
GHERAS STORY GENERATION PERFORMANCE REPORT
=====
Story ID      : 62
Status       : completed
Age group    : 4-5
Image style  : anime
Total scenes : 3
Initial pages target : 3
Started at   : 2026-05-06 11:23:35.361
Ended at    : 2026-05-06 11:25:32.603
-----
1. LLM response time : 16.707s
2. Image generation time wall-clock : 99.652s (3 scene images + 1 cover image)
   Image accumulated service time : 184.043s
3. TTS generation time wall-clock : 10.569s (3 scene audios)
   TTS accumulated service time : 28.902s
4. First viewable ready time : 71.643s
5. End-to-end full pipeline time : 117.244s
-----
Detailed image events:
- image:scene 1 image 37.747s [2026-05-06 11:23:52.892 -> 2026-05-06 11:24:30.639]
- image:scene 3 image 48.326s [2026-05-06 11:23:54.629 -> 2026-05-06 11:24:42.955]
- image:scene 2 image 53.192s [2026-05-06 11:23:53.789 -> 2026-05-06 11:24:46.982]
- image:cover image 44.778s [2026-05-06 11:24:47.767 -> 2026-05-06 11:25:32.544]
Detailed TTS events:
- tts:scene 2 9.589s [2026-05-06 11:23:54.628 -> 2026-05-06 11:24:04.216]
- tts:scene 1 10.474s [2026-05-06 11:23:53.785 -> 2026-05-06 11:24:04.258]
- tts:scene 3 8.839s [2026-05-06 11:23:55.515 -> 2026-05-06 11:24:04.354]
=====

```

Figure 7.1. Runtime benchmark output for the 4–5 age-group multimodal story-generation pipeline, showing LLM response time, image-generation latency, TTS latency, first-viewable readiness time, and total end-to-end execution time.

```

=====
GHERAS STORY GENERATION PERFORMANCE REPORT
=====
Story ID      : 60
Status       : completed
Age group    : 6-7
Image style  : storybook
Total scenes : 5
Initial pages target : 3
Started at   : 2026-05-05 12:48:52.131
Ended at    : 2026-05-05 12:51:03.431
-----
1. LLM response time : 13.051s
2. Image generation time wall-clock : 117.646s (5 scene images + 1 cover image)
   Image accumulated service time : 290.857s
3. TTS generation time wall-clock : 63.221s (5 scene audios)
   TTS accumulated service time : 22.975s
4. First viewable ready time : 70.465s
5. End-to-end full pipeline time : 131.298s
-----
Detailed image events:
- image:scene 1 image 37.834s [2026-05-05 12:49:05.687 -> 2026-05-05 12:49:43.523]
- image:scene 3 image 40.394s [2026-05-05 12:49:06.694 -> 2026-05-05 12:49:47.086]
- image:scene 2 image 56.275s [2026-05-05 12:49:06.282 -> 2026-05-05 12:50:02.561]
- image:cover image 44.937s [2026-05-05 12:50:03.142 -> 2026-05-05 12:50:48.080]
- image:scene 5 image 51.894s [2026-05-05 12:50:04.245 -> 2026-05-05 12:50:56.140]
- image:scene 4 image 59.522s [2026-05-05 12:50:03.808 -> 2026-05-05 12:51:03.333]
Detailed TTS events:
- tts:scene 3 3.870s [2026-05-05 12:49:07.136 -> 2026-05-05 12:49:11.003]
- tts:scene 2 5.031s [2026-05-05 12:49:06.693 -> 2026-05-05 12:49:11.723]
- tts:scene 1 5.810s [2026-05-05 12:49:06.281 -> 2026-05-05 12:49:12.090]
- tts:scene 5 3.001s [2026-05-05 12:50:04.581 -> 2026-05-05 12:50:07.585]
- tts:scene 4 5.264s [2026-05-05 12:50:04.238 -> 2026-05-05 12:50:09.502]
=====

```

Figure 7.2. Runtime benchmark output for the 6–7 age-group multimodal story-generation pipeline, illustrating the effect of increased scene count on image-generation workload and total pipeline latency.

```

=====
GHERAS STORY GENERATION PERFORMANCE REPORT
=====
Story ID           : 59
Status            : completed
Age group         : 8-9
Image style       : watercolor
Total scenes      : 7
Initial pages target : 3
Started at        : 2026-05-05 11:14:35.528
Ended at          : 2026-05-05 11:17:42.854
-----
1. LLM response time           : 25.192s
2. Image generation time wall-clock : 160.923s (7 scene images + 1 cover image)
   Image accumulated service time : 380.507s
3. TTS generation time wall-clock : 97.830s (7 scene audios)
   TTS accumulated service time  : 28.728s
4. First viewable ready time    : 76.996s
5. End-to-end full pipeline time : 187.327s
-----
Detailed image events:
- image:scene 1 image 42.957s [2026-05-05 11:15:01.679 -> 2026-05-05 11:15:44.640]
- image:scene 2 image 42.450s [2026-05-05 11:15:02.707 -> 2026-05-05 11:15:45.158]
- image:scene 3 image 48.910s [2026-05-05 11:15:03.574 -> 2026-05-05 11:15:52.482]
- image:scene 6 image 41.107s [2026-05-05 11:15:56.036 -> 2026-05-05 11:16:37.142]
- image:cover image 44.710s [2026-05-05 11:15:53.587 -> 2026-05-05 11:16:38.298]
- image:scene 5 image 45.402s [2026-05-05 11:15:55.237 -> 2026-05-05 11:16:40.635]
- image:scene 4 image 49.514s [2026-05-05 11:15:54.455 -> 2026-05-05 11:16:43.969]
- image:scene 7 image 65.456s [2026-05-05 11:16:37.147 -> 2026-05-05 11:17:42.602]
Detailed TTS events:
- tts:scene 3 3.515s [2026-05-05 11:15:04.419 -> 2026-05-05 11:15:07.927]
- tts:scene 1 6.181s [2026-05-05 11:15:02.701 -> 2026-05-05 11:15:08.881]
- tts:scene 2 5.579s [2026-05-05 11:15:03.574 -> 2026-05-05 11:15:09.149]
- tts:scene 6 2.714s [2026-05-05 11:15:56.757 -> 2026-05-05 11:15:59.473]
- tts:scene 4 4.532s [2026-05-05 11:15:55.228 -> 2026-05-05 11:15:59.766]
- tts:scene 5 3.778s [2026-05-05 11:15:56.034 -> 2026-05-05 11:15:59.810]
- tts:scene 7 2.429s [2026-05-05 11:16:38.102 -> 2026-05-05 11:16:40.531]
=====

```

Figure 7.3. Runtime benchmark output for the 8–9 age-group multimodal story-generation pipeline, demonstrating the highest overall latency due to expanded narrative complexity and additional generated multimedia assets.

## 7.2 Qualitative Observations

### 7.2.1 User Experience

The multi-step story creation wizard feels intuitive. Parents pick options from visual grids with icons and descriptions, which is much better than typing values into text fields. The step indicator at the top shows where you are in the process.

The story generating page turned out well. Instead of just showing a spinner, we have a progress bar, step indicators, and rotating fun facts about children’s reading. It keeps the parent engaged during the 2-minute wait. The auto-redirect to the story viewer when generation completes feels smooth.

The story viewer with its split layout (illustration + text) looks like a real storybook.

The generated PDF output also provided a readable offline story format integrating Arabic text and generated illustrations into a single document suitable for downloading and sharing.





Figure 7.4. Mobile-responsive story-viewer interface during multimodal story reading.



Figure 7.5. Progressive-loading interface displayed during asynchronous multimodal story generation.

### 7.2.2 Arabic Content

All the UI text is in Arabic, and the generated stories are in Arabic too. The RTL layout works correctly throughout the app.

### 7.2.3 Onboarding

We added an onboarding tour that appears on first login. It shows tooltip-style popups pointing to different parts of the interface, explaining what each section does. This helps parents who might not be very tech-savvy.

### 7.2.4 Arabic Story Quality

The LLM consistently produced well-structured Arabic narratives with age-appropriate vocabulary and coherent plot arcs across all six test cases. Stories for younger age groups (4–5) were notably simpler in sentence structure and vocabulary, while 8–9 age group stories contained richer descriptive language and more complex event sequences. The target behavior was organically embedded in story events in all cases. The story title in all cases was produced in Arabic and included the child's name as the protagonist.

### 7.2.5 Image-Text Alignment

The image-text semantic alignment was evaluated as strong across generated images. The Flux Pro model demonstrated reliable adherence to scene-specific image prompts, correctly depicting described settings (home, garden, school, forest), actions, and emotional contexts.

### 7.2.6 Character Visual Consistency

The deterministic seed strategy and CHARACTER\_LOCK prompt constraints improved character consistency across multiple generated scenes. In most generated stories, the child protagonist preserved relatively stable facial appearance, hairstyle, skin tone, and clothing attributes throughout the narrative sequence. The consistency became stronger when a reference image was provided during generation, as the visual conditioning mechanism supplied the image-generation model with more reliable identity information.

However, minor inconsistencies occasionally appeared in some scenes, these inconsistencies typically involved slight hairstyle variation, clothing-detail changes, or altered facial proportions.



Figure 7.6. Consecutive generated scenes demonstrating relative character consistency across the multi-modal storytelling pipeline.

### 7.2.7 Art-Style Consistency

The supported visual styles including watercolor, cartoon, anime, pixel art, storybook, and paper-cut styles were generally preserved consistently throughout the generated stories. Style-specific characteristics such as brush textures, color saturation, line thickness, and shading behavior remained visually recognizable across most scenes.

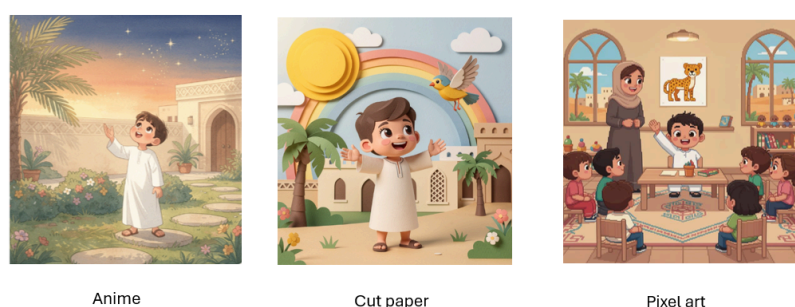


Figure 7.7. Comparison of generated illustrations across some supported visual styles within the Gheras platform.

### 7.2.8 Audio Narration Clarity

The generated narration audio was generally understandable and appropriately paced for children's storytelling scenarios. The use of Arabic diacritics within the narration text improved pronunciation

quality and reduced ambiguity during speech synthesis. Most generated narration maintained acceptable fluency and pronunciation clarity under standard Modern Standard Arabic vocabulary.

Pronunciation quality was strongest for short and medium-length narration segments containing simple vocabulary. Longer sentences or morphologically complex Arabic words occasionally produced less natural pronunciation or prosodic irregularities. Nevertheless, the generated audio remained sufficiently intelligible for child-oriented story narration.

The implemented caching mechanism also improved user experience during repeated story playback because previously generated narration audio could be reused instantly without additional synthesis latency.

## 7.3 Error Analysis

### 7.3.1 Known Issues

Although the implemented Gheras system remained functional during testing, several non-critical limitations were identified in the final system behavior.

First, the story-generation progress mechanism relies on periodic polling rather than WebSockets. This design is simple and stable, but it is not fully real-time; progress updates depend on the polling interval and may therefore lag slightly behind the actual backend generation state.

Second, the frontend generation page depends on navigation state to receive the story-generation payload. If the user refreshes the generation page before the story identifier is persisted in the URL or another durable location, the payload may be lost and the page redirects the user back to the story-creation form. A future improvement would be to store the story identifier in the URL or session storage. Finally, some error cases may still appear to the user as general failure messages rather than highly specific diagnostic feedback. Although the frontend displays backend error messages when available, failed generation, timeout, or unavailable external AI services may still require clearer user-facing explanations and retry options.

## 7.4 Comparison with Similar Systems

We compared Gheras with existing children’s storytelling platforms to evaluate its technical and personalization capabilities. The comparison focuses on the availability of AI-driven story generation, illustration support, behavioral targeting, cultural alignment, Arabic language support, personalization features, and additional educational functionalities. As shown in Table 7.3, existing storytelling systems primarily focus on entertainment and creative storytelling, while Gheras integrates behavioral guidance, Arabic linguistic support, and culturally aligned educational content within a unified AI-powered platform. Unlike other systems that offer generic behavioral guidance or limited linguistic support, Gheras integrates specialized behavioral tasks and educational content specifically engineered to resonate with Saudi and Islamic values. By combining AI-generated narratives with culturally-rooted illustrations and progress tracking.

Table 7.3. Feature comparison with existing platforms

| Feature              | Gheras              | StoryBee [8] | Tiny Teller AI [5] | StoryPark [7]     |
|----------------------|---------------------|--------------|--------------------|-------------------|
| AI text generation   | Yes                 | Yes          | Yes                | Yes               |
| AI illustrations     | Yes                 | Yes          | No                 | Partial           |
| Behavioral targeting | Yes                 | No           | No                 | Partial           |
| Cultural alignment   | Yes (Islamic/Saudi) | No           | No                 | Partial (General) |
| Arabic language      | Yes                 | Yes          | No                 | No                |

| Feature               | Gheras                 | StoryBee [8]                             | Tiny Teller AI [5]    | StoryPark [7]                              |
|-----------------------|------------------------|------------------------------------------|-----------------------|--------------------------------------------|
| Child personalization | Yes (name, appearance) | Yes                                      | Yes                   | Partial                                    |
| PDF export            | Yes                    | No                                       | No                    | No                                         |
| Progress tracking     | Yes                    | No                                       | Limited feedback only | Partial                                    |
| Behavior tasks        | Yes (Specific)         | No                                       | No                    | Partial                                    |
| Platform              | Web platform           | Web and Mobile Application (iOS/Android) | Software Application  | Web Platform and Educational Support Tools |

## 7.5 Discussion of Limitations

### 7.5.1 Dependence on External APIs

The AI Services layer is entirely dependent on three external commercial APIs: OpenAI (LLM and TTS) and BFL (Flux image generation). Any service outage, API version deprecation, or policy change by these providers directly affects system availability. The current implementation has no fallback to alternative providers in the event of primary API failure.

### 7.5.2 Scalability and Platform Limitations

One limitation is scalability. Python threading works fine for a few concurrent users, but it does not scale efficiently for large workloads. Each story-generation request may occupy a Python thread for more than two minutes during multimodal generation. With a large number of simultaneous users, the system could eventually exhaust the available worker threads. A more scalable solution would involve Celery with a dedicated message queue, which is specifically designed for long-running asynchronous tasks. In addition, the current platform is implemented as a responsive web application rather than a native mobile application. Although the frontend operates correctly on mobile browsers, a dedicated mobile application could provide additional capabilities such as push notifications, offline access, and improved mobile performance.

### 7.5.3 Offline Accessibility Limitation

Another limitation is platform accessibility. The current implementation does not support offline functionality, since story generation, media retrieval, and PDF generation all depend on active backend and external API communication. A service worker for caching completed stories could improve offline usability for previously generated content.

## 8 CONTRIBUTION AND IMPACT

This chapter discusses the principal contributions of the Gheras platform, its practical impact, and the lessons learned throughout the development and evaluation process.

### 8.1 Novel Features and Capabilities

Gheras introduces several novel technical and educational capabilities that distinguish it from existing AI storytelling systems. One of the platform's primary innovations is its support for culturally adaptive AI-generated storytelling specifically designed for Arabic-speaking children within Saudi and Islamic contexts. The platform generates stories in Modern Standard Arabic enhanced with diacritics (tashkeel), improving pronunciation clarity, reading comprehension, and early language acquisition for young learners.

Another distinctive capability of Gheras is its integration of behavioral guidance directly into the storytelling workflow. The platform generates stories according to predefined behavioral objectives such as honesty, kindness, patience, responsibility, and respect. In addition, the system reinforces these values by generating age-appropriate moral guidance and post-story behavioral tasks aligned with the selected educational objective.

The system also provides advanced personalization through profile-driven story generation. Stories are dynamically adapted according to the child's name, age group, gender, appearance characteristics, selected story genre, and illustration style. This personalization enhances emotional engagement by positioning the child as an active participant within the narrative environment.

From a technical perspective, Gheras integrates multiple AI services within a unified architecture, including large language models for story generation, AI-based illustration generation, and text-to-speech narration synthesis. This multimodal integration enables children to interact with stories through reading, listening, and visual learning simultaneously. Furthermore, the platform supports automatic PDF storybook generation, allowing stories to be downloaded, stored, and reused for offline educational activities.

#### 8.1.1 Improvements Over Existing Solutions

Gheras addresses several important limitations observed in existing storytelling platforms. Many current systems either lack Arabic language support entirely or provide only basic multilingual functionality without considering Arabic readability, pronunciation, or linguistic structure. Gheras improves upon these systems by generating stories in linguistically accurate Arabic enhanced with diacritics, making the content more accessible for children in early reading stages.

Existing storytelling systems also primarily focus on entertainment and creative engagement rather than structured behavioral intervention. In contrast, Gheras incorporates behavioral targeting into the story generation process itself, enabling the system to generate narratives designed to address undesirable behaviors while reinforcing positive moral and social values.

Another major improvement involves cultural and religious contextualization. Most existing AI storytelling systems rely on generalized Western-oriented narratives and visual representations, limiting their suitability for children from Arab and Islamic communities. Gheras addresses this limitation by incorporating Saudi cultural elements, Islamic ethical values, and culturally appropriate visual representations into both the narrative and illustration generation processes.

Additionally, existing systems often provide limited parental involvement and monitoring capabilities. Gheras improves usability by integrating progress tracking mechanisms that allow parents to monitor generated stories, reading activity, completed behavioral tasks, and child engagement over time. This transforms storytelling from a standalone entertainment activity into a measurable educational and developmental process.

## 8.2 Practical and Industrial Relevance

There's a real market need for Arabic-language children's educational content, especially in Saudi Arabia where the government is investing heavily in education through Vision 2030. Most educational tech products in the region are translations of English-language tools, and they often miss cultural nuances.

Gheras could be relevant for:

- Saudi families who want culturally appropriate stories for their children
- Schools and kindergartens looking for behavioral development tools
- Child psychologists who want to use storytelling as a therapeutic technique
- Publishing companies interested in AI-generated children's content in Arabic

The platform could also be adapted for other Arabic-speaking countries with different cultural preferences.

## 8.3 Lessons Learned

Throughout the development of the Gheras platform, the team gained valuable technical and project-management experience related to full-stack software engineering, multimodal AI integration, asynchronous systems, and collaborative project execution. The implementation process provided practical insight into the challenges of combining frontend technologies, backend architecture, databases, and external AI services into a single real-time interactive platform. In addition to technical development skills, the project also reinforced the importance of planning, task coordination, iterative testing, and structured teamwork throughout the project lifecycle.

### 8.3.1 Technical Lessons

- We designed the REST API first, then built the frontend around it. This made integration smoother and helped identify architectural and data-flow problems early in development.
- The adapter pattern used for AI generation kept the system modular. The frontend remains independent from the internal AI-generation logic and only interacts with backend status endpoints and generated results.
- The development process highlighted several important lessons regarding multimodal AI-system integration. One major observation was that image generation constituted the primary runtime bottleneck within the pipeline, making asynchronous orchestration and progressive loading essential for maintaining acceptable user experience.
- Another important lesson involved the significance of prompt engineering for multimodal consistency. Character appearance, narrative coherence, and image-text alignment were highly sensitive to prompt structure, visual constraints, and repeated identity conditioning. Small prompt modifications frequently produced noticeable changes in generated outputs.
- The project also demonstrated the challenges associated with Arabic AI generation, particularly regarding diacritics consistency and TTS pronunciation quality. The interaction between the language model and the TTS system showed that narration quality depended strongly on the correctness of generated Arabic text.
- The implementation process reinforced the importance of asynchronous coordination, defensive error handling, and modular architecture when integrating multiple external AI services into a single real-time application pipeline.
- Python threading was fast and practical for the prototype implementation, but it revealed scalability limitations for long-running AI-generation workloads. For future large-scale deployment, Celery and Redis would provide a more robust task-management solution.
- Building a fully Arabic RTL interface was more challenging than initially expected. The frontend design process required reconsidering many layout assumptions, navigation directions, and compo-

ment alignments. Tailwind CSS logical properties such as start and end instead of left and right significantly simplified RTL support.

- Storing behaviors, genres, and visual styles in the database rather than hardcoding them improved maintainability and flexibility, allowing content options to be modified without redeploying the system.

### ***8.3.2 Project Management Lessons***

- The project tasks were divided according to specialization areas. Maha focused primarily on backend development, while Sara focused on frontend implementation and interface design.
- Amjad and Fatima focused on AI services integration, orchestration logic, and integration between the AI pipeline, backend, and frontend components.
- The clear separation between frontend and backend responsibilities through the REST API architecture significantly improved collaboration efficiency and reduced integration conflicts between team members.
- Weekly milestones helped maintain steady project progress. Each development cycle focused on implementing a small set of features incrementally rather than attempting to complete all functionality simultaneously.
- Continuous testing throughout development proved essential. Delaying testing until the end of implementation phases occasionally resulted in bugs that became more difficult to debug and fix under time pressure.
- The project also highlighted the importance of communication and coordination when integrating independently developed components into a unified system. Frequent synchronization between frontend, backend, and AI-service development tasks was necessary to maintain compatibility across the complete pipeline.

## 9 CONCLUSION AND FUTURE WORK

This chapter summarizes the project outcomes, evaluates achievement of the original objectives, and discusses future directions for extending and scaling the Gheras platform.

### 9.1 Project Outcomes

After the implementation and testing phases, the Gheras project successfully achieved the following outcomes:

- Development of a complete multimodal AI storytelling platform integrating frontend, backend, database, and AI-generation components into a unified Arabic storytelling system.
- Implementation of a React-based frontend, Django REST backend, PostgreSQL database, and modular AI services layer for text generation, image synthesis, and narration audio generation.
- Development of a backend containing sixteen REST API endpoints supporting authentication, CRUD operations, child-profile management, story generation, progress tracking, file downloads, reading analytics, behavior tracking, and statistical features.
- Implementation of seven relational database models with validated relationships and integrity constraints for managing users, children, stories, story pages, reading logs, behaviors, and related metadata.
- Development of a responsive Arabic right-to-left (RTL) frontend containing ten pages, more than eight reusable UI components, and a centralized routing structure.
- Implementation of user authentication, child-profile creation, multi-step story-generation forms, asynchronous progress tracking, multimodal story viewing, narration playback, and downloadable illustrated PDF export.
- Integration of OpenAI language and TTS models with the FLUX image-generation API through an asynchronous orchestration pipeline implemented by StoryOrchestrator.
- Implementation of an AI adapter architecture that separates AI-generation logic from the remaining backend application logic, improving modularity and maintainability.
- Support for age-adaptive story generation, multiple visual styles, deterministic character-conditioning strategies, reference-image conditioning, progressive multimedia loading, synchronized narration audio generation, and error-handling mechanisms.
- Achievement of average response times below approximately 200 ms for standard read-oriented API endpoints under normal testing conditions.
- Achievement of an average PDF generation time of approximately 300 ms depending on story size and image count.
- Maintenance of a frontend production build size below approximately 400 KB in compressed form, contributing to efficient frontend loading behavior.
- Successful handling of concurrent story-generation requests during testing without observed synchronization failures between independently generated story objects.
- Implementation of behavioral-progress visualization features, including behavior-distribution statistics and weekly reading-activity tracking through interactive frontend analytics components.
- Successful generation of personalized Arabic stories containing coherent narration text, AI-generated illustrations, and narration audio across multiple age groups, visual styles, and behavioral themes.
- Demonstration of the feasibility of integrating multimodal generative AI technologies into a real-time interactive Arabic storytelling platform.

### 9.2 Achievement of Objectives

The objectives established during Graduation Project I were successfully achieved through the design, implementation, and evaluation of the Gheras platform. The project successfully delivered a complete AI-powered web platform capable of generating personalized Arabic children's stories that combine



educational storytelling, behavioral guidance, image generation, and audio narration within a unified interactive environment.

One of the primary objectives of the project was to develop a culturally and religiously appropriate storytelling platform tailored for Arabic-speaking children in Saudi society. This objective was achieved by designing story-generation workflows that incorporate Saudi cultural elements, Islamic values, and simplified Modern Standard Arabic suitable for children aged 4–9. The generated stories are customized according to the child’s profile, including age group, gender, preferences, and targeted behavioral trait, ensuring both educational relevance and emotional engagement.

Another major objective was to integrate multiple artificial intelligence technologies into a single coordinated pipeline. The implemented system successfully combines Large Language Models for narrative generation, text-to-image models for illustration synthesis, and text-to-speech services for audio narration generation. These AI components operate through a modular orchestration architecture that enables automated generation of complete multimedia stories consisting of text, illustrations, and narration audio.

The project also aimed to provide high levels of personalization and behavioral customization. This objective was fulfilled through the implementation of child profile management, story preference selection, behavioral targeting, and customizable illustration styles. Parents can generate stories focused on specific behavioral issues such as honesty, kindness, empathy, or responsibility while adapting the visual and narrative style to the child’s interests and developmental level.

In addition, the system successfully achieved the objective of supporting parent involvement and monitoring through a progress-tracking dashboard. The implemented platform records reading activities, story interactions, and completion status, enabling parents to monitor engagement and behavioral development over time. This transformed the platform from a simple storytelling application into an interactive educational and behavioral support system.

From a technical perspective, the project objectives related to software engineering and system integration were also accomplished. The final implementation included a React-based frontend, Django REST backend, PostgreSQL database, and AI services layer connected through RESTful APIs. The platform successfully supports authentication, child management, story generation, PDF downloading, and analytics functionalities through a complete modular architecture.

Finally, the project achieved its usability and accessibility objectives by providing a responsive web-based platform with an intuitive interface suitable for both parents and children. Features such as read-aloud narration, illustrated story pages, and downloadable PDF stories enhanced user interaction and accessibility while maintaining a child-friendly experience. Overall, the implemented system successfully fulfilled the functional, technical, educational, and cultural objectives defined during the initial project planning phase.

Table 9.1 summarizes the main objectives established during Graduation Project I and how they were achieved through the implementation of the Gheras platform.

Table 9.1. Achievement of Project Objectives

| Objective                                          | How It Was Achieved                                                                                                                         |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Develop an AI-powered Arabic storytelling platform | Implemented a complete multimodal web platform for generating personalized Arabic children’s stories with illustrations and narration audio |
| Provide culturally appropriate Arabic storytelling | Integrated Saudi cultural elements, Islamic values, and simplified Modern Standard Arabic suitable for children aged 4–9                    |

| Objective                                                | How It Was Achieved                                                                                                         |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Personalize stories for each child                       | Child profiles, age groups, gender, preferences, behavioral traits, and visual styles were integrated into story generation |
| Integrate multiple AI technologies                       | Combined LLM-based story generation, FLUX image synthesis, and TTS narration within a unified orchestration pipeline        |
| Support behavioral guidance and educational storytelling | Implemented targeted behavioral themes such as honesty, kindness, empathy, and responsibility                               |
| Enable parent monitoring and engagement tracking         | Developed reading logs, story-interaction tracking, completion monitoring, and behavioral analytics dashboard               |
| Implement a complete full-stack architecture             | Developed React frontend, Django REST backend, PostgreSQL database, and RESTful AI-services integration                     |
| Support multimedia story generation                      | Generated synchronized narration text, illustrations, narration audio, and downloadable PDF stories                         |
| Provide responsive and accessible user experience        | Designed a responsive RTL web interface with illustrated pages, narration playback, and mobile-browser support              |
| Support authentication and child management              | Implemented authentication, child-profile management, story generation, analytics, and PDF-download functionalities         |

Summary of the primary project objectives and their implementation outcomes in the Gheras platform.

## 9.3 Future Extensions

### 9.3.1 Backend Scalability and Task Management

- Replace Python threading with Celery and Redis. This would provide:
  - Proper job queuing with retries and error handling
  - Scalability through independently managed worker processes
  - Task monitoring and management capabilities
  - Scheduled background tasks such as weekly reading reminders

### 9.3.2 Mobile Application Development

- Develop a mobile application using React Native or Flutter to provide:
  - Push notifications when a story is ready
  - Offline story reading through downloadable stories
  - Improved performance and user experience on mobile devices
  - Access to device features such as the camera for child photos

### ***9.3.3 Story and Content Enhancements***

- Allow parents to generate sequels for existing stories using the same characters
- Allow parents to select from curated story structures such as adventure, mystery, and friendship
- Introduce a co-creation mode where parents can edit or extend AI-generated stories before finalization
- Provide shareable story links for grandparents, relatives, or friends
- Integrate voice-selection options for narration customization

### ***9.3.4 Multilingual and Adaptive Educational Support***

- Support story generation in additional languages such as English and French while maintaining appropriate cultural and religious customization for each language
- Support multilingual narration alongside multilingual story generation
- Adapt vocabulary difficulty according to the child's language-development level in addition to age-group classification
- Introduce language-assessment tests during child-profile creation to estimate linguistic comprehension and personalize story complexity accordingly

### ***9.3.5 Advanced Analytics and Behavioral Insights***

- Provide detailed reading analytics such as time spent per page and most frequently re-read stories
- Generate behavioral recommendations suggesting which behavior to target next based on the child's reading history
- Provide monthly parent reports summarizing the child's reading progress and engagement journey
- Develop an advanced dashboard allowing parents and teachers to input detailed behavioral observations and monitor long-term progress trends
- Allow parents to customize schedules for automatic story delivery and reading reminders

### ***9.3.6 Educational and Behavioral Collaboration***

- Collaborate with educational and behavioral specialists to improve the educational quality, behavioral effectiveness, and developmental appropriateness of generated stories
- Incorporate expert-guided behavioral frameworks and child-development recommendations into the story-generation process

### ***9.3.7 Interactive Voice-Based Story Engagement***

- Introduce conversational AI interactions enabling children to ask questions about generated stories and characters in real time

## **9.4 Deployment, Scalability, and Sustainability**

### ***9.4.1 Deployment***

The system is configured to support production-oriented deployment through environment-based configuration management, where sensitive settings such as SECRET\_KEY, ALLOWED\_HOSTS, CORS origins, and API credentials are loaded from environment variables rather than hardcoded values.

The backend architecture supports deployment using Gunicorn as the WSGI server together with WhiteNoise for static-file serving without requiring a separate static-file web server. The implementation also includes production-oriented security configurations such as XSS filtering, content-type protection, and frame restrictions.

PostgreSQL is used as the primary relational database, while Docker was used during development for PostgreSQL container management and environment isolation. A typical deployment scenario could involve hosting the Django backend on a VPS or cloud platform such as AWS or Azure, while the frontend production build could be distributed through a CDN or static hosting service.

### 9.4.2 Scalability

Although the current implementation operates successfully for moderate workloads, several improvements would be required for large-scale deployment. The current background-generation workflow relies on Python threading and asynchronous orchestration, which may become inefficient under very high concurrent workloads.

Future scalability improvements could include:

- migrating long-running generation tasks to Celery with Redis,
- introducing Redis-based caching for frequently accessed endpoints,
- using CDN distribution for generated media assets,
- introducing object storage such as Amazon S3 for generated images and child photos,
- and scaling AI-generation workloads independently through distributed or serverless execution architectures.

Database scalability could also be improved in future deployments through query optimization and read-replica architectures if platform usage increases substantially.

### 9.4.3 Sustainability

The system architecture was designed with maintainability and extensibility in mind. The codebase applies clear separation between frontend, backend, database, and AI services layers, reducing coupling between components and simplifying future modification.

The modular AI adapter architecture also allows AI providers and models to be replaced or upgraded independently as newer or more cost-efficient models become available. In addition, the normalized database structure supports future feature expansion without requiring major architectural redesign.

Finally, the use of environment-based configuration management enables the same codebase to operate across development, testing, and production environments with minimal modification, improving long-term maintainability and deployment flexibility.

## 9.5 Conclusion

The development of Gheras demonstrated the feasibility of integrating generative artificial intelligence technologies into a unified platform for personalized Arabic children's storytelling. The implemented platform combined story generation, illustrations, audio narration, and behavioral guidance within a single interactive environment tailored to children aged 4–9. By incorporating personalization, cultural adaptation, and multimodal interaction, the platform provided a more engaging and contextually relevant storytelling experience for both children and parents.

The implementation process also highlighted the importance of modular architecture in AI-driven applications. The integration of frontend components, backend services, databases, and external AI models enabled the platform to operate cohesively while supporting scalability and maintainability. Testing and evaluation showed that the platform was capable of generating coherent personalized stories with acceptable usability and performance. However, several practical challenges were identified, including generation latency, and maintaining accurate Arabic diacritization.

Overall, the achieved results demonstrate the potential of combining generative AI with culturally adaptive storytelling to support behavioral and educational objectives for Arabic-speaking children. The completed platform establishes a strong foundation for future improvements and further development of AI-based child-centered educational systems.

## REFERENCES

- [1] م. نصير, "فاعلية القصة في تعديل سلوك طفل ما قبل المدرسة," *مجلة العلوم التربوية و النفسية*, vol. 3, no. 24, pp. 150–163, Oct. 2019, doi: 10.26389/AJSRP.T100619.
- [2] !. منصور, "دور القصة في تنمية المهارات اللغوية لدى طفل الروضة-رياض الأطفال بسكرة أنموذجاً," *جامعة محمد خيضر*, 2019. بسكرة, الجزائر. [Online]. Available: <http://archives.univ-biskra.dz/bitstream/123456789/17485/1/>
- [3] N. Zaklas and T. Gazit, "The Experience of Using Ai-Generated Social Stories as a Behavioral Intervention Tool for Children," *SSRN*, 2025, doi: 10.2139/ssrn.5276473.
- [4] J. Chen, "Enjoyment of AI-generated stories blending art and science: impact on preschoolers' pro-environmental attitudes," *Frontiers in Psychology*, vol. 16, p. 1579510, July 2025, doi: 10.3389/fpsyg.2025.1579510.
- [5] M. Kim, T. Kim, A. Nguyen, E. N. Gomez, and J. Jin, "TinyTeller AI, an AI-based Adaptive Storytelling Application," in *2024 Artificial Intelligence x Humanities, Education, and Art (AIxHEART)*, IEEE, Sept. 2024, pp. 44–51. doi: 10.1109/AIxHeart62327.2024.00017.
- [6] Merjovaara, T. Nousiainen, L. Turja, and S. Isotalo, "Digital Stories with Children: Examining Digital Storytelling as a Pedagogical Process in ECEC," *Journal of Early Childhood Education Research*, vol. 9, no. 1, pp. 99–123, 2020.
- [7] L. Ye, J. Jiang, D. Chang, and P. Liu, "Storypark: Leveraging large language models to enhance children story learning through child-ai collaboration storytelling," *arXiv preprint arXiv:2405.06495*, 2024, [Online]. Available: <https://arxiv.org/html/2405.06495v1>
- [8] "Storybee." [Online]. Available: <https://storybee.app/>
- [9] M. T. Hidayat, "Effectiveness of AI-Based Personalised Reading Platforms in Enhancing Reading Comprehension.," *Journal of Learning and Development*, vol. 11, no. 1, pp. 115–125, 2024.
- [10] A. Celikyilmaz, E. Clark, and J. Gao, "Evaluation of text generation: A survey," *arXiv:2006.14799*, 2020.
- [11] S. Alotaibi, "Exploring GPT-4 capabilities in generating paraphrased sentences for the Arabic language," *Applied Sciences*, vol. 15, no. 8, 2025.
- [12] M. AlKhamissi, E. Abu Farha, and W. Magdy, "Taqyim: Evaluating Arabic NLP tasks using ChatGPT models," *arXiv:2306.16322*, 2023.
- [13] Inception Institute of Artificial Intelligence, "Jais and Jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models," *arXiv:2308.16149*, 2023.
- [14] Qwen Team, "Qwen3 technical report," *arXiv:2505.09388*, 2025.
- [15] OpenAI, "GPT-4o mini documentation."
- [16] Hugging Face, "Model cards for Jais-2-8B-Chat, LFM2.5-1.2B-Instruct, and Qwen3-Coder-Next."
- [17] Liquid AI, "LFM2.5-1.2B-Instruct model card."
- [18] B. J. Richards, "Type/token ratios: What do they really tell us?," *Journal of Child Language*, vol. 14, no. 2, pp. 201–209, 1987.
- [19] T. Inspector, "Lexical diversity and language assessment: Type-token ratio measures," 2013.
- [20] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proc. ICLR*, 2020.
- [21] L. Zheng, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Proc. NeurIPS Datasets and Benchmarks*, 2023.
- [22] C.-H. Chiang and H.-Y. Lee, "Can large language models be an alternative to human evaluations?," *arXiv:2305.01937*, 2023.
- [23] T. Kocmi and C. Federmann, "Large language models are state-of-the-art evaluators of translation quality," in *Proc. EAMT*, 2023.
- [24] Black Forest Labs, "FLUX.1 [schnell] model card."
- [25] Black Forest Labs, "FLUX.2 documentation and overview."
- [26] OpenAI, "GPT-Image-1-mini model documentation."

- [27] Tongyi-MAI, “Z-Image-Turbo model card.”
- [28] F. Carlsson, “Multilingual and multimodal vector representations with M-CLIP,” *arXiv:2202.04304*, 2022.
- [29] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, “DreamBooth: Fine tuning text-to-image diffusion models for subject-driven generation,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 22500–22510.
- [30] C. Schuhmann, “LAION-5B: An open large-scale dataset for training next generation image-text models,” in *Proc. NeurIPS Datasets and Benchmarks*, 2022.
- [31] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Proc. NeurIPS*, 2023.
- [32] Y. Jia, “Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions,” in *Proc. IEEE ICASSP*, 2018, pp. 4779–4783.
- [33] J. Shen, “Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions,” *arXiv:1712.05884*, 2017.
- [34] H. Zhu *et al.*, “OmniVoice: Towards omnilingual zero-shot text-to-speech with diffusion language models,” *arXiv preprint arXiv:2604.00688*, 2026, doi: 10.48550/arXiv.2604.00688.
- [35] OpenAI, “GPT-4o-mini-TTS model documentation.”
- [36] C.-C. Lo *et al.*, “MOSNet: Deep learning-based objective assessment for voice conversion,” in *Proc. Interspeech*, 2019, pp. 1541–1545.
- [37] G. Mittag, B. Naderi, M.-A. Chehadi, and S. Moller, “NISQA: A deep CNN-self-attention model for multidimensional speech quality prediction with crowdsourced datasets,” in *Proc. Interspeech*, 2021, pp. 2127–2131.
- [38] ITU-T, “Methods for subjective determination of transmission quality,” International Telecommunication Union, Recommendation P.800, 1996.
- [39] M. Al-Haj and J. Manar, “Automatic speech recognition of Arabic phonemes: Challenges and advances,” in *Proc. IEEE ICCAIE*, 2014, pp. 203–208.
- [40] A. Triantafyllopoulos, J. Poppe, and B. Schuller, “An overview of affective speech synthesis and conversion in the deep learning era,” *arXiv:2210.14065*, 2022.

# APPENDICES

## Project Links

- **Project Code Link:** [h](#)
- **Demo Video Link:** [hti](#)